

UNIVERSITÀ POLITECNICA DELLE MARCHE

LABORATORIO DI AUTOMAZIONE

CORSO DI LAUREA TRIENNALE IN INGEGNERIA
INFORMATICA E DELL'AUTOMAZIONE



Ducted Fan

**Sviluppo della telemetria, identificazione del modello e
simulazione del Double Propeller Ducted-Fan**

Relatori:

Prof. Bonci Andrea

Dott. di Biase Alessandro

Tesina di:

Di Tizio Nicolas

Marcucci Simone

Morresi Tommaso

Nalli Alberto

Anno Accademico 2025/2026



Dipartimento di Ingegneria dell'Informazione

Indice

Introduzione	3
1 Sistema	6
1.1 Double Propeller Ducted-Fan. Struttura	6
1.2 Controllo del Double Propeller Ducted-Fan	8
2 Hardware	9
2.1 STM32 NUCLEO-H745ZI-Q. La scheda di controllo	9
2.1.1 STM32H745ZI-TQ6. Architettura	10
2.1.2 STM32H745ZI-TQ6. Timer	10
2.1.3 STM32H745ZI-TQ6. Periferiche di comunicazione	10
2.1.4 STM32H745ZI-TQ6. Eventi asincroni	10
2.1.5 NUCLEO-H745ZI-Q. Caratteristiche della scheda	11
2.2 TATTU LiPo. Batteria ricaricabile	12
2.2.1 TATTU LiPo. Caratteristiche fisiche	13
2.2.2 TATTU LiPo. Caratteristiche tecniche e considerazioni operative . . .	13
2.3 PowerSafe Twin ADV. MNR-electronics	14
2.3.1 PowerSafe Twin ADV. Caratteristiche fisiche e tecniche	15
2.4 DFRobot Power Module. Convertitore di tensione	15
2.5 Power Distribution Board	17
2.6 Turnigy AereoDrive SK3-3536 1400KV	17
2.6.1 Turnigy AereoDrive SK3-3536 1400 KV. Caratteristiche fisiche e tecniche	18
2.7 Turnigy Plush 40A. Electronic Speed Controller	19
2.7.1 Turnigy Plush 40A. Caratteristiche fisiche	19
2.7.2 Turnigy Plush 40A. Risoluzione e controllo dei motori	19
2.8 APC 9x4.5. Eliche	20
2.8.1 Caratteristiche fisiche	20
2.9 Hitec HS-82MG Gear micro servo	21
2.9.1 HS-82MG Hitec. Caratteristiche fisiche	21
2.9.2 HS-82MG Hitec. Caratteristiche tecniche e controllo	23
2.10 BNO055 Bosch Sensortec. Unità di misura inerziale e magnetometro	24
2.10.1 BNO055 Bosch Sensortec. Caratteristiche fisiche	24
2.10.2 BNO055 Bosch Sensortec. Infrastruttura di alimentazione, comunica- zione ed integrazione del sensore	24
2.10.3 BNO055 Bosch Sensortec. Sensori inerziali e sensor fusion	25
2.10.4 BNO055 Bosch Sensortec. Modalità operative	27
2.11 VL53L1X ST. Sensore Time-of-Flight per la misurazione a lunga distanza . .	28
2.11.1 Il principio di misura di un sensore Time-of-Flight (ToF)	28
2.11.2 VL53L1X STMicroelectronics. Caratteristiche fisiche	28
2.11.3 VL53L1X STMicroelectronics. Caratteristiche tecniche	29

2.12	Schema dei collegamenti	31
2.12.1	Tabelle dei collegamenti	32
3	Firmware	35
3.1	Architettura software event-driven	35
3.2	Gestione del microcontrollore dual-core	36
3.3	Organizzazione modulare del codice	36
3.4	Configurazione delle periferiche	38
3.4.1	Timer	38
3.4.2	Periferiche di comunicazione	39
3.5	Gestione dei sensori	39
3.5.1	VL53L1X — Acquisizione di quota	39
3.5.2	BNO055 — Acquisizione dell'assetto	40
3.6	Diagrammi di flusso	40
3.6.1	Diagramma di flusso del VL53L1X	41
3.6.2	Diagramma di flusso del BNO055	43
3.6.3	Diagramma di flusso del firmware complessivo	44
3.7	Protocollo di comunicazione seriale con MATLAB	46
3.7.1	Comandi ricevuti dal firmware	46
3.7.2	ACK, watchdog e diagnostica UART	47
3.8	Telemetria robusta a frequenza fissa	48
3.9	Safe startup e logiche di arresto	51
4	Telemetria	52
4.1	Ruolo della telemetria nel progetto	52
4.2	Architettura del canale di comunicazione	52
4.3	Parametrizzazione dello script di acquisizione	53
4.4	Protocollo di scambio dati	54
4.4.1	Flusso di <i>uplink</i> : comandi	54
4.4.2	Flusso di <i>downlink</i> : telemetria	55
4.5	Watchdog firmware e keep-alive lato MATLAB	56
4.6	Loop di acquisizione MATLAB	56
4.6.1	Scheduling anti-burst	57
4.6.2	Parsing CSV e validazione delle righe	57
4.6.3	Terminazione asincrona di emergenza	58
4.6.4	Chiusura controllata con onCleanup	59
4.7	Profili di eccitazione	59
4.7.1	Segnali a gradino	61
4.7.2	Segnali armonici e chirp	61
4.7.3	Sequenze pseudo-casuali multilivello	62
4.7.4	Eccitazioni multi-asse	63
4.8	Validazione del link telematico	63

5	Identificazione del modello	65
5.1	Approccio data-driven	65
5.2	Pre-elaborazione dei dati	65
5.3	Selezione dell'architettura del modello	67
5.4	Stima del modello e validazione	68
5.4.1	Analisi del guadagno statico	70
5.5	Il regolatore PID: principi di funzionamento	70
5.5.1	Le tre azioni di controllo	71
5.5.2	Implementazione discreta in forma posizionale	72
5.6	Sintesi del controllore PIDF	72
5.6.1	Architettura del controllore	72
5.6.2	Sintesi tramite pidtune	73
5.6.3	Discretizzazione per il firmware	75
5.7	Guadagni finali e parametri PID	75
5.8	Considerazioni sull'approccio black-box	76
6	Simulazione 3D	77
6.1	Obiettivi e architettura del simulatore	77
6.1.1	Architettura a doppio loop temporale	77
6.1.2	Interfaccia interattiva	78
6.2	Parametrizzazione fisica e geometrica	78
6.3	Modello dinamico del corpo rigido	79
6.3.1	Momento ribaltante per eccentricità del baricentro	79
6.3.2	Momenti aerodinamici dei flap	79
6.3.3	Modello di spinta calibrato sui dati reali	80
6.4	Modello visco-elastico delle funi	80
6.5	Calibrazione empirica dei parametri	82
6.6	Risultati della simulazione	82
6.7	De-rating dei guadagni per il transfer al prototipo fisico	84
6.8	Limiti del simulatore e relazione con i capitoli successivi	85
7	Conclusioni e Sviluppi Futuri	86
7.1	Analisi critica delle cause di mancato volo libero	86
7.1.1	Verifica analitica del deficit di spinta	86
7.1.2	Distribuzione delle masse ed elevata inerzia perimetrale	89
7.1.3	Eccentricità del baricentro e momento ribaltante gravitazionale	89
7.1.4	Deficit di autorità di controllo dei flap	89
7.1.5	L'effetto mascheramento del vincolo	90
7.2	Sviluppi futuri e linee guida per la riprogettazione	90
7.2.1	Riprogettazione del sistema propulsivo	90
7.2.2	Centratura e accostamento delle masse	91
7.2.3	Aumento dell'autorità di controllo	91
7.2.4	Riduzione del peso strutturale	92
7.3	Considerazioni conclusive	92

Introduzione

Il presente documento illustra le attività di sviluppo, caratterizzazione e controllo svolte sul prototipo *Double Propeller Ducted-Fan (DPDF)*, un velivolo a propulsione intubata controrotante. Il lavoro ha preso avvio da una necessaria fase di rigenerazione tecnica del prototipo: l'intera struttura meccanica è stata riprogettata e ristampata in 3D, mentre l'elettronica di bordo e i cablaggi sono stati integralmente riassemblati e collaudati.

Il contributo originale del presente lavoro si articola attorno a tre assi principali. Il primo è la realizzazione di un'**infrastruttura di telemetria** dedicata: il firmware implementato su STM32 NUCLEO-H745ZI-Q trasmette a 50 [Hz], su porta seriale a 230 400 [baud], un flusso CSV continuo che include angoli di assetto, quota, comandi motori e servo, contatori di errore UART e *flag* di stato. Lo *script* MATLAB `AcquiringData.m` gestisce lato PC la ricezione, l'archiviazione su file e la generazione in tempo reale dei segnali di eccitazione, rendendo possibile l'acquisizione di *dataset* strutturati senza interruzioni manuali.

Il secondo asse è l'**identificazione di sistema**. I *dataset* acquisiti durante sessioni di eccitazione PRBS e a gradino sono stati elaborati in MATLAB tramite la funzione `tfest` del *System Identification Toolbox*, che ha permesso di stimare le funzioni di trasferimento a tempo discreto della risposta di *roll* e *pitch* del velivolo rispetto all'angolo imposto ai rispettivi *flap*. I modelli identificati sono stati utilizzati per la sintesi automatica dei regolatori PIDF tramite `pidtune` e per la verifica della risposta in anello chiuso in simulazione.

Il terzo asse è lo sviluppo di un **simulatore fisico tridimensionale** in MATLAB (`Sim.m`). Il simulatore riproduce la dinamica 6-DOF del *DPDF* vincolato nella configurazione a corde adottata in laboratorio, incorporando modelli di spinta calibrati sui dati reali, un modello elastico delle quattro corde di vincolo, l'azione aerodinamica dei *flap* e quattro regolatori PID interattivi. Lo strumento ha permesso di studiare le interazioni tra gli assi di controllo e di verificare la coerenza dei parametri fisici stimati rispetto al comportamento osservato nella telemetria.

È necessario anticipare una limitazione hardware strutturale del prototipo. L'analisi della spinta erogabile dai due propulsori nella configurazione attuale — motori *Turnigy SK3-3536 1400 KV* con eliche *APC 9×4.5* alimentati a 14.8 [V] — evidenzia una spinta totale massima di circa $13 \div 14$ [N], inferiore alla forza peso del sistema assemblato ($m \approx 2.2$ [kg], $F_p \approx 21.6$ [N]). Tale deficit preclude il decollo in configurazione libera ed è la ragione per cui tutti i test sperimentali descritti in questo documento sono stati condotti con il drone vincolato all'interno della gabbia di sicurezza. Questa condizione non pregiudica la validità del percorso di identificazione e simulazione: i modelli stimati descrivono la risposta angolare del sistema nella sua configurazione di test reale, che costituisce il contesto sperimentale di riferimento dell'intero lavoro.

In luce degli obiettivi esposti, la trattazione è stata articolata in sei capitoli:

- Nel **Capitolo 1** viene introdotta l'architettura meccanica del *DPDF*, descrivendo i criteri di ricostruzione della struttura e l'approccio di controllo adottato nel progetto.
- Nel **Capitolo 2** vengono richiamate le principali componenti hardware e i relativi collegamenti elettrici, con un focus sulle specifiche tecniche che influenzano lo sviluppo del firmware e la precisione della telemetria.

- Nel **Capitolo 3** viene descritta l'architettura del firmware implementato su STM32 NUCLEO-H745ZI-Q: struttura modulare del codice, *loop* di controllo, gestione della sicurezza e protocollo di comunicazione seriale con MATLAB.
- Nel **Capitolo 4** viene illustrata l'infrastruttura di telemetria: il protocollo di trasmissione, la ricezione e il salvataggio in MATLAB, le prove di eccitazione eseguite e i *dataset* raccolti.
- Nel **Capitolo 5** viene presentata la fase di identificazione di sistema: le sequenze di eccitazione impiegate, la stima delle funzioni di trasferimento di *roll* e *pitch* tramite *tfest*, la sintesi dei regolatori tramite *piddtune* e la verifica in anello chiuso.
- Nel **Capitolo 6** viene descritto il simulatore fisico tridimensionale: il modello dinamico adottato, la parametrizzazione delle forze fisiche, il modello delle corde di vincolo e il confronto qualitativo con i dati di telemetria reali.
- Nel **Capitolo 7** vengono presentate le conclusioni del lavoro: un'analisi critica delle cause che hanno impedito il volo libero del prototipo — distribuzione delle masse, eccentricità del baricentro, deficit di autorità di controllo dei *flap* e effetto mascheramento del vincolo — e le linee guida per la riprogettazione e i possibili sviluppi futuri.

1 Sistema

Nel presente capitolo viene descritta la struttura meccanica del *Double Propeller Ducted-Fan (DPDF)* e viene introdotto l'approccio di controllo adottato nel progetto.

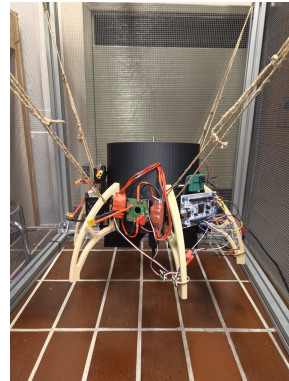
1.1 Double Propeller Ducted-Fan. Struttura

Il prototipo del *Double Propeller Ducted Fan (DPDF)* si sviluppa attorno a una struttura cilindrica cava alta **220 [mm]**, con un diametro esterno di **235 [mm]** e uno interno di **230 [mm]**, progettata per alloggiare al proprio interno tutte le componenti necessarie al volo. Questa scelta costruttiva, che prevede il montaggio interno del sistema propulsivo, rappresenta il principale tratto distintivo del progetto rispetto alle architetture dei droni convenzionali. Racchiudere le eliche all'interno della struttura portante offre infatti un notevole vantaggio in termini di sicurezza: le parti rotanti non sono esposte, riducendo drasticamente i pericoli in caso di contatto fisico accidentale sia con l'operatore umano sia con l'ambiente circostante.

Analizzando la configurazione interna, nella porzione superiore del cilindro trovano posto due motori in configurazione controrotante, saldamente ancorati a una struttura a croce che funge sia da base di supporto sia da rinforzo contro le sollecitazioni meccaniche. La disposizione meccanica del sistema è illustrata in Figura 1.1.



(a) Struttura di sostegno dei motori



(b) Vista frontale

Figura 1.1: *Struttura del ducted fan*

Dal punto di vista teorico, l'adozione di rotori controrotanti permette di generare spinta verticale limitando al contempo la rotazione del drone attorno all'asse Z, grazie ai momenti torcenti opposti prodotti dalle eliche. Tuttavia, i test sperimentali hanno evidenziato che questa compensazione passiva, per quanto utile, non è sufficiente a stabilizzare in modo del tutto autonomo la dinamica di imbardata (*yaw*), rendendo imprescindibile l'implementazione di un'azione di controllo dedicata.

Un'ulteriore criticità legata al *design* strutturale del *DPDF* è la sua naturale suscettibilità

a rotazioni indesiderate sul piano definito dagli assi X e Y. Tali perturbazioni rischiano di compromettere la stabilità del sistema, potendo causare una completa perdita di controllo del mezzo. Per limitare questo fenomeno, la parte inferiore del drone è stata equipaggiata con due **flap** direzionali: uno dedicato al controllo dell'assetto lungo l'asse delle ascisse e l'altro lungo l'asse delle ordinate, disposti in modo tale da risultare perpendicolari tra loro.

Questi *flap* agiscono come vere e proprie superfici aerodinamiche di controllo per la compensazione dei disturbi di rollio (*roll*) e beccheggio (*pitch*). Il principio di regolazione si basa sull'interazione tra i *flap* e il flusso d'aria discendente generato dai motori: ruotando di un piccolo angolo, il *flap* devia la traiettoria del getto d'aria che lo investe, alterando la distribuzione delle pressioni tra la faccia esposta al flusso (lato di pressione) e quella opposta (lato di depressione). Tale gradiente di pressione genera una forza aerodinamica risultante, approssimabile come perpendicolare alla superficie del *flap* stesso, in grado di imprimere al velivolo un momento correttivo sull'assetto. Affinché il controllo sia efficace, la direzione e il verso di questa forza vengono gestiti in modo da contrastare esattamente il momento perturbativo iniziale. Dal punto di vista meccanico, ciascun *flap* è vincolato a un'estremità al rispettivo servomotore di attuazione, mentre l'estremità opposta è inserita in un apposito modulo strutturale che ne garantisce il mantenimento del parallelismo rispetto al suolo.

Per quanto riguarda l'elettronica di bordo, le componenti hardware necessarie al funzionamento del sistema (che saranno oggetto di un'analisi dettagliata nel Capitolo 2) sono state collocate lungo la superficie esterna del cilindro. La loro disposizione è stata studiata con l'obiettivo di ottenere il miglior bilanciamento possibile delle masse. Sempre all'esterno sono stati integrati sei piedi d'appoggio, disposti in corrispondenza dei vertici di un esagono regolare inscritto nella circonferenza di base del *DPDF*, assicurando così un supporto a terra stabile e una ripartizione omogenea dei pesi.

Infine, dal punto di vista realizzativo, l'intera struttura cilindrica, i *flap*, i piedi di appoggio e i supporti per l'hardware e per i motori sono stati fabbricati tramite **stampa 3D**, impiegando il **PLA** come materiale principale. Le singole parti sono state stampate separatamente per poi essere assemblate e fissate tra loro, andando a costituire il telaio definitivo del sistema. Una panoramica visiva che illustra gli ingombri, la geometria generale della struttura esterna e il dettaglio dei sostegni dei motori è riportata in Figura 1.2.

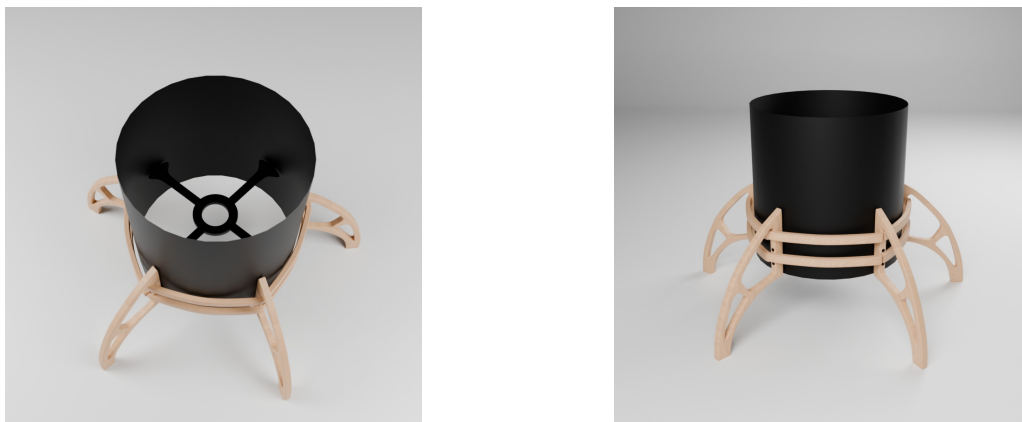


Figura 1.2: *Render Ducted Fan*

1.2 Controllo del Double Propeller Ducted-Fan

La configurazione del *DPDF*, pur ottimizzata per l'efficienza del flusso d'aria, presenta complessità dinamiche dovute a tolleranze costruttive, asimmetrie tra i propulsori e flussi turbolenti interni al condotto. Tali fattori rendono difficoltosa una descrizione accurata tramite modelli lineari derivati analiticamente e rendono la taratura empirica dei regolatori un processo incerto e non riproducibile.

Il presente lavoro risponde a questa sfida attraverso un approccio **data-driven** basato su tre livelli. Il primo livello è la costruzione di un'*infrastruttura di telemetria* che trasforma il drone in una piattaforma capace di registrare ogni variabile di stato rilevante a 50 [Hz] e di ricevere in tempo reale segnali di eccitazione strutturati generati da MATLAB. Il secondo livello è l'*identificazione di sistema*: i *dataset* acquisiti durante prove di eccitazione PRBS vengono elaborati per stimare le funzioni di trasferimento $G_{\text{roll}}(z)$ e $G_{\text{pitch}}(z)$, che descrivono per ciascun asse il legame tra angolo imposto al *flap* e risposta angolare del corpo. Il terzo livello è la *simulazione*: un modello fisico 3D calibrato sui dati reali permette di verificare il comportamento dei regolatori sintetizzati in un ambiente sicuro, prima della validazione sul sistema fisico.

L'insieme di questi tre livelli costituisce la pipeline completa del progetto e la struttura attorno alla quale è organizzato il presente documento, descritta nei Capitoli 3–6.

2 Hardware

Nel presente capitolo vengono descritte le componenti hardware impiegate nel progetto per la realizzazione e il controllo del *DPDF*. Per ciascun elemento verranno illustrati il ruolo specifico all'interno del sistema, le principali specifiche fisiche e tecniche, nonché le rispettive modalità di utilizzo e interfacciamento. A conclusione dell'analisi, verrà presentato un diagramma esplicativo accompagnato da una tabella riepilogativa, al fine di chiarire l'architettura complessiva dei collegamenti elettrici e logici.

2.1 STM32 NUCLEO-H745ZI-Q. La scheda di controllo

La scheda *STM32 NUCLEO-H745ZI-Q* [1] è una piattaforma di sviluppo avanzata prodotta da *STMicroelectronics*. Il cuore del sistema è il microcontrollore *STM32H745ZI-TQ6*, appartenente alla famiglia *STM32H7* ad alte prestazioni. Questo dispositivo è stato concepito per agevolare lo sviluppo, il *debug* e la prototipazione rapida di applicazioni *embedded* di elevata complessità. All'interno del progetto, la *NUCLEO-H745ZI-Q* ha assolto il ruolo di unità di calcolo centrale per lo sviluppo e il collaudo del firmware di gestione di tutte le periferiche e delle logiche di volo del *DPDF*. L'implementazione e il caricamento delle librerie software dedicate sono stati eseguiti avvalendosi dell'ambiente di sviluppo integrato *STM32CubeIDE*. Di seguito viene fornita una descrizione dettagliata delle principali caratteristiche del microcontrollore *STM32H745ZI-TQ6*, la cui architettura generale è illustrata in Figura 2.1.



Figura 2.1: Il microcontrollore *STM32H745ZI-TQ6*

2.1.1 STM32H745ZI-TQ6. Architettura

Il microcontrollore *STM32H745ZI-TQ6* si distingue per un'architettura *dual-core* a 32-bit. Essa integra un processore primario ad altissime prestazioni, l'*Arm Cortex-M7* (capace di operare a una frequenza massima di **480 MHz**), affiancato da un coprocessore *Arm Cortex-M4* (con frequenza massima di **240 MHz**). Per le specifiche esigenze di questo progetto, l'esecuzione del firmware e la gestione delle librerie relative ai sensori e agli attuatori sono state interamente demandate al core *Cortex-M4*. Sebbene l'hardware supporti frequenze operative nettamente superiori, in fase di dimensionamento si è scelto di impostare la frequenza di clock effettiva a **75 MHz**. Tale configurazione è stata generata e definita mediante l'interfaccia grafica *Clock Configuration* fornita dal file di progetto `.ioc`.

2.1.2 STM32H745ZI-TQ6. Timer

Per quanto riguarda il comparto delle temporizzazioni, il microcontrollore mette a disposizione **15 timer** ad alta risoluzione e **5 Low-Power Timer**. Questi moduli risultano fondamentali per la misurazione del tempo, la schedulazione di eventi periodici e, soprattutto, per la generazione dei segnali PWM (*Pulse-Width Modulation*) necessari al controllo della catena cinematica. Nello specifico, il firmware sviluppato sfrutta i seguenti moduli:

- **TIM2**: deputato alla generazione dei segnali PWM per il controllo di posizione dei servomotori dei *flap*.
- **TIM4**: impiegato per la generazione dei segnali PWM destinati agli ESC (*Electronic Speed Controllers*) per la regolazione del regime di rotazione dei motori principali.
- **TIM6**: configurato per gestire la sequenza temporizzata di *safe startup* (avvio in sicurezza) e per il pilotaggio diagnostico dei LED integrati sulla scheda.
- **TIM7**: utilizzato come *timebase* per la generazione periodica dell'*interrupt* che innesci l'aggiornamento del ramo di controllo IMU-servomotori, seguendo un paradigma di tipo *event-driven*.

2.1.3 STM32H745ZI-TQ6. Periferiche di comunicazione

Il microcontrollore integra un vasto set di periferiche di comunicazione. Ai fini dell'interfacciamento hardware richiesto dal *DPDF*, assumono particolare rilevanza i protocolli di comunicazione seriale sincrona e asincrona, tra cui *I²C*, *USART/UART* e *SPI*. Tali interfacce sono state impiegate per stabilire il collegamento dati tra l'unità di controllo, i sensori di assetto/quota e gli strumenti di sviluppo e telemetria. Il chip offre inoltre il supporto nativo per bus di comunicazione di livello superiore, quali *CAN*, *USB OTG* ed *Ethernet*, sebbene questi ultimi non siano stati impiegati nell'attuale implementazione del sistema.

2.1.4 STM32H745ZI-TQ6. Eventi asincroni

La gestione degli eventi asincroni e delle interruzioni hardware è orchestrata dal modulo **NVIC** (*Nested Vectored Interrupt Controller*). Questo componente hardware consente di trattare in modo deterministico ed estremamente efficiente le routine di servizio delle interruzioni (*ISR*), garantendo una corretta gerarchia di priorità *pre-emptive*. L'efficienza del

NVIC rappresenta un requisito cruciale per garantire la stabilità temporale e l'affidabilità dell'architettura *event-driven* alla base del firmware di controllo del volo.

2.1.5 NUCLEO-H745ZI-Q. Caratteristiche della scheda

Caratteristiche fisiche

- Dimensioni: 133.34×70.00 [mm].
- Peso: 120 [g].

Programmazione e Debug (ST-LINK/V3E)

La scheda integra il modulo *ST-LINK/V3E*, il quale consente la programmazione e il *debug* del firmware direttamente tramite connessione USB. Tale modulo fornisce inoltre due interfacce ausiliarie di fondamentale utilità durante le fasi di sviluppo:

- **VCP (Virtual COM Port)**: emula una porta seriale su interfaccia USB, agevolando lo scambio di dati e la telemetria.
- **MSC (Mass Storage Class)**: espone la scheda al sistema operativo host come una memoria di massa esterna, semplificando le operazioni di caricamento dei file binari.

Gestione dell'alimentazione

La piattaforma garantisce un'elevata flessibilità per quanto concerne l'alimentazione, ammettendo tre configurazioni principali:

- Alimentazione a 5 V fornita tramite la porta USB Micro-B dell'interfaccia ST-LINK.
- Alimentazione esterna compresa tra 7 V e 12 V mediante l'apposito Jack esterno.
- Alimentazione diretta a 3.3 V tramite il pin *Vin* esterno.

Indipendentemente dalla sorgente primaria adottata, il microcontrollore è stabilizzato per operare a 3.3 V per mezzo di regolatori di tensione interni. La selezione della modalità di alimentazione avviene attraverso l'impostazione meccanica di specifici *bridge* (*jumper*) che cortocircuitano determinate coppie di pin. Di conseguenza, qualora si opti per sorgenti diverse dalla connessione USB standard (*ST-LINK*), è indispensabile verificare preventivamente e con attenzione la corretta configurazione di tali ponticelli. Il dettaglio dei *bridge* rilevanti per l'alimentazione è illustrato in Figura 2.2.

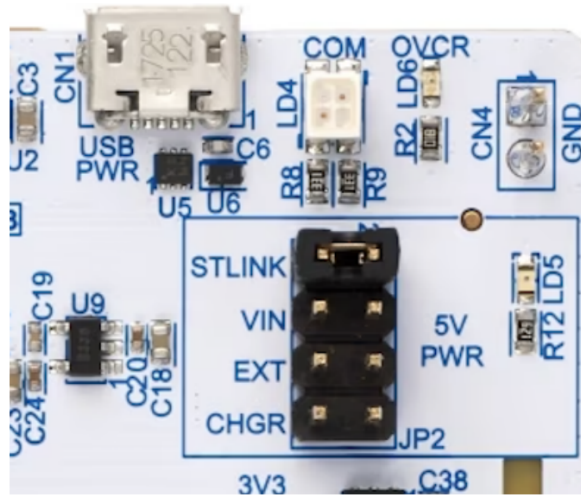
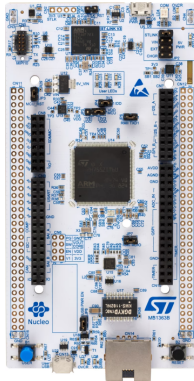


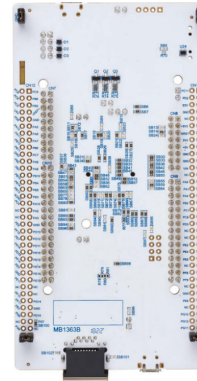
Figura 2.2: *NUCLEO-H745ZI-Q. Dettaglio esplicativo dei bridge*

Formato fisico ed espandibilità

Appartenendo alla linea *Nucleo-144* di *STMicroelectronics*, la scheda di sviluppo mette a disposizione due connettori di tipo *Morpho*, i quali espongono e rendono liberamente accessibili tutti i 144 pin del microcontrollore. Questo sistema di espansione proprietario prende il nome di *ST Zio*. A ulteriore supporto della prototipazione, la *NUCLEO-H745ZI-Q* garantisce la piena compatibilità hardware e di piedinatura (*pinout*) per l'alloggiamento degli *shield* standardizzati per Arduino Uno R3. Una vista complessiva della scheda e delle sue interfacce è riportata in Figura 2.3.



(a) *Vista frontale*



(b) *Vista posteriore*

Figura 2.3: *La scheda di sviluppo NUCLEO-H745ZI-Q*

2.2 TATTU LiPo. Batteria ricaricabile

Motori, servomotori ed ESC richiedono specifici livelli di tensione per il loro funzionamento, tensioni che non possono essere fornite dalla *NUCLEO H745ZI-Q*. Si rende, quindi,

necessario l’inserimento nel progetto di batterie ricaricabili. In base alle prestazioni richieste dal progetto, si è optato per le **TATTU LiPo**.

2.2.1 TATTU LiPo. Caratteristiche fisiche

- Dimensioni: $74 \times 35.5 \times 29$ [mm].
- Peso: 150 [g].
- Numero di celle: 4.

2.2.2 TATTU LiPo. Caratteristiche tecniche e considerazioni operative

Tensione

L’accumulatore adottato per l’alimentazione del sistema è una batteria *TATTU LiPo* (*Litio-Polimero*) configurata in architettura **4S**, ossia composta da 4 celle collegate in serie. Essendo la tensione nominale di ogni singola cella pari a **3.7 [V]**, il pacco batteria fornisce una tensione nominale complessiva di **14.8 [V]**. In condizioni di carica completa, la tensione per singola cella raggiunge i **4.2 [V]**, portando la tensione totale a **16.8 [V]**. Al fine di preservare la vita utile dell’accumulatore e rallentarne il degrado chimico, è fondamentale mantenere la tensione il più possibile prossima al valore nominale, evitando periodi prolungati di permanenza in stato di carica massima e, soprattutto, fenomeni di scarica profonda. A tale scopo, è strettamente necessario evitare che la tensione complessiva scenda al di sotto dei **12.8 [V]**, garantendo in questo modo che la tensione di ciascuna cella non si riduca mai oltre la soglia critica di **3.2 [V]**.

Capacità

La capacità, misurata in [mAh], indica la quantità di carica elettrica che la batteria è in grado di immagazzinare e successivamente erogare nel tempo. Nel caso in esame, la capacità nominale della singola *LiPo* è pari a **1300 [mAh]**, il che significa che l’accumulatore è teoricamente in grado di erogare una corrente continua di 1.3 [A] per la durata di un’ora. In prima approssimazione, l’autonomia temporale del sistema può essere stimata attraverso la seguente relazione:

$$t_h = \frac{C [\text{mAh}]}{A [\text{mA}]} \quad (2.1)$$

dove t_h rappresenta il tempo di funzionamento stimato (espresso in ore), C è la capacità della batteria in [mAh] e A è la corrente media assorbita dal carico, espressa in [mA].

Al fine di incrementare l’autonomia di volo, nel presente progetto si è optato per il collegamento in **parallelo** di due unità *TATTU LiPo* identiche, raddoppiando di fatto la capacità totale a disposizione fino a **2600 [mAh]**. Tale parallelo è stato realizzato in sicurezza impiegando il modulo *PowerSafe Twin*, prodotto da *MNR-electronics*, le cui caratteristiche verranno approfondite nella sezione successiva.

Capacità di scarica (C-rating)

La capacità di scarica definisce la corrente massima che l’accumulatore può erogare in modo continuativo senza subire danni strutturali o surriscaldamenti. Le batterie *TATTU LiPo*

impiegate presentano un rateo di scarica nominale pari a **75C**. Di conseguenza, la corrente massima teorica erogabile dalla singola batteria è calcolabile come segue:

$$A_{max} = C_s \cdot C \Rightarrow A_{max} = 75 [C] \cdot 1.3 [A] = 97.5 [A] \quad (2.2)$$

dove A_{max} è la corrente massima continua espressa in [A], C_s è il rateo di scarica caratteristico e C indica la capacità nominale dell'accumulatore (convertita in Ampere-ora). Una rappresentazione visiva del pacco batteria assemblato e impiegato durante le sessioni di prova è fornita in Figura 2.4.



Figura 2.4: TATTU LiPo 4S 1300mAh 75C

2.3 PowerSafe Twin ADV. MNR-electronics

Al fine di garantire la massima sicurezza e affidabilità del sistema di alimentazione a doppio pacco batteria, è stato integrato un modulo di ridondanza attiva: il **PowerSafe Twin ADV**, prodotto da *MNR-electronics*.

Questo dispositivo rappresenta una soluzione avanzata per la gestione sicura dell'energia di bordo. Il modulo, infatti, effettua un monitoraggio continuo delle principali variabili di stato (quali tensione e corrente), selezionando in tempo reale l'accumulatore ottimale per l'alimentazione del carico. Qualora, ad esempio, la tensione di una batteria dovesse scendere al di sotto di una soglia critica di sicurezza, il sistema commuterebbe in modo del tutto automatico sull'unità secondaria, scongiurando così qualsiasi interruzione transitoria o calo significativo nell'erogazione di potenza ai motori e all'elettronica.

Analizzando l'architettura interna del sistema, è possibile distinguere tre sezioni funzionali principali:

- **Modulo di potenza:** è preposto alla gestione della commutazione hardware tra i pacchi batteria, al bilanciamento e alla distribuzione della corrente verso il carico. Questa sezione integra inoltre i circuiti di protezione necessari per isolare il sistema in caso di guasto elettrico (es. cortocircuito o sovracorrente).
- **Modulo di controllo:** basato su un microprocessore dedicato, ha il compito di campionare e analizzare costantemente le grandezze elettriche (come i livelli di tensione, gli assorbimenti di corrente e i potenziali squilibri tra le celle), implementando fisicamente la logica algoritmica che governa la ridondanza attiva.

- **Modulo di allarme:** fornisce un *feedback* diagnostico all'operatore, segnalando l'insorgenza di eventuali anomalie operative o cali di tensione critici attraverso indicatori luminosi (LED) e avvisi acustici.

Una raffigurazione del dispositivo impiegato per l'assemblaggio del *DPDF* è riportata in Figura 2.5.

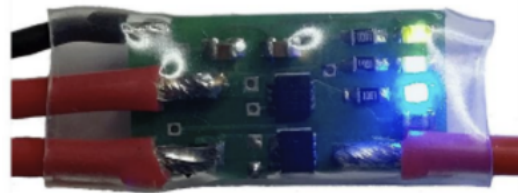


Figura 2.5: *PowerSafe Twin ADV MNR-electronics*

2.3.1 PowerSafe Twin ADV. Caratteristiche fisiche e tecniche

- Dimensioni con involucro protettivo: $80 \times 59 \times 26$ [mm].
- Peso, considerando sia l'involucro protettivo che i cablaggi (connettori XT60): 100 [g].
- Corrente massima gestibile: 100 [A] (in continua).
- Numero di celle per pacco batteria gestibili: $[3S \div 8S]$.
- Tensione massima ammessa in ingresso: ≈ 33.6 [V]. Considerando il minimo 3.7 [V] e il massimo 4.2 [V] per cella: $[11.1 \div 33.6]$ [V].

2.4 DFRobot Power Module. Convertitore di tensione

Al fine di garantire una corretta e sicura distribuzione dell'energia elettrica a tutte le periferiche di bordo, si è reso necessario l'impiego di moduli convertitori di tensione (*DC-DC converter*), in grado di adattare la potenza erogata dalle batterie alle specifiche esigenze di ciascun dispositivo.

Il componente selezionato per questo scopo è il *DFRobot Power Module* [2]. Il dispositivo riceve in ingresso la tensione non regolata proveniente dal pacco batterie e la ripartisce, mettendo a disposizione differenti opzioni di alimentazione a seconda del terminale di uscita impiegato:

- **Uscita diretta (Pass-through):** una prima soluzione progettuale fornita dal dispositivo consiste nell'erogare in uscita il medesimo valore di tensione applicato in ingresso. In questa configurazione, il modulo opera di fatto come un ponte elettrico per il trasferimento diretto della potenza.
- **Uscita stabilizzata a 5 [V]:** utilizzando dei pin dedicati presenti sulla scheda, il dispositivo è in grado di erogare una tensione regolata a 5 [V], selezionabile e attivabile mediante un apposito interruttore (*switch*) hardware.

- **Uscita variabile:** infine, il modulo presenta un terminale la cui tensione di uscita è regolabile. Grazie alla presenza di un potenziometro (o *trimmer*), è possibile tarare finemente la tensione desiderata su valori intermedi, fino a un massimo coincidente con la tensione di ingresso.

A causa degli elevati assorbimenti di corrente e per garantire la massima affidabilità del sistema di attuazione aerodinamica, la presenza dei due servomotori ha richiesto l'integrazione di **due convertitori di tensione distinti** all'interno dell'architettura hardware.

Lo schema esplicativo che illustra la disposizione dei terminali di uscita è richiamato in Figura 2.6, mentre una fotografia del modulo convertitore effettivamente impiegato sul prototipo è riportata in Figura 2.7.

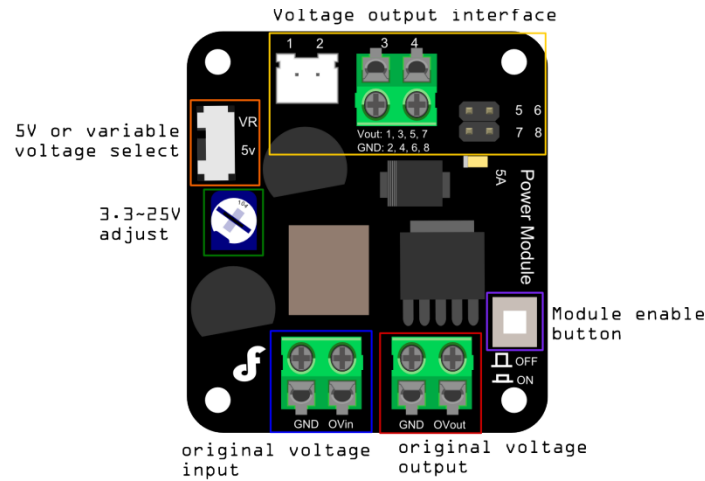


Figura 2.6: *DFRobot Power Module. Disposizione dei terminali di uscita*

Caratteristiche fisiche

- Dimensioni: $46 \times 50 \times 20$ [mm].
- Peso: 35 [g].

Caratteristiche tecniche

- Intervallo di tensione in ingresso: $[3.6 \div 25]$ [V].
- Intervallo di tensione esprimibile in uscita: $[3.3 \div 25]$ [V].
- Massima potenza erogabile: 25 [W].
- Frequenza di variazione: 350 [kHz].

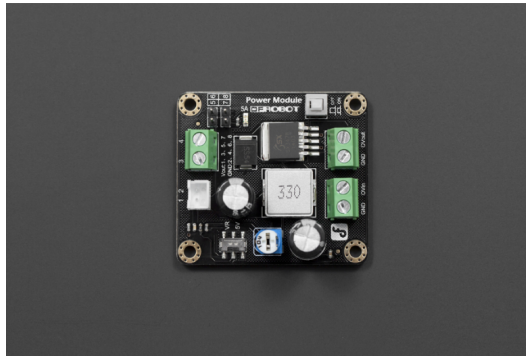


Figura 2.7: *DFRobot Power Module*

2.5 Power Distribution Board

Con *Power Distribution Board (PDB)* si fa riferimento ad un sottosistema elettrico la cui principale funzione è la **distribuzione in modo sicuro ed efficiente dell'energia fornita dal pacco batterie agli ESC** e, conseguentemente, ai motori. Ai fini progettuali, è stata selezionata una PDB, il cui punto di prelievo del segnale presenta un connettore **XT60**, adatto al pacco batterie. Il dispositivo integrato nel contesto operativo possiede inoltre quattro uscite, ciascuna in grado di erogare al massimo una corrente di **20 [A]**, e ha un peso complessivo di **27.3 [g]**. Una vista del componente è riportata in Figura 2.8.

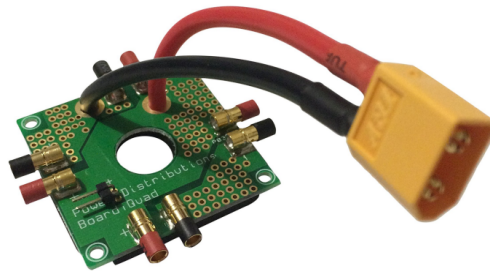


Figura 2.8: *Power Distribution Board*

2.6 Turnigy AereoDrive SK3-3536 1400KV

Il **Turnigy AereoDrive SK3 3536 1400KV** è un motore *brushless* trifase a magneti permanenti, il cui funzionamento richiede un *Electronic Speed Controller (ESC)* per la corretta commutazione delle fasi. Invertendo due delle tre fasi è possibile invertire anche il senso di rotazione del motore. Nel progetto sono stati impiegati due motori di questo tipo in configurazione controrotante, così da ridurre le coppie di reazione delle eliche e contenere le rotazioni indesiderate di *yaw*, senza però riuscire a compensarle completamente. Tale scelta è stata inoltre necessaria per garantire la spinta richiesta al sollevamento del sistema.

2.6.1 Turnigy AereoDrive SK3-3536 1400 KV. Caratteristiche fisiche e tecniche

- Tensione operativa: $[11.1 \div 16.8]$ [V].
- Valore KV: 1400 RPM/V.
- Potenza massima: 590 [W].
- Corrente massima: 40 [A].
- Resistenza interna: $[0.021 \div 0.025]$ [Ω].
- Corrente a vuoto: 0.021 [A].
- Diametro dell'albero motore: 5 [mm].
- Peso: 111 [g].
- Spaziatura fori di montaggio: 25×25 [mm].
- Connettori *bullet* da 3.5 [mm].
- Numero dei poli: 12.

Il valore "KV" indica il numero di giri al minuto (RPM) che il motore compie per ogni volt applicato a vuoto, in formula:

$$RPM = KV \cdot V \quad (2.3)$$

A tensioni di alimentazione maggiori rispetto a quelle specificate il motore rischia di assorbire troppa corrente e surriscaldarsi. I giri al minuto massimi dipenderanno dalla tensione applicata e dalle dimensioni delle eliche. Nel presente progetto, il motore in interesse deve essere alimentato a batteria, vale a dire in corrente continua, lasciando spazio alla necessità di un dispositivo che trasformi la tensione continua erogata dalle batterie in un segnale trifase a corrente alternata. Il dispositivo in questione è il precedentemente citato *Electronic Speed Controller*. Il motore adottato nel progetto è mostrato in Figura 2.9.



Figura 2.9: *Turnigy Aereo Drive SK3 3536 1400 KV*

2.7 Turnigy Plush 40A. Electronic Speed Controller

Il **Turnigy Plush 40A** [3] è un controllore elettronico di velocità per motori *brushless* *BLDC*. Il suo compito è convertire la tensione continua di alimentazione nelle tre fasi necessarie al pilotaggio del motore. Il dispositivo è di tipo *sensorless*: la posizione del rotore viene stimata indirettamente attraverso la forza controelettromotrice, senza impiegare sensori dedicati. Il comando di velocità viene invece ricevuto dal microcontrollore sotto forma di segnale PWM, sulla base del quale l'ESC genera la corretta sequenza di commutazione delle fasi. Nel *Double Propeller Ducted Fan* sono impiegati due *Turnigy Plush 40A*, uno per ciascun motore; il componente è riportato in Figura 2.10.

2.7.1 Turnigy Plush 40A. Caratteristiche fisiche

- Corrente nominale: 40 [A].
- Corrente di picco: 55 [A].
- Presenza di un *Battery Eliminator Circuit (BEC)*: 5 [V] a 3 [A].
- Celle di batteria necessarie per il corretto funzionamento del dispositivo: $[2 \div 6]$.
- Peso: 39 [g].
- Dimensioni in [mm]: $60 \times 24 \times 15$.



Figura 2.10: *Turnigy Plush 40A*

2.7.2 Turnigy Plush 40A. Risoluzione e controllo dei motori

L'ESC *Turnigy Plush 40A* gestisce la potenza erogata ai motori mediante un segnale di controllo PWM (*Pulse-Width Modulation*) operante a una frequenza di **50 [Hz]**. Prima di

consentire l'azionamento dei rotori, il dispositivo impone l'esecuzione di una rigorosa procedura di sicurezza definita **armamento** (*arming*), la quale viene innescata fornendo in ingresso un segnale PWM con un *duty cycle* costantemente mantenuto al **5%**.

Lo stato operativo e l'esito della procedura di inizializzazione vengono comunicati all'operatore attraverso una diagnostica basata su specifici *feedback* acustici emessi dall'ESC stesso:

Attesa di un segnale di armamento valido

Un'emissione sonora ripetuta con una cadenza di due secondi segnala che l'ESC è correttamente alimentato dal lato di potenza, ma non rileva sul canale di controllo un segnale PWM idoneo a consentire l'armamento (ovvero il comando di "motore al minimo").

Esito positivo dell'operazione di armamento

L'avvenuto sblocco di sicurezza e la conseguente operatività del sistema propulsivo sono confermati in modo inequivocabile da una sequenza composta da tre segnali acustici brevi, seguiti da una singola emissione prolungata.

Esito negativo dell'operazione di armamento

Una rapida e ininterrotta sequenza di segnali sonori evidenzia una condizione di anomalia o un errore nella procedura di inizializzazione. Tale eventualità è tipicamente riconducibile al rilevamento di un valore iniziale del segnale PWM non conforme ai requisiti di sicurezza, oppure a un incremento troppo repentino del *duty cycle* prima che il sistema si sia stabilizzato.

Sulla base delle evidenze sperimentali raccolte durante le fasi preliminari di collaudo, è emersa l'opportunità di mantenere il segnale di armamento costante per un intervallo temporale compreso tra i **2** e i **6** [s] prima di impartire effettivi comandi di trazione. Inoltre, la caratterizzazione pratica del sistema propulsivo ha permesso di definire che l'intervallo di lavoro utile dei motori, ai fini della generazione della spinta per il *DPDF*, corrisponde a un *duty cycle* strettamente circoscritto tra il **5%** e il **10%**.

2.8 APC 9x4.5. Eliche

2.8.1 Caratteristiche fisiche

- Diametro: 9 [in], pari a 228.6 [mm].
- Passo: 4.5 [in], pari a 114.3 [mm].
- Peso: 17.9 [g].
- Materiale: Nylon rinforzato con fibra di vetro (*Fiberglass-Reinforced Nylon, FRN*).

La geometria dell'elica impiegata è riportata in Figura 2.11.



Figura 2.11: *Elica APC 9x4.5*

2.9 Hitec HS-82MG Gear micro servo

Come discusso nella descrizione della struttura meccanica, il controllo dell'assetto in *roll* e *pitch* è affidato a due *flap* montati nella parte inferiore del condotto. Tali superfici, investite dal getto prodotto dai motori controrotanti, generano l'azione aerodinamica correttiva necessaria a contrastare le rotazioni indesiderate del velivolo. Il movimento di ciascun *flap* è affidato a un servomotore dedicato, in particolare nel progetto sono stati integrati due servomotori **Hitec HS-82MG** [4].

2.9.1 HS-82MG Hitec. Caratteristiche fisiche

- Dimensioni: $29.8 \times 12.0 \times 29.6$ [mm].
- Peso: 19.0 [g].

Le dimensioni del servomotore sono riportate in Figura 2.12.

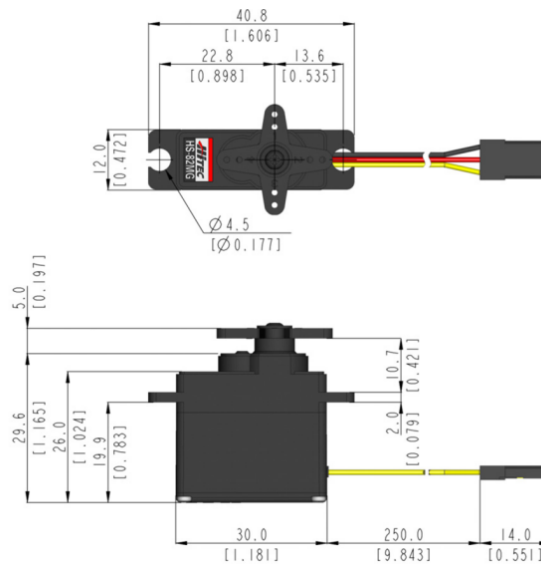


Figura 2.12: *HS-82MG Hitec. Misure*

Coppia di stasi (*Holding torque*)

Rappresenta il momento torcente massimo che il servomotore è in grado di esercitare per mantenere fissa la propria posizione angolare, contrastando in modo statico una sollecitazione meccanica esterna. Per il modello *HS-82*, tale parametro si attesta a **2.2 [kg·cm]** con un'alimentazione di **4.8 [V]**, incrementandosi fino a **2.7 [kg·cm]** qualora la tensione di ingresso venga elevata a **6.0 [V]**.

Coppia di stallo (*Stall torque*)

Definisce il momento torcente massimo teorico che l'attuatore è in grado di sviluppare in condizioni di velocità angolare nulla, ovvero quando il rotore è bloccato da un carico inamovibile. I valori nominali garantiti per questo dispositivo sono pari a **2.8 [kg·cm]** a **4.8 [V]** e **3.4 [kg·cm]** a **6.0 [V]**.

Velocità angolare

Indica il tempo richiesto per compiere una determinata escursione angolare in assenza di carico meccanico. Alimentato a una tensione di **4.8 [V]**, il servomotore copre un angolo di 60° in **0.12 [s]**; elevando l'alimentazione a **6.0 [V]**, il tempo di rotazione si riduce a **0.10 [s]**. Esprimendo il dato in relazione a un singolo grado di variazione, si ottiene un tempo di azionamento pari a **0.0020 [s/°]** a **4.8 [V]** e **0.0017 [s/°]** a **6.0 [V]**.

Corrente operativa

Rappresenta l'assorbimento medio di corrente del servomotore durante la sua normale movimentazione (misurato in assenza di carico applicato all'albero). In tali condizioni, durante un'escursione di 60°, si registra un assorbimento di **220 [mA]** operando a **4.8 [V]**, valore che sale a **280 [mA]** se il dispositivo è alimentato a **6.0 [V]**.

Corrente di stallo

Costituisce l'assorbimento massimo di corrente che si verifica quando il motore è alimentato ma si trova in una condizione di blocco meccanico forzato. In questa situazione critica, la richiesta di corrente raggiunge i **1450 [mA]** con un'alimentazione a **4.8 [V]**, e si eleva fino a **1800 [mA]** a **6.0 [V]**.

Corsa operativa

Fa riferimento all'escursione angolare totale che il servomotore è in grado di coprire in risposta a un segnale di comando PWM standard. Nel caso specifico del modello *HS-82*, tale intervallo utile è pari a **120°**. Una rappresentazione visiva dell'attuatore è riportata in Figura 2.13.



Figura 2.13: *HS-82MG*

2.9.2 HS-82MG Hitec. Caratteristiche tecniche e controllo

Alimentazione

L'attuatore opera in un intervallo di tensione nominale compreso tra **4.8 e 6.0 [V]**. Sebbene l'impiego di tensioni prossime al limite superiore consenta un sensibile miglioramento delle prestazioni dinamiche del dispositivo, tale configurazione comporta un incremento della dissipazione termica per effetto Joule e un maggiore *stress* a carico della circuiteria elettronica interna. Al fine di garantire un equilibrio ottimale tra reattività e affidabilità del sistema nel tempo, si è scelto di stabilizzare la tensione di alimentazione in ingresso al valore di **5 [V]**.

Controllo

La regolazione della posizione angolare del rotore è affidata a un segnale di controllo di tipo PWM (*Pulse-Width Modulation*). Per l'interfacciamento con il microcontrollore, è stata selezionata una frequenza di commutazione pari a **50 [Hz]**, con un'ampiezza di segnale di **3.3 [V]**.

L'architettura di controllo è stata definita per soddisfare i seguenti requisiti operativi:

- **Configurazione neutra:** per ottenere il posizionamento dei *flap* in configurazione iniziale (perfettamente perpendicolari al piano del suolo), è necessario fornire un impulso di attivazione con durata pari a **1.5 [ms]**, corrispondente a un *duty cycle* del **7.5%**.
- **Rotazione oraria:** l'escursione angolare in senso orario viene comandata modulando la durata dell'impulso nell'intervallo compreso tra **1.5 e 2.1 [ms]**.
- **Rotazione antioraria:** specularmente, il movimento in senso antiorario si ottiene operando nell'intervallo di temporizzazione tra **0.9 e 1.5 [ms]**.

La disposizione dei terminali di collegamento del servomotore e la relativa piedinatura sono illustrate in Figura 2.14.



Figura 2.14: *HS-82MG Hitec. Disposizione dei terminali*

2.10 BNO055 Bosch Sensortec. Unità di misura inerziale e magnetometro

Per garantire un accurato controllo dell'assetto del *DPDF*, risulta indispensabile conoscere in tempo reale l'esatto orientamento del sistema rispetto a una terna di riferimento spaziale fissa. Si rende pertanto necessario l'impiego di un dispositivo in grado di rilevare ed elaborare tale informazione cinematica. Il **BNO055**, prodotto da *Bosch Sensortec*, è un sensore di orientamento assoluto a 9 assi che, grazie alla sua elevata accuratezza e al notevole livello di integrazione, rappresenta la soluzione hardware ideale per questo scopo.

Il dispositivo si configura come un *System in Package (SiP)* il quale, come documentato nel *datasheet* ufficiale [5], integra al proprio interno: un accelerometro triassiale a 14 bit, un giroscopio triassiale a 16 bit, un sensore geomagnetico triassiale e un microcontrollore a 32 bit basato su architettura *Arm Cortex-M0+*. Il compito primario di tale microcontrollore è l'esecuzione *on-board* di un avanzato algoritmo proprietario di *sensor fusion*.

Questo algoritmo elabora e combina in tempo reale i dati grezzi provenienti dall'accelerometro, dal giroscopio e dal magnetometro, filtrando i disturbi e fornendo direttamente in uscita misure di assetto stabili e precise. Tali dati possono essere estratti sotto forma di quaternioni oppure come angoli di Eulero (rollio, beccheggio e imbardata, ovvero *roll*, *pitch* e *yaw*), unitamente ai vettori di accelerazione lineare e gravitazionale.

Nel contesto del presente progetto, il chip *BNO055* è stato adottato nella sua versione pre-assemblata su un'apposita scheda di *breakout*, soluzione che ne semplifica notevolmente l'interfacciamento e il cablaggio elettrico. Per maggiore completezza descrittiva, nei paragrafi successivi verranno riportate le caratteristiche dimensionali e di peso riferite sia al singolo chip sia all'intero modulo di *breakout*.

2.10.1 BNO055 Bosch Sensortec. Caratteristiche fisiche

- Dimensioni del modulo: $20 \times 24 \times 2$ mm.
- Dimensioni del chip: $3.8 \times 5.2 \times 1.1$ mm.
- Massa del modulo: 3 g.
- Massa del chip: ≈ 150 mg.

2.10.2 BNO055 Bosch Sensortec. Infrastruttura di alimentazione, comunicazione ed integrazione del sensore

Alimentazione

Il chip *BNO055* dispone di due pin di alimentazione distinti, denominati **VDD** e **VDDIO**,

dedicati rispettivamente all'alimentazione della circuiteria interna del sensore e delle interfacce logiche digitali. I relativi intervalli di tensione operativa sono compresi tra **2.4 e 3.6 [V]** per il terminale *VDD*, e tra **1.7 e 3.6 [V]** per il terminale *VDDIO*. Tuttavia, impiegando il dispositivo preassemblato su scheda di *breakout*, tali ingressi vengono gestiti e unificati internamente dalla circuiteria della scheda stessa. Di conseguenza, l'alimentazione all'intero modulo viene fornita attraverso un unico terminale di ingresso, denominato **VIN**, che accetta tensioni comprese nell'intervallo $2.4 \div 3.6$ [V]. Tra i diversi profili di gestione energetica previsti dal costruttore, per lo sviluppo del progetto si è optato per la configurazione in **Normal mode**, al fine di garantire la piena e continua operatività del sensore.

Protocollo di comunicazione

Per quanto concerne l'interfacciamento con il microcontrollore *host*, il *BNO055* supporta nativamente i protocolli di comunicazione seriale *I²C* e *UART*. Per le specifiche esigenze architetturali del *DPDF*, l'acquisizione della telemetria inerziale è stata implementata avvalendosi esclusivamente del bus di comunicazione *I²C*. Il sensore è compatibile sia con la trasmissione in *Standard Mode*, operante a una frequenza di clock di **100 [kHz]**, sia in *Fast Mode*, raggiungendo velocità fino a **400 [kHz]**. Sia il sensore integrato sia la relativa *breakout board* espongono i terminali standard **SDA** (*Serial Data*) e **SCL** (*Serial Clock*) necessari per instaurare la comunicazione digitale. La mappatura completa dei pin (*pinout*) del modulo utilizzato è richiamata in Figura 2.15.

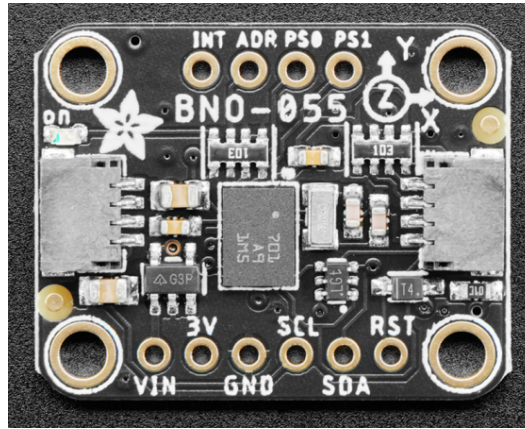


Figura 2.15: *BNO055 Bosch Sensortec. Configurazione dei pin e breakout board*

2.10.3 BNO055 Bosch Sensortec. Sensori inerziali e sensor fusion

Il *BNO055* integra al proprio interno un accelerometro, un giroscopio e un magnetometro in un singolo dispositivo, combinandone le misurazioni attraverso un algoritmo proprietario di *sensor fusion*. Nello specifico, l'accelerometro è un sensore **MEMS** capacitivo dedicato alla misurazione dell'accelerazione lineare lungo i tre assi; il giroscopio è un sensore MEMS a struttura vibrante preposto al rilevamento della velocità angolare; infine, il magnetometro rileva il campo magnetico terrestre, fornendo un riferimento assoluto per l'orientamento spaziale. L'architettura di sistema del sensore è schematizzata in Figura 2.16.

Accelerometro

Il modulo accelerometrico presenta una risoluzione di **14 bit** e supporta fondoscale di misura selezionabili tra ± 2 [g], ± 4 [g], ± 8 [g] e ± 16 [g]. Ai fini del progetto, il sensore è stato configurato con un intervallo operativo di ± 4 [g] e una banda passante di **62.5 [Hz]**.

Giroscopio

Il giroscopio integrato vanta una risoluzione di **16 bit** e mette a disposizione intervalli di misura compresi tra ± 125 [°/s] e ± 2000 [°/s]. Abilitando l'algoritmo di *sensor fusion*, il dispositivo adotta automaticamente il fondoscala massimo di ± 2000 [°/s], operando con una banda passante di **32 [Hz]**.

Magnetometro

Il magnetometro è in grado di rilevare intensità di campo magnetico fino a ± 1300 [μ T] lungo gli assi X e Y, e fino a ± 2500 [μ T] lungo l'asse Z. La risoluzione della misura è pari a **13 bit** per gli assi planari (X, Y) e a **15 bit** per l'asse verticale (Z). Quando l'algoritmo di *sensor fusion* è attivo, la banda passante operativa del magnetometro è impostata a **10 [Hz]**.

Sensor fusion

Presi singolarmente, i tre trasduttori inerziali presentano limiti fisici intrinseci e complementari: il giroscopio è soggetto a deriva temporale (*drift*), l'accelerometro risente fortemente delle accelerazioni lineari non gravitazionali (come vibrazioni e transitori del telaio) e il magnetometro è altamente sensibile alle perturbazioni elettromagnetiche ambientali. La logica di *sensor fusion* sfrutta congiuntamente i dati grezzi provenienti dai tre componenti per elaborare una stima dell'orientamento nettamente più stabile, robusta e affidabile.

Come riportato nella documentazione tecnica del *BNO055*, tale elaborazione matematica si fonda sull'implementazione di un **Filtro di Kalman Esteso (EKF)**. Nel presente lavoro, questa modalità è stata mantenuta costantemente attiva; nello specifico, l'architettura di controllo sviluppata sfrutta direttamente le stime degli angoli di rollio, beccheggio e imbardata (*roll*, *pitch* e *yaw*) calcolate *on-board* dal sensore, mentre non si è reso necessario l'impiego diretto delle ulteriori grandezze cinematiche disponibili. A seguire, è riportata una rappresentazione schematica del *System in Package BNO055* sviluppato da *Bosch Sensortec*.

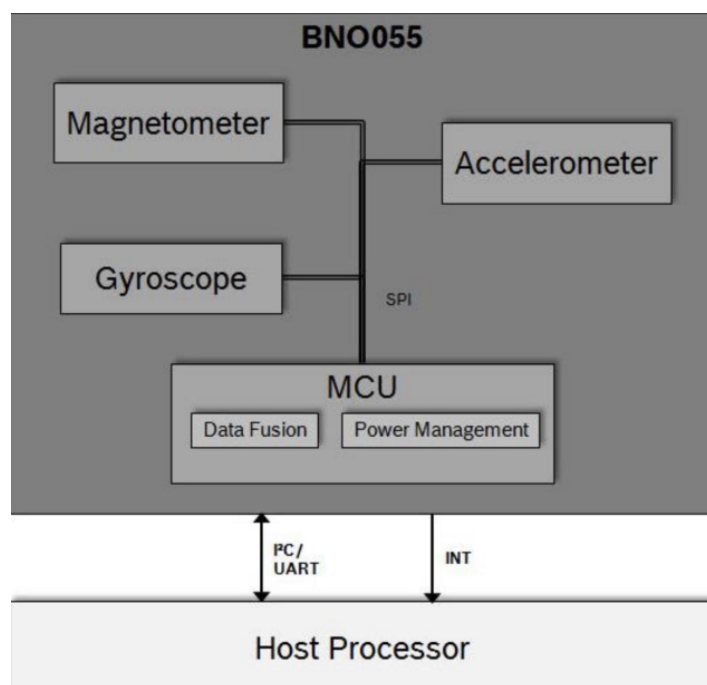


Figura 2.16: BNO055 Bosch Sensortec. Architettura di sistema

2.10.4 BNO055 Bosch Sensortec. Modalità operative

Modalità operative con Sensor Fusion (*Fusion Modes*)

Quando il dispositivo viene configurato in una delle modalità operative provviste di *sensor fusion*, l'algoritmo interno esegue automaticamente, in *background*, la calibrazione continua dei trasduttori. Le modalità disponibili sono le seguenti:

- **IMU**: stima l'orientamento spaziale combinando esclusivamente i dati dell'accelerometro e del giroscopio. Grazie alla sua elevata reattività, questa modalità risulta particolarmente indicata per applicazioni altamente dinamiche.
- **M4G (*Magnetometer for Gyroscope*)**: presenta un comportamento analogo alla modalità IMU, ma impiega il magnetometro in sostituzione del giroscopio per correggere la stima dell'orientamento, vincolandola al campo magnetico terrestre.
- **COMPASS**: il dispositivo opera come una bussola elettronica a stato solido, fornendo una stima dell'orientamento sul piano orizzontale riferita al Nord magnetico.
- **NDOF (*Nine Degrees of Freedom*)**: sfrutta congiuntamente i dati provenienti da tutti e tre i sensori (accelerometro, giroscopio e magnetometro) per restituire la stima dell'orientamento assoluto più accurata e completa possibile.

Modalità operative senza Sensor Fusion (*Non-Fusion Modes*)

Le modalità operative prive di fusione sensoriale consentono all'utente di acquisire direttamente i dati grezzi (*raw data*) misurati dai singoli trasduttori o da specifiche combinazioni di essi, aggirando del tutto l'algoritmo di elaborazione interna.

Nello specifico, il dispositivo può essere configurato per abilitare esclusivamente l'accelerometro (**ACCONLY**), il magnetometro (**MAGONLY**) o il giroscopio (**GYROONLY**), oppure per attivare specifiche coppie di sensori (**ACCMAG**, **ACCGYRO**, **MAGGYRO**). Infine, la modalità **AMG** abilita simultaneamente tutti e tre i trasduttori, demandando tuttavia l'onere computazionale dell'eventuale fusione dei dati al microcontrollore *host* esterno.

Ai fini dello sviluppo del presente progetto, tali configurazioni a dati grezzi non sono state impiegate. La scelta progettuale è infatti ricaduta in modo esclusivo sulla modalità di fusione **NDOF**, ritenuta la soluzione più idonea ed efficiente per demandare al sensore il calcolo dell'assetto e ottenere direttamente una stima stabile e affidabile dell'orientamento del drone.

2.11 VL53L1X ST. Sensore Time-of-Flight per la misurazione a lunga distanza

Al fine di controllare correttamente il *DPDF*, è necessario stimare la distanza dal suolo durante il volo. Per questo scopo è stato adottato il **VL53L1X** di *STMicroelectronics* [6]. Nel progetto non è stato utilizzato il sensore in forma nativa, ma il modulo *IRIS11A0J9776* prodotto da *Pololu*, scelto per semplificare l'integrazione elettronica grazie alla relativa *breakout board*.

2.11.1 Il principio di misura di un sensore Time-of-Flight (ToF)

La misura *Time-of-Flight* si basa sul tempo impiegato da un impulso luminoso per compiere un tragitto di andata e ritorno tra il sensore e l'oggetto da rilevare. Nel caso del *VL53L1X*, l'emissione avviene tramite un laser **VCSEL**, mentre la radiazione riflessa viene raccolta da un rilevatore sensibile, tipicamente uno **SPAD array**. Una volta noto il tempo di volo Δt , la distanza si ricava da:

$$d = \frac{c \cdot \Delta t}{2} \quad (2.4)$$

dove d è la distanza dall'oggetto e c è la velocità della luce. Il fattore 2 tiene conto del percorso di andata e ritorno del segnale. Nel caso in esame, il *VL53L1X* utilizza la tecnologia **dToF** (*Direct Time-of-Flight*), adatta a misure di distanza accurate e robuste per il controllo di quota del sistema.

2.11.2 VL53L1X STMicroelectronics. Caratteristiche fisiche

A seguire, un breve elenco delle principali caratteristiche fisiche:

- Dimensioni del chip: $4.90 \times 1.25 \times 1.56$ [mm].
- Dimensioni del modulo: $17.50 \times 12 \times 2.56$ [mm].
- Massa del chip: 30 [mg].
- Massa del modulo: 0.5 [g] (senza i *pin header*).

2.11.3 VL53L1X STMicroelectronics. Caratteristiche tecniche

Alimentazione

Il *VL53L1X* presenta un unico ingresso di alimentazione, indicato come **VDD**, con intervallo operativo $[2.6 \div 3.5]$ [V].

Protocollo di comunicazione

Il sensore comunica esclusivamente tramite protocollo I^2C e supporta la *Fast Mode* a **400 [kHz]**. I terminali messi a disposizione per la comunicazione sono **SDA** e **SCL**.

XSHUT e GPIO 1 (Interrupt)

Il *VL53L1X* dispone dell'ingresso **XSHUT**, usato per spegnimento, riavvio o reset hardware del dispositivo, e dell'uscita **GPIO1**, utilizzata come linea di *interrupt* per segnalare eventi al microcontrollore. Nel progetto, **XSHUT** è stato impiegato per il reset forzato del sensore, mentre **GPIO1** è stato usato per notificare la disponibilità di una nuova misura, semplificando la gestione firmware e la temporizzazione. La configurazione dei pin del modulo e lo schema a blocchi del sensore sono riportati in Figura 2.17.

Il dettaglio dell'emettitore ottico del sensore è mostrato in Figura 2.18.

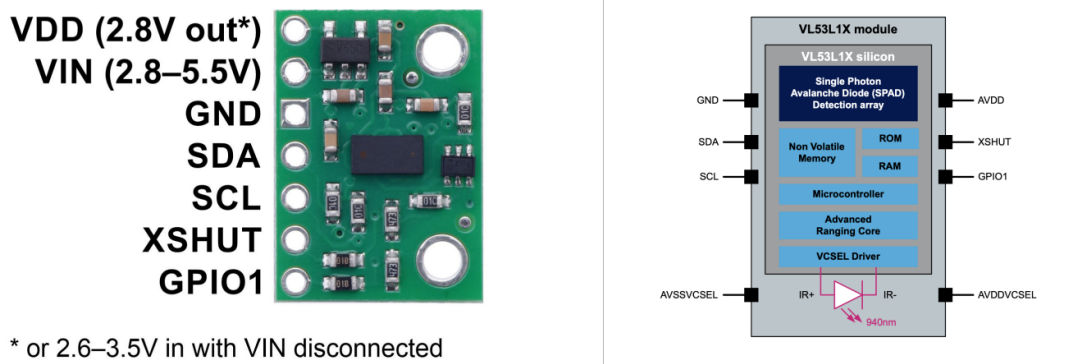


Figura 2.17: *Pin Configuration (sinistra) e System Block Diagram (destra) del VL53L1X*



Figura 2.18: *VL53L1X, dettaglio sull'emettitore*

Field of View e Region of Interest

Il *Field of View* rappresenta l'angolo massimo entro il quale il dispositivo è in grado di rilevare un oggetto; per il sensore in esame, il **FoV** è pari a circa **27°**. Il *VL53L1X* mette inoltre a disposizione la funzione **Region of Interest (R.O.I.)**, che consente di restringere il *Field of View* effettivamente utilizzato durante la misura, risultando utile soprattutto nel caso di oggetti di piccole dimensioni. Nel presente progetto, tuttavia, il sensore è impiegato per la misura dell'altitudine del *DPDF*; di conseguenza, la funzione R.O.I. non è stata utilizzata e il FoV è stato lasciato al valore nominale, come illustrato in Figura 2.19.

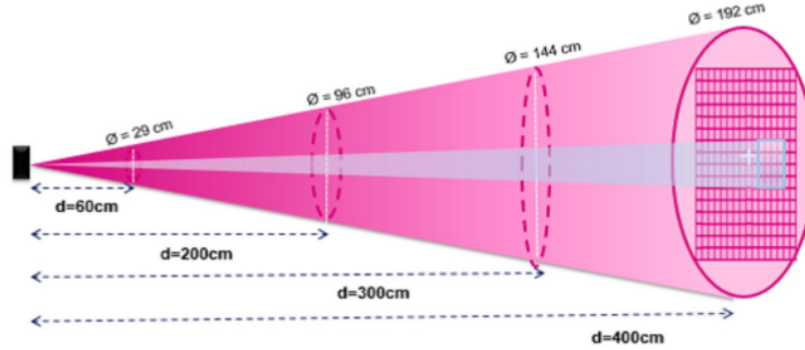


Figura 2.19: *Field of View del VL53L1X*

Prestazioni di misura, Timing Budget e modalità operative di misura

Il *VL53L1X* offre buone prestazioni di misura in termini di accuratezza e rapidità. Il dispositivo è in grado di misurare distanze fino a **4 [m]**, con frequenze che possono arrivare fino a **50 [Hz]**. Per adattarsi a scenari applicativi differenti, il sensore consente di selezionare la modalità di misura più adatta, bilanciando portata, robustezza e velocità di acquisizione. A seguire, una tabella riassuntiva delle modalità di misura offerte dal dispositivo, riportata in Tabella 2.1:

Modalità	Range max in condizioni di scarsa luce (approssimato)	Range max con luce ambientale
Short	136 [cm]	135 [cm]
Medium	290 [cm]	76 [cm]
Long	360 [cm]	73 [cm]

Tabella 2.1: *VL53L1X – Modalità di misura*

La modalità *Short ranging* è progettata per acquisizioni ravvicinate, garantendo letture stabili e robuste anche in ambienti caratterizzati da forte luminosità. Al contrario, la modalità *Long ranging* privilegia la massima portata spaziale, accettando tuttavia misurazioni intrinsecamente più sensibili al rumore ottico. Tra queste due opzioni estreme si colloca la configurazione *Medium ranging*, che offre un compromesso bilanciato tra distanza operativa e robustezza del dato.

Per soddisfare gli specifici requisiti del *DPDF*, la scelta progettuale è ricaduta sulla modalità

Short ranging, che è stata mantenuta come unica configurazione operativa per l'intera durata dello sviluppo e dei test.

Al fine di gestire in modo ottimale il compromesso intrinseco tra velocità di campionamento e precisione della misura, si è fatto ricorso al parametro di **Timing Budget** del sensore. Con tale termine si definisce il tempo complessivo allocato al dispositivo per l'esecuzione di una singola misurazione di distanza. Questa grandezza è espressa in [ms] e può essere configurata su valori compresi nell'intervallo $20 \div 1000$ [ms]. All'aumentare del *Timing Budget* corrisponde fisiologicamente una riduzione della frequenza di acquisizione; tuttavia, un tempo di integrazione ottica più lungo garantisce un netto miglioramento in termini di accuratezza e stabilità della lettura.

La documentazione tecnica ufficiale suggerisce che un *Timing Budget* pari a **33 [ms]** rappresenti un eccellente compromesso tra reattività dinamica e affidabilità del dato; pertanto, la configurazione del firmware è stata impostata su tale valore. A supporto di questa scelta costruttiva, in Figura 2.20 è riportato un grafico estratto direttamente dallo *User Manual* del *VL53L1X*, il quale illustra le prestazioni e l'andamento della misurazione a fronte di diversi valori del *Timing Budget* adottato.

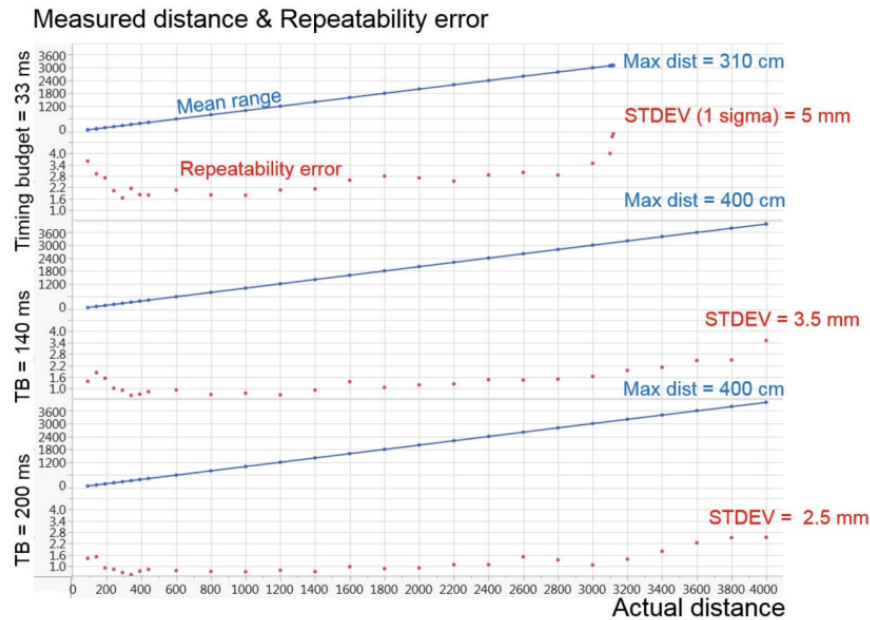


Figura 2.20: *VL53L1X*, prestazioni di misura

2.12 Schema dei collegamenti

Lo schema elettrico complessivo del sistema è riportato in Figura 2.21.

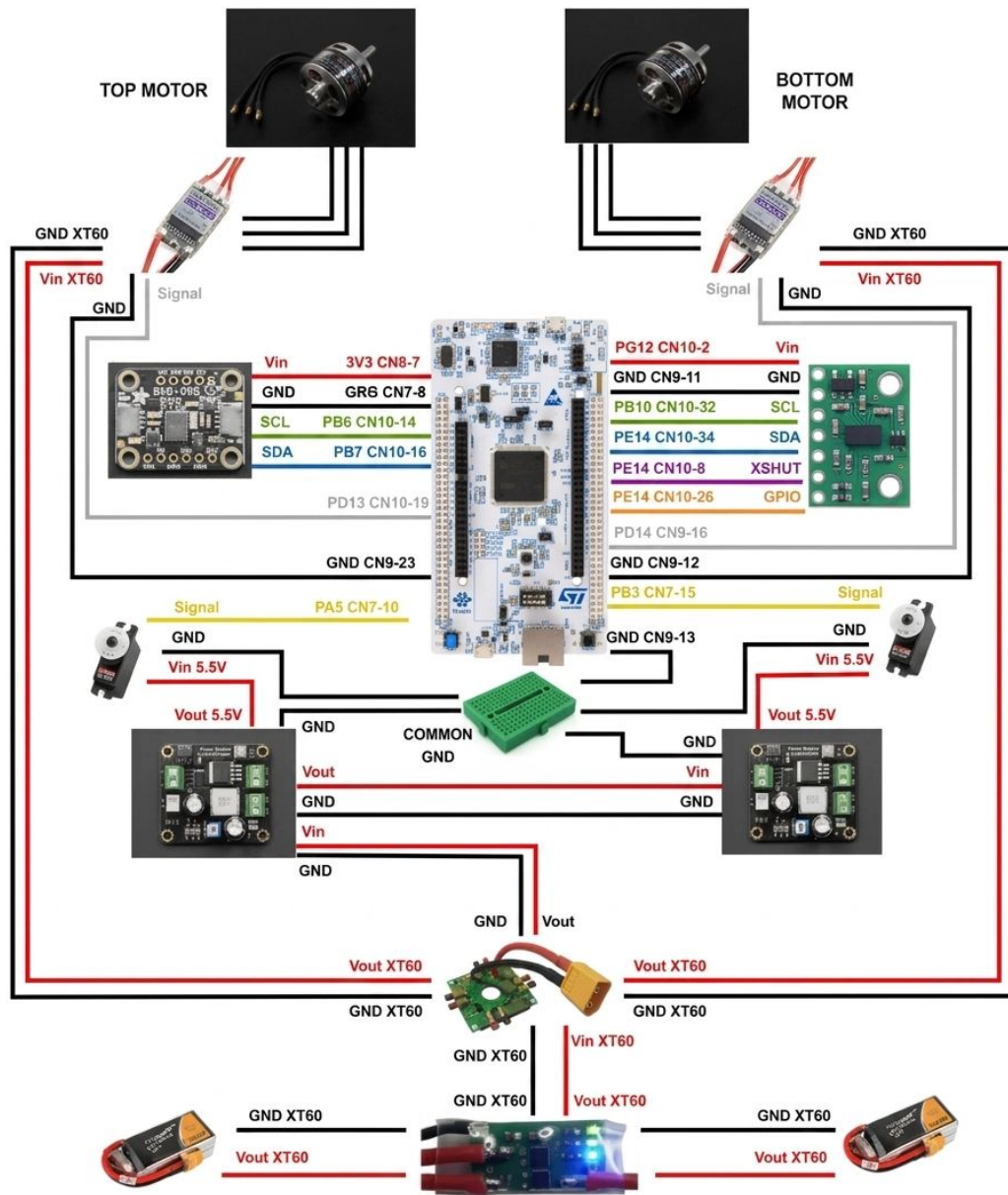


Figura 2.21: Schema dei collegamenti

2.12.1 Tabelle dei collegamenti

Di seguito vengono dettagliati i collegamenti elettrici intercorrenti tra i principali componenti hardware del sistema e la scheda di controllo centrale. Le tabelle presentate in questa sezione sono state richiamate puntualmente nel testo per agevolare la consultazione e garantire la replicabilità dell'architettura hardware.

Collegamenti tra NUCLEO-H745ZI-Q e sensore VL53L1X

Il sensore di distanza *VL53L1X* (STMicroelectronics) è stato interfacciato direttamente al-

la scheda *NUCLEO-H745ZI-Q*. Il dettaglio delle connessioni pin-to-pin è riportato nella Tabella 2.2.

VL53L1X	NUCLEO-H745ZI-Q
Vin	PG12, CN10-2
GND	GND, CN10-5
SCL	SCL I2C2: PB10, CN10-32
SDA	SDA I2C2: PB11, CN10-34
XSHUT	PE14, CN10-8
GPIO	CN10-26

Tabella 2.2: Collegamenti *NUCLEO-H745ZI-Q* e *VL53L1X*

Collegamenti tra **NUCLEO-H745ZI-Q** e sensore **BNO055**

Analogamente, anche l'unità inerziale *BNO055* (Bosch Sensortec) è stata collegata in modo diretto alla piattaforma di controllo. Lo schema di cablaggio adottato è sintetizzato nella Tabella 2.3.

BNO055	NUCLEO-H745ZI-Q
Vin	3.3V, CN8-7
GND	GND, CN8-11
SCL	SCL I2C1: PB6, CN10-14
SDA	SDA I2C1: PB7, CN10-16

Tabella 2.3: Collegamenti *NUCLEO-H745ZI-Q* e *BNO055*

Collegamenti tra convertitori (**DFRobot**), servomotori e **NUCLEO-H745ZI-Q**

Al fine di garantire il corretto azionamento del comparto servomotori, risulta essenziale stabilire un riferimento di massa comune. Per tale ragione, è stata impiegata una *breadboard* esterna funzionante da nodo di distribuzione per la massa (GND) proveniente dalla *NUCLEO-H745ZI-Q* (pin CN7-8). L'alimentazione di potenza dei servomotori è stata invece prelevata direttamente dai terminali di uscita dei convertitori di tensione collegati in serie. Le configurazioni di cablaggio per il servo deputato al controllo del rollio (*roll*) e per quello del beccheggio (*pitch*) sono riportate rispettivamente in Tabella 2.4 e Tabella 2.5.

DF-Robot (Roll)	Roll-servo	Massa Comune	NUCLEO-H745ZI-Q
Vout-3	Vin (5.5V)	-	-
Vout-GND	GND	C.GND	GND, CN7-8
-	Signal	-	TIM2-CH1: PA5, CN7-10

Tabella 2.4: Collegamenti *NUCLEO-H745ZI-Q* e *Roll-servo*

DF-Robot (Pitch)	Pitch-servo	Massa Comune	NUCLEO-H745ZI-Q
Vout-3	Vin (5.5V)	-	-
Vout-GND	GND	C.GND	GND, CN7-8
-	Signal	-	TIM2-CH2: PB3, CN7-15

Tabella 2.5: Collegamenti NUCLEO-H745ZI-Q e Pitch-servo

Collegamenti tra Power Distribution Module, ESC e NUCLEO-H745ZI-Q

Per quanto concerne il comparto propulsivo, gli ESC ricevono l'alimentazione dal *Power Distribution Module*, il quale è a sua volta interfacciato al comparto batterie (PSM) e ai motori tramite connettori trifase dedicati. Anche in questo caso è tassativo che il terminale di massa degli ESC sia equipotenziale rispetto alla massa della *NUCLEO-H745ZI-Q*, mentre il terminale del segnale logico deve essere connesso al rispettivo pin PWM della scheda. I cablaggi afferenti all'ESC del motore superiore (*Top-motor*) e a quello inferiore (*Bottom-motor*) sono illustrati nelle Tabelle 2.6 e 2.7.

Power Distribution Module	Top-motor-ESC	NUCLEO-H745ZI-Q
Vout XT-60	Vin XT-60	-
GND XT-60	GND XT-60	-
-	Signal	TIM4-CH2: PD13, CN10-19
-	GND	GND

Tabella 2.6: Collegamenti NUCLEO-H745ZI-Q e Top-motor-ESC

Power Distribution Module	Bottom-motor-ESC	NUCLEO-H745ZI-Q
Vout XT-60	Vin XT-60	-
GND XT-60	GND XT-60	-
-	Signal	TIM4-CH3: PD14, CN9-16
-	GND	GND

Tabella 2.7: Collegamenti NUCLEO-H745ZI-Q e Bottom-motor-ESC

Nota: il dispositivo che riceve il segnale PWM deve condividere lo stesso riferimento di massa del dispositivo che genera tale segnale.

3 Firmware

Il presente capitolo illustra l'architettura software sviluppata per la gestione e il controllo del *Double Propeller Ducted-Fan (DPDF)*. Il firmware è stato concepito non come un sistema di controllo a sé stante, ma come componente di una pipeline più ampia che comprende l'acquisizione strutturata dei dati, l'identificazione del modello dinamico in ambiente MATLAB e la successiva validazione dei regolatori sul prototipo fisico. Per questa ragione, il microcontrollore non si limita a calcolare l'azione di controllo: deve allo stesso tempo trasmettere al PC ospite un flusso di telemetria stabile, ricevere comandi in tempo reale e garantire la sicurezza del sistema in ogni fase operativa.

Per soddisfare questi rigorosi requisiti, il firmware è stato strutturato secondo un'architettura modulare ed *event-driven*. È infatti imprescindibile che il microcontrollore garantisca:

1. Un campionamento deterministico delle grandezze di stato (angoli di Eulero forniti dalla IMU BNO-055 e quota fornita dal ToF VL53L1X), con *jitter* di fase contenuto;
2. Un sistema di telemetria ad alta frequenza, capace di trasmettere a 50 [Hz] su porta seriale a 230 400 [baud] lo stato completo del sistema, inclusi angoli, quota, comandi PWM applicati agli attuatori, contatori di errore UART e flag di stato;
3. Un protocollo di comando sicuro, affinché si possa testare il drone fisico in sicurezza, per evitare possibili malfunzionamenti e comportamenti anomali;
4. Una gestione robusta della comunicazione UART, con meccanismi di ricezione, parsing, diagnostica degli errori e ripristino della comunicazione;
5. Un'infrastruttura di controllo flessibile, pronta a recepire i guadagni ottimali del PID calcolati in simulazione, per procedere alla validazione sul prototipo fisico in condizioni di massima sicurezza.

Nelle sezioni successive verranno analizzati nel dettaglio l'architettura del software complessiva, la gestione del microcontrollore dual-core, l'organizzazione modulare del codice, la configurazione delle periferiche, la gestione dei sensori, i diagrammi di flusso del sistema, l'architettura di controllo a quattro PID, il protocollo di comunicazione seriale con MATLAB e le principali logiche di sicurezza implementate durante i test sul DPDF reale.

3.1 Architettura software event-driven

L'infrastruttura firmware sviluppata per il DPDF si fonda su un'architettura *event-driven*, ovvero guidata dagli eventi temporali e hardware. A differenza dei paradigmi puramente sequenziali, basati sul *polling* continuo delle periferiche, l'architettura adottata fa sì che l'unità di calcolo intervenga quando si verifica un evento rilevante: la disponibilità di una nuova misura dal sensore di quota, la scadenza di un timer periodico, l'arrivo di un comando dalla porta seriale oppure la pressione di un pulsante fisico di emergenza.

Per garantire questa reattività, le *Interrupt Service Routine* (ISR) e le callback HAL associate a timer e periferiche sono state mantenute estremamente leggere. Il loro compito principale è aggiornare flag logiche, contatori o variabili di stato, senza eseguire calcoli onerosi o accessi prolungati alle periferiche. L'elaborazione vera e propria — acquisizione dati via I^2C , compensazione e filtraggio della quota, aggiornamento dei segnali PWM, gestione dei comandi e composizione della telemetria — viene invece eseguita nel ciclo principale `while(1)`, che monitora lo stato delle flag ed esegue i blocchi computazionali solo quando il rispettivo sottosistema lo richiede.

Questa separazione tra eventi, gestiti in ISR, ed elaborazione, gestita nel main loop, presenta tre vantaggi concreti per il presente progetto. In primo luogo, riduce il tempo di permanenza all'interno delle ISR, lasciando il core libero di gestire gli altri task del firmware. In secondo luogo, limita il rischio di inconsistenze sui dati condivisi, poiché le elaborazioni principali vengono eseguite nel contesto sequenziale del main loop. Infine, rende il sistema più facilmente verificabile: ogni evento aggiorna una flag o una variabile di stato, ogni flag abilita un blocco di codice ben identificabile e l'ordine di esecuzione risulta più semplice da analizzare durante le prove sperimentali.

3.2 Gestione del microcontrollore dual-core

Il microcontrollore *STM32H745ZI-TQ6*, descritto nel Capitolo 2, è caratterizzato da un'architettura *dual-core* che integra un processore *Arm Cortex-M7* e un coprocessore *Arm Cortex-M4*. Sebbene l'intera logica di controllo sia eseguita dal Cortex-M4, anche il Cortex-M7 riveste un ruolo essenziale nella fase di avvio del sistema.

Per ragioni architetturali interne al microcontrollore, la configurazione del clock e l'inizializzazione della *Power Domain* principale sono di esclusiva competenza del Cortex-M7, in quanto core primario. Il Cortex-M4 resta dunque in attesa, in modalità a basso consumo (*Wait For Event*, WFE), fino a quando il Cortex-M7 non gli segnala il completamento di tale configurazione tramite un *Hardware Semaphore* (HSEM). Solo dopo aver ricevuto questo segnale, il Cortex-M4 esce dallo stato di attesa e procede con l'inizializzazione delle proprie periferiche e con l'avvio del main loop di controllo.

Una conseguenza pratica di questa architettura, di particolare rilevanza in fase di sviluppo, riguarda la procedura di *flashing* del firmware. Il binario destinato al Cortex-M7 deve essere caricato prima di quello destinato al Cortex-M4: in caso contrario, il Cortex-M4 rimane indefinitamente bloccato in attesa del segnale HSEM che non verrà mai generato, e il sistema risulta apparentemente non operativo. L'ambiente di sviluppo STM32CubeIDE consente di gestire entrambi i target all'interno dello stesso progetto, garantendo la corretta sequenza di programmazione.

3.3 Organizzazione modulare del codice

Per favorire la leggibilità, la manutenibilità e la verificabilità del firmware, il codice sorgente è stato organizzato in moduli funzionalmente coesi. Ogni modulo è composto, dove previsto, da un *header file* `.h`, che dichiara l'interfaccia pubblica del modulo, e da un *source file* `.c`, che ne contiene l'implementazione. Le configurazioni di basso livello generate da

STM32CubeMX sono state mantenute separate dai moduli applicativi, in modo da non appesantire il file `main.c` con dettagli implementativi non direttamente legati alla logica del sistema.

I principali moduli applicativi sviluppati sono i seguenti:

- **main.c** — contiene la sequenza di inizializzazione del sistema, la procedura di *safe startup*, il ciclo operativo principale basato sulle flag di evento e la chiamata coordinata agli altri moduli. Al suo interno sono inoltre gestite le callback principali associate a timer, GPIO e UART. In particolare, il file `main.c` coordina la ricezione e il parsing dei comandi provenienti da MATLAB, l'aggiornamento dei PWM, la composizione della telemetria e la gestione delle principali condizioni di sicurezza.
- **config.h e config.c** — raccolgono in un unico punto i principali parametri numerici del firmware, tra cui i limiti PWM per motori e servomotori, la posizione neutra dei servo, i fattori di conversione angolo-PWM e la struttura di configurazione dei regolatori. Centralizzare questi valori rende esplicito il confine tra comando richiesto, comando ammissibile e limite imposto dall'hardware reale, evitando la dispersione di valori numerici direttamente all'interno del codice.
- **pwm.h e pwm.c** — incapsulano la gestione dei segnali PWM destinati ai motori e ai servomotori. Il modulo espone funzioni di alto livello per l'avvio dei canali PWM, l'aggiornamento dei comandi e lo spegnimento degli attuatori. In questo modo la logica applicativa può richiamare funzioni come `set_pwm_motors()` e `set_pwm_servos()` senza intervenire direttamente sui registri dei timer.
- **pid.h e pid.c** — implementano la struttura del regolatore PID discreto e alcune funzioni di supporto, tra cui il filtro mediano utilizzato nella catena di acquisizione della quota. Nel firmware attuale le strutture PID vengono inizializzate e mantenute nel codice come predisposizione per successive integrazioni, ma non vengono utilizzate per generare i PWM durante le prove open-loop comandate da MATLAB.
- **imu.h e imu.c** — costituiscono un livello di astrazione sopra il driver del sensore BNO055. Il modulo si occupa dell'inizializzazione dell'IMU, della lettura degli angoli di Eulero e della gestione del riferimento iniziale, così da ottenere valori relativi di roll, pitch e yaw rispetto alla posa iniziale del DPDF.
- **tof.h e tof.c** — contengono le funzioni di supporto associate alla misura di quota fornita dal VL53L1X. In particolare, il modulo implementa la compensazione geometrica della distanza misurata dal sensore rispetto all'inclinazione del velivolo, utilizzando gli angoli di roll e pitch forniti dall'IMU.
- **Driver BNO055 e VL53L1X** — oltre ai moduli applicativi, il progetto include i driver necessari all'interfacciamento con i sensori. I file relativi al BNO055 e alla libreria VL53L1X forniscono le funzioni di basso livello per la comunicazione con i dispositivi, mentre i moduli `imu.c` e `tof.c` ne semplificano l'utilizzo all'interno della logica del firmware.

Questa suddivisione consente, in fase di sviluppo, di intervenire su un modulo alla volta riducendo il rischio di introdurre regressioni in funzionalità già validate. Le modifiche al

firmware sono state pertanto effettuate come modifiche puntuali e mirate, conservando per quanto possibile la struttura e le interfacce esistenti.

3.4 Configurazione delle periferiche

La configurazione hardware del Cortex-M4 è stata generata tramite STM32CubeMX a partire dal file di progetto `.ioc`, ed è stata mantenuta coerente con i collegamenti pin-to-pin descritti nel Capitolo 2. Si descrivono di seguito le periferiche principali impiegate dal firmware.

3.4.1 Timer

I timer impiegati svolgono ruoli funzionalmente distinti:

- **TIM2**: configurato in modalità *PWM Generation*, genera sui canali CH1 e CH2 i segnali di controllo dei servomotori di roll e pitch. La frequenza di funzionamento è pari a 50 [Hz], coerentemente con le specifiche dei servomotori HS-82MG.
- **TIM4**: anch'esso configurato in modalità *PWM Generation*, genera sui canali CH2 e CH3 i segnali destinati agli ESC Turnigy Plush 40A che pilotano i due motori brushless. Anche in questo caso la frequenza è pari a 50 [Hz], come richiesto dagli ESC stessi.
- **TIM3**: utilizzato come base temporale per la generazione periodica dei campioni di telemetria. A ogni interrupt viene aggiornato il tempo telemetrico e viene incrementato il numero di campioni pendenti da trasmettere. Questa scelta permette di separare la temporizzazione della telemetria dalla trasmissione effettiva via UART, che viene invece gestita nel ciclo principale.
- **TIM6**: impiegato per la sequenza temporizzata di *safe startup* e per il pilotaggio diagnostico dei LED integrati sulla scheda.
- **TIM7**: utilizzato come *timebase* per la generazione periodica dell'interrupt che abilita l'aggiornamento del ramo IMU-attuatori. La sua scadenza imposta la flag `actuate_servo_control`, che il main loop intercetta per eseguire la lettura del BNO055, aggiornare i valori operativi dei servomotori e applicare i PWM agli attuatori. Nella configurazione di test open-loop i PWM sono ricevuti da MATLAB; la stessa struttura temporale rimane tuttavia compatibile con la successiva integrazione dei PID di assetto, tarati a partire dal modello identificato.

```
1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2 {
3     if (htim->Instance == TIM6) {
4         HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
5         current_number_of_toggles++;
6     }
7     else if (htim->Instance == TIM7) {
8         actuate_servo_control = true;
9     }
10    else if (htim->Instance == TIM3) {
```

```

11     telemetry_time_ms += TELEMETRY_PERIOD_MS;
12 if (telemetry_pending < TELEMETRY_MAX_PENDING) {
13     telemetry_pending++;
14 } else {
15     telemetry_overrun++;
16 }
17 }
18 }
19
20 }

```

Listing 3.1: Callback periodica dei timer utilizzati per safe startup, ramo IMU–attuatori e telemetria.

I valori di Prescaler e Auto-Reload Register (ARR) di ciascun timer sono stati calcolati a partire dalla frequenza di clock del bus APB1, configurata a 75 [MHz], secondo la relazione standard:

$$f_{PWM} = \frac{f_{clk}}{(Prescaler + 1) \cdot (ARR + 1)} \quad (3.1)$$

Nel caso dei timer PWM, tale relazione determina la frequenza del segnale generato; nel caso dei timer usati come *timebase*, determina invece la frequenza con cui vengono generati gli interrupt periodici.

3.4.2 Periferiche di comunicazione

Il firmware utilizza due tipologie principali di periferiche di comunicazione:

- **USART3:** configurata in modalità *full-duplex* asincrona a 230 400 [baud], con 8 bit di dato, 1 bit di stop e nessuna parità. Questo canale costituisce il collegamento bidirezionale con il PC ospite: in trasmissione veicola la telemetria seriale, mentre in ricezione raccoglie i comandi inviati da MATLAB. La scelta di un baud rate elevato è motivata dalla necessità di sostenere sia il flusso telemetrico sia l'invio asincrono dei comandi, considerando che ogni pacchetto trasmesso è costituito da una stringa CSV di lunghezza variabile.
- **I²C1 e I²C2:** configurati in *Fast Mode* a 400 [kHz], sono dedicati rispettivamente alla comunicazione con il sensore BNO055 e con il sensore VL53L1X. L'impiego di due bus distinti, anziché di un unico bus condiviso, evita possibili conflitti e permette di gestire i due sensori in modo indipendente.

La gestione degli interrupt associati a queste periferiche è affidata al *Nested Vectored Interrupt Controller* (NVIC), configurato con livelli di priorità coerenti con la criticità temporale di ciascuna sorgente di evento.

3.5 Gestione dei sensori

3.5.1 VL53L1X — Acquisizione di quota

L'acquisizione della quota avviene tramite il sensore Time-of-Flight VL53L1X, descritto nel Capitolo 2. Il firmware utilizza il driver ufficiale fornito da STMicroelectronics, integrato attraverso un sottile livello di adattamento alle funzioni HAL.

La sequenza di inizializzazione del sensore prevede: il rilascio del segnale XSHUT, il caricamento del firmware interno tramite `VL53L1_WaitDeviceBooted()`, l'inizializzazione tramite `VL53L1_DataInit()`, l'impostazione della modalità *Short ranging* e del *Timing Budget* a 33 [ms], e infine l'avvio della misura continua tramite `VL53L1_StartMeasurement()`.

Per non bloccare il microcontrollore durante il tempo di integrazione ottica, l'acquisizione segue un approccio basato sulla disponibilità del dato. Quando il sensore segnala la presenza di una nuova misura, il firmware imposta la relativa flag di acquisizione; la lettura effettiva viene poi eseguita nel ciclo principale. In questo modo la callback rimane leggera e il main loop può procedere con la lettura della misura tramite `VL53L1_GetRangingMeasurementData()` e con il successivo riavvio dell'acquisizione tramite `VL53L1_ClearInterruptAndStartMeasurement()`.

La misura grezza così ottenuta viene successivamente compensata rispetto all'inclinazione istantanea del velivolo, in modo da fornire al regolatore di quota una stima coerente con la variabile fisica realmente controllata — la coordinata verticale — piuttosto che con la distanza grezza misurata dal sensore lungo il proprio asse.

3.5.2 BNO055 — Acquisizione dell'assetto

Il BNO055 è responsabile del monitoraggio dell'assetto tridimensionale del velivolo. Il firmware utilizza il driver *platform-agnostic* fornito da Bosch Sensortec, adattato al contesto HAL per le funzioni di comunicazione *I²C*, di temporizzazione e di gestione del reset.

La sequenza di inizializzazione del sensore comprende: la verifica del corretto collegamento del dispositivo, la configurazione dei parametri interni, l'impostazione del remap degli assi e dei relativi segni, e l'impostazione della modalità operativa. Come anticipato nel Capitolo 2, è stata scelta la modalità di fusion NDOF, in cui l'algoritmo proprietario del sensore combina i dati provenienti da accelerometro, giroscopio e magnetometro per restituire una stima stabile e robusta dell'orientamento assoluto.

Subito dopo l'inizializzazione, e prima dell'avvio del ciclo di controllo, il firmware acquisisce un campione di posa iniziale tramite le funzioni del modulo IMU, che memorizzano gli angoli di Eulero correnti come riferimento. Tutte le letture successive vengono espresse come scostamento rispetto a questa posa iniziale: in questo modo i PID di assetto lavorano su angoli relativi alla configurazione di equilibrio meccanico in cui il drone si trova all'accensione, evitando la dipendenza dall'orientamento magnetico assoluto e semplificando il confronto con la simulazione.

Le letture periodiche degli angoli avvengono nel ramo IMU-servo a 50 [Hz], frequenza coerente con la massima banda passante della *sensor fusion* interna al BNO055 in modalità NDOF. I valori di roll, pitch e yaw vengono mantenuti internamente come grandezze angolari e, al momento della composizione della telemetria, vengono convertiti in centesimi di grado. Questa rappresentazione consente di trasmettere informazione angolare con precisione di 0.01° utilizzando soli interi nella stringa di telemetria.

3.6 Diagrammi di flusso

Per illustrare in modo sintetico la logica decisionale e le dipendenze temporali dei diversi task del firmware, vengono di seguito presentati tre diagrammi di flusso: due dedicati alla gestione dei singoli sensori, e uno che sintetizza l'esecuzione complessiva del firmware.

3.6.1 Diagramma di flusso del VL53L1X

Il diagramma di Figura 3.1 riassume la sequenza di inizializzazione e di acquisizione del sensore di quota. Il flusso si conclude con l'attivazione della modalità di misura continua, dopodiché il main loop interroga periodicamente la disponibilità del dato senza bloccarsi.

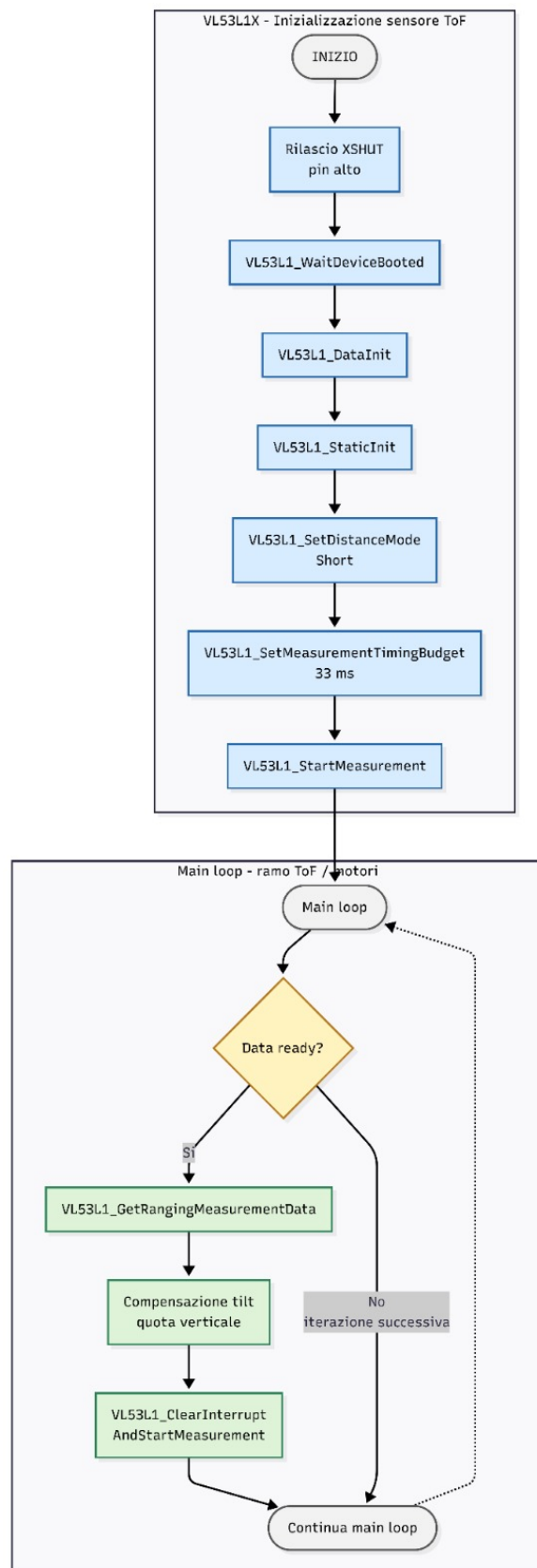


Figura 3.1: Diagramma di flusso del sensore VL53L1X

3.6.2 Diagramma di flusso del BNO055

Il diagramma di Figura 3.2 illustra la sequenza di astrazione dell'API Bosch. Particolarmente rilevante è il blocco dedicato all'acquisizione della posa iniziale (DPDF_BNO055_firmware_read_init), che fissa il sistema di riferimento angolare rispetto al quale tutte le misure successive saranno espresse.

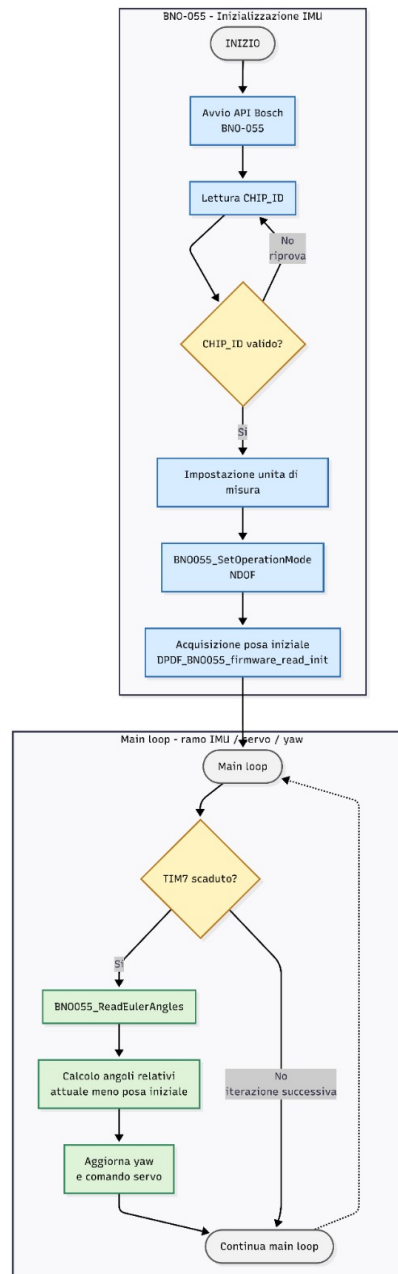


Figura 3.2: Diagramma di flusso del sensore BNO055

3.6.3 Diagramma di flusso del firmware complessivo

Il diagramma di Figura 3.3 sintetizza la cooperazione tra i moduli software. Il flusso si articola in due macro-fasi:

1. **Inizializzazione e safe startup:** il sistema attende un trigger esterno di sicurezza (User button) prima di avviare l'alimentazione ai sensori e di generare i segnali di armamento per gli ESC dei motori.
2. **Main loop di controllo:** il ciclo infinito gestisce due percorsi asincroni e indipendenti. Il primo, attivato dalla scadenza di TIM7, gestisce la correzione di assetto tramite i servomotori (roll e pitch) e la correzione differenziale di yaw sui motori. Il secondo, attivato dalla disponibilità di un nuovo dato dal ToF, gestisce la portanza verticale dei motori principali. La separazione di questi due percorsi assicura che un eventuale ritardo di un sensore non comprometta la reattività dell'intero sistema.

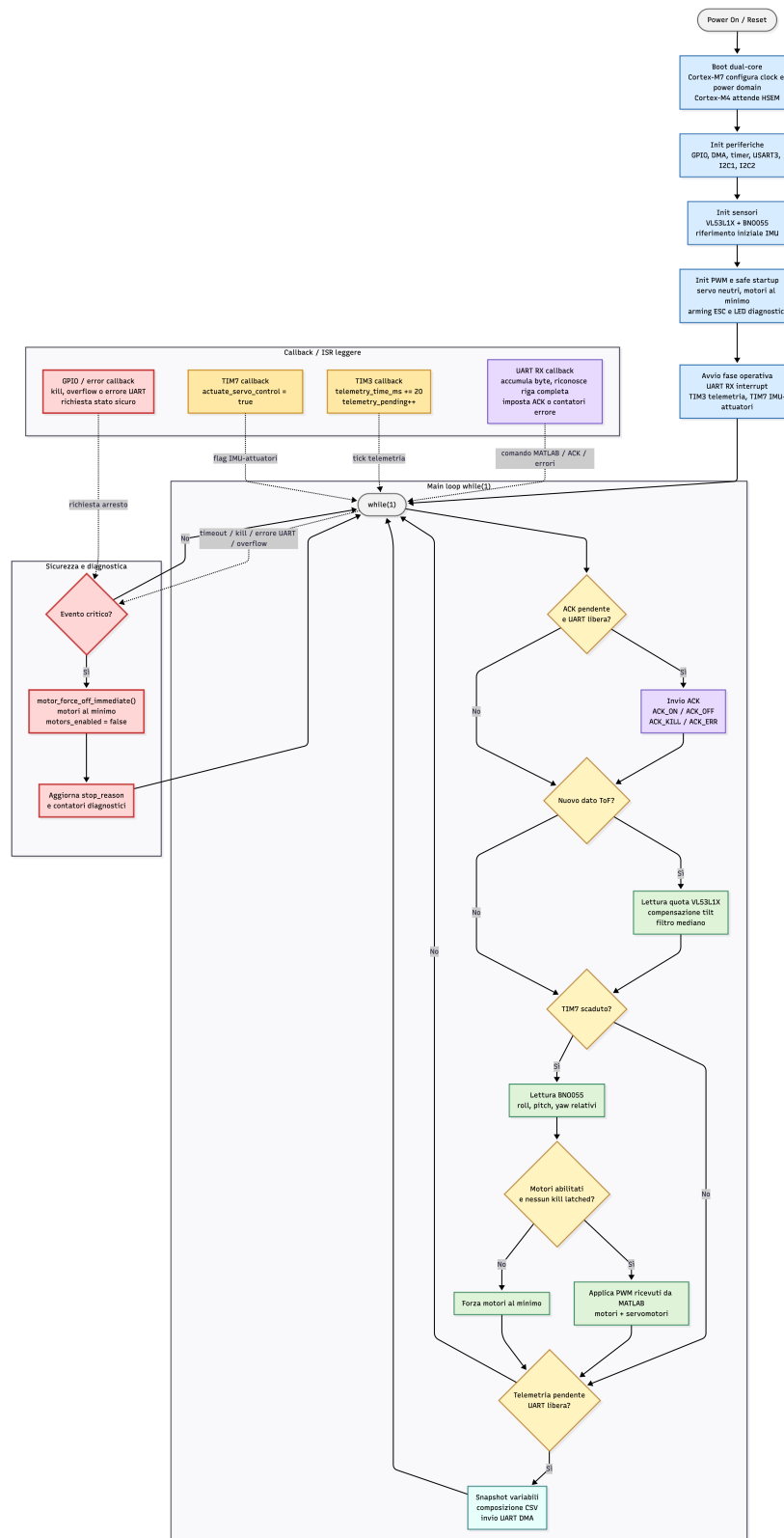


Figura 3.3: Diagramma di flusso complessivo del firmware

3.7 Protocollo di comunicazione seriale con MATLAB

La comunicazione tra MATLAB e il firmware avviene tramite un protocollo testuale bidirezionale sulla porta seriale USART3, configurata a 230 400 [baud]. Questo canale costituisce il collegamento principale tra l'ambiente di acquisizione off-board e il prototipo fisico: MATLAB invia i comandi da applicare agli attuatori, mentre il firmware li riceve, li interpreta, li valida e restituisce in telemetria lo stato effettivamente raggiunto dal sistema.

La UART è configurata in modalità *full-duplex* asincrona, con 8 bit di dato, 1 bit di stop e nessuna parità, così da sostenere sia l'invio dei comandi da MATLAB sia la trasmissione periodica della telemetria. La ricezione dei comandi è gestita direttamente nel firmware tramite la callback UART: i byte ricevuti vengono accumulati in un buffer fino al carattere di terminazione, `\n` oppure `\r`; una volta completata la riga, il comando viene interpretato e validato.

La descrizione dettagliata del formato di telemetria e delle modalità di acquisizione lato PC è rimandata al Capitolo 4; in questa sezione vengono invece illustrati i comandi che il firmware accetta in ricezione e le relative risposte di conferma.

3.7.1 Comandi ricevuti dal firmware

Il firmware riconosce quattro comandi distinti, terminati dal carattere di nuova riga (LF):

- **CMD,top,bottom,roll,pitch** — comando di attuazione utilizzato durante le prove sperimentali. I quattro campi numerici rappresentano, nell'ordine: PWM del motore superiore, PWM del motore inferiore, PWM del servo di roll, PWM del servo di pitch. Tutti i valori sono espressi in unità CCR (*Capture-Compare Register*) e vengono saturati in lettura entro i limiti hardware definiti in `config.h`. In particolare, `LOWER_LIMIT_MOTOR` e `UPPER_LIMIT_MOTOR` delimitano l'intervallo PWM ammesso per gli ESC dei motori, mentre `LOWER_LIMIT_SERVO`, `CENTER_SERVO` e `UPPER_LIMIT_SERVO` definiscono rispettivamente il limite inferiore, la posizione neutra e il limite superiore dei servomotori associati ai flap. Le costanti `CCR_PER_DEGREE` e `MAX_FLAP_ANGLE_DEG` permettono invece di collegare il comando PWM alla corrispondente escursione angolare dei flap, rendendo esplicito il vincolo meccanico imposto agli attuatori. I valori dei servomotori vengono aggiornati dopo la saturazione, mentre i valori dei motori vengono applicati solo se i motori risultano abilitati e non è presente una condizione di *kill* latched.

```
1 else if (strncmp(rx_buffer, "CMD,", 4) == 0) {
2
3     int top, bottom, roll, pitch;
4
5     if (sscanf(rx_buffer, "CMD,%d,%d,%d,%d",
6               &top, &bottom, &roll, &pitch) == 4) {
7
8         if (top < LOWER_LIMIT_MOTOR) top = LOWER_LIMIT_MOTOR;
9         if (top > UPPER_LIMIT_MOTOR) top = UPPER_LIMIT_MOTOR;
10        if (bottom < LOWER_LIMIT_MOTOR) bottom = LOWER_LIMIT_MOTOR;
11        if (bottom > UPPER_LIMIT_MOTOR) bottom = UPPER_LIMIT_MOTOR;
12
13        if (roll < LOWER_LIMIT_SERVO) roll = LOWER_LIMIT_SERVO;
14        if (roll > UPPER_LIMIT_SERVO) roll = UPPER_LIMIT_SERVO;
```

```

15         if (pitch < LOWER_LIMIT_SERVO) pitch = LOWER_LIMIT_SERVO;
16         if (pitch > UPPER_LIMIT_SERVO) pitch = UPPER_LIMIT_SERVO;
17
18         ext_roll_pwm = (uint16_t) roll;
19         ext_pitch_pwm = (uint16_t) pitch;
20     }
21 }
22
23 }

```

Listing 3.2: Parsing del comando CMD e saturazione dei valori PWM ricevuti da MATLAB.

- **MOTOR_ON** — richiede al firmware di passare dallo stato disarmato allo stato armato. In risposta, il firmware abilita la possibilità di applicare i successivi comandi PWM ai motori, mantenendo inizialmente i valori al limite inferiore, e risponde con **ACK_ON**.
- **MOTOR_OFF** — richiede il ritorno allo stato disarmato. Il firmware forza i motori al PWM minimo, risponde con **ACK_OFF** e disabilita l'esecuzione dei comandi CMD successivi fino al prossimo **MOTOR_ON**.
- **KILL** — comando di emergenza che ferma immediatamente i motori, indipendentemente dallo stato corrente, e attiva una condizione di arresto *latched*. In tale stato i comandi motore non possono essere applicati fino a una nuova abilitazione esplicita del sistema. Il firmware risponde con **ACK_KILL**.

Qualsiasi linea ricevuta che non corrisponda a uno di questi quattro formati genera una risposta **ACK_ERR**, utile lato PC per individuare rapidamente errori di sintassi nel canale.

La saturazione software rappresenta un elemento essenziale della robustezza del protocollo. MATLAB può generare profili di eccitazione anche molto variabili, ma il firmware non applica mai direttamente valori fuori dal range ammesso dall'hardware. In questo modo il protocollo seriale non viene interpretato come un accesso diretto agli attuatori, ma come una richiesta che il microcontrollore deve validare prima dell'applicazione.

3.7.2 ACK, watchdog e diagnostica UART

Il sistema di *acknowledgement* consente a MATLAB di verificare che i comandi inviati siano stati ricevuti e interpretati correttamente dal firmware. A ogni comando di stato il firmware associa una risposta specifica: **ACK_ON** per l'abilitazione dei motori, **ACK_OFF** per lo spegnimento controllato, **ACK_KILL** per l'arresto di emergenza e **ACK_ERR** per righe non valide o condizioni di errore.

```

1 typedef enum {
2     ACK_NONE = 0,
3     ACK_ON,
4     ACK_OFF,
5     ACK_KILL,
6     ACK_ERR
7 } ack_t;
8
9 typedef enum {
10     STOP_NONE = 0,
11     STOP_CMD_TIMEOUT,

```

```

12     STOP_UART_ERROR,
13     STOP_RX_OVERFLOW,
14     STOP_MOTOR_OFF,
15     STOP_KILL_CMD,
16     STOP_BUTTON
17 } stop_reason_t;
18
19 volatile ack_t ack_pending = ACK_NONE;
20 volatile stop_reason_t stop_reason = STOP_NONE;
21
22 }

```

Listing 3.3: Enumerazioni utilizzate per gli acknowledge e per la codifica delle cause di arresto.

Gli *acknowledge* vengono gestiti tramite una variabile di stato dedicata e trasmessi nel ciclo principale solo quando la UART risulta libera. In questo modo la risposta al comando non viene inviata direttamente all'interno della callback di ricezione, mantenendo leggera la ISR e riducendo il rischio di interferenze con la trasmissione periodica della telemetria. Quando è presente un *acknowledge* pendente, esso viene inviato con priorità rispetto alla telemetria, così da permettere a MATLAB di sincronizzarsi rapidamente con lo stato interno del firmware.

A protezione della comunicazione è inoltre previsto un *watchdog* sui comandi provenienti da MATLAB. Quando i motori sono abilitati, il firmware verifica che i comandi di attuazione continuino ad arrivare entro una finestra temporale prefissata. Se tale condizione non viene rispettata, la mancanza di aggiornamenti viene interpretata come possibile perdita di comunicazione o blocco lato MATLAB; di conseguenza, il firmware forza i motori al valore minimo e aggiorna la causa dell'arresto.

La diagnostica UART permette inoltre di distinguere diverse condizioni anomale legate alla comunicazione seriale. Il contatore `uart_bad_lines` tiene traccia delle righe ricevute che non rispettano il formato previsto dal protocollo, `uart_rx_overflows` segnala eventuali saturazioni del buffer di ricezione, mentre `uart_rx_errors` registra errori hardware della periferica UART. Queste informazioni vengono incluse nella telemetria e consentono di riconoscere, durante l'analisi dei dati, se un comportamento anomalo del sistema sia dovuto alla dinamica fisica del DPDF oppure a un problema di comunicazione.

Nel complesso, il protocollo seriale non si limita a trasferire comandi numerici da MATLAB al firmware, ma introduce un livello di controllo sullo stato della comunicazione. Gli *acknowledge*, il *watchdog* e i contatori diagnostici permettono di rendere osservabile e verificabile l'interazione tra PC e microcontrollore durante l'intera prova sperimentale.

3.8 Telemetria robusta a frequenza fissa

La telemetria rappresenta uno degli elementi centrali del firmware sviluppato, poiché consente di trasformare il DPDF in una piattaforma sperimentale osservabile. L'obiettivo non è soltanto trasmettere le grandezze misurate, ma generare un flusso dati periodico, coerente e diagnosticabile, utilizzabile nelle successive fasi di identificazione, modellazione e simulazione in MATLAB.

Nel firmware, il periodo nominale della telemetria è definito dalla macro `TELEMETRY_PERIOD_MS`, posta pari a 20 [ms]. Tale valore corrisponde a una frequenza di 50 [Hz], scelta per acquisire

campioni sufficientemente ravvicinati nel tempo senza saturare eccessivamente il canale seriale. La dimensione massima della stringa trasmessa è invece definita da `STANDARD_MESSAGE_LENGTH`, mentre `TELEMETRY_MAX_PENDING` limita il numero massimo di campioni che possono rimanere in attesa di trasmissione.

La generazione del *tick* telemetrico è affidata al timer TIM3. A ogni interrupt periodico, il firmware non costruisce né trasmette direttamente la stringa dati, ma aggiorna il tempo della telemetria e incrementa il numero di campioni pendenti. Questa scelta mantiene leggera la routine di interrupt, evitando di eseguire all'interno della ISR operazioni più onerose come la formattazione della stringa CSV o la trasmissione UART.

La trasmissione effettiva viene gestita nel ciclo principale. Il firmware procede all'invio solo se sono presenti campioni pendenti, se la UART non è occupata e se non ci sono messaggi di *acknowledge* da trasmettere con priorità. Se la UART rimane occupata o la trasmissione DMA precedente non è ancora conclusa, i campioni vengono accumulati fino al limite definito da `TELEMETRY_MAX_PENDING`. Al superamento di tale limite viene incrementato il contatore `telemetry_overrun`, successivamente inserito nella telemetria stessa. In questo modo eventuali problemi di temporizzazione o saturazione del canale seriale diventano informazioni diagnostiche disponibili durante l'analisi dei dati.

La riga telemetrica viene costruita in formato CSV tramite `snprintf()` e contiene sia le grandezze fisiche del sistema sia informazioni relative ai comandi applicati e allo stato diagnostico del firmware. Il formato utilizzato è il seguente:

```

1  if (telemetry_pending > 0 &&
2      !uart_tx_busy &&
3      ack_pending == ACK_NONE) {
4
5      telemetry_pending--;
6
7      int len = snprintf(telemetry_tx_buffer,
8                          sizeof(telemetry_tx_buffer),
9                          "%lu,%d,%d,%d,%d,%d,%d,%u,%u,%d,%lu,%lu,%lu,%lu,%lu\n",
10                         sample_time,
11                         roll_cdeg_snapshot,
12                         pitch_cdeg_snapshot,
13                         yaw_cdeg_snapshot,
14                         alt_mm_snapshot,
15                         top_pwm_snapshot,
16                         bottom_pwm_snapshot,
17                         roll_pwm_snapshot,
18                         pitch_pwm_snapshot,
19                         motors_enabled_snapshot,
20                         uart_rx_errors,
21                         uart_rx_overflows,
22                         uart_bad_lines,
23                         telemetry_overrun,
24                         stop_reason);
25
26      if (len > 0 && len < (int)sizeof(telemetry_tx_buffer)) {
27          uart_tx_busy = true;
28          HAL_UART_Transmit_DMA(&huart3,
29                                (uint8_t*)telemetry_tx_buffer,
30                                (uint16_t)len);
31      }

```

```

32 }
33
34 }

```

Listing 3.4: Composizione della riga telemetrica e trasmissione non bloccante tramite DMA.

La presenza di campi diagnostici come `uart_rx_errors`, `uart_rx_overflows`, `uart_bad_lines`, `telemetry_overnrun` e `stop_reason` permette di distinguere un comportamento fisico del drone da un eventuale problema di comunicazione o acquisizione. In questo modo ogni prova sperimentale può essere analizzata mettendo in relazione lo stato del DPDF, i comandi effettivamente applicati e la qualità del collegamento seriale.

La trasmissione della riga avviene tramite DMA, utilizzando `HAL_UART_Transmit_DMA()`. Questa scelta evita di bloccare il ciclo principale durante l'invio dei dati seriali: una volta avviata la trasmissione, il firmware imposta la variabile `uart_tx_busy` e continua la propria esecuzione. Al termine dell'invio, la callback di completamento riporta `uart_tx_busy` a `false`, rendendo nuovamente disponibile il canale UART.

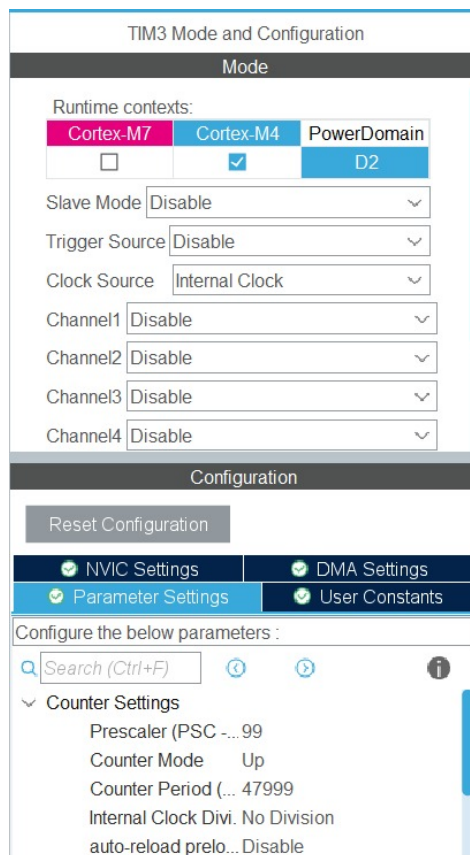


Figura 3.4: Configurazione di TIM3 per la generazione del tick telemetrico

Nel complesso, la telemetria implementata non si limita a inviare periodicamente misure sensoriali, ma costituisce un meccanismo di osservazione e diagnostica del test. La temporizzazione tramite TIM3, l'accumulo controllato dei campioni pendenti, lo *snapshot* coerente

delle variabili, la trasmissione DMA e l'inclusione di informazioni diagnostiche permettono di ottenere dataset più affidabili e interpretabili per le successive elaborazioni in MATLAB.

3.9 Safe startup e logiche di arresto

L'avvio del firmware è gestito tramite una sequenza di *safe startup*, eseguita prima della fase operativa vera e propria. Lo scopo di questa procedura è portare il sistema in una condizione iniziale prevedibile, evitando che i motori ricevano comandi non desiderati durante il transitorio di accensione e inizializzazione.

Durante questa fase vengono avviati i segnali PWM e i motori vengono mantenuti al valore minimo di comando, così da consentire agli ESC di riconoscere correttamente il segnale di armamento. La procedura è accompagnata da segnalazioni diagnostiche tramite LED, utili per fornire all'operatore un'indicazione visiva dello stato di avanzamento dell'avvio.

Terminata la fase di avvio, il firmware mantiene comunque attivi diversi livelli di protezione. I valori PWM ricevuti da MATLAB vengono applicati ai motori solo se questi risultano abilitati e se non è presente una condizione di arresto *latched*. In caso contrario, il firmware forza entrambi i motori al valore minimo, impedendo che un comando precedente o non valido possa mantenere attiva la propulsione.

La procedura di spegnimento immediato dei motori è centralizzata nella funzione

```
motor_force_off_immediate(),
```

richiamata nelle condizioni critiche. Tale funzione riporta i PWM dei motori al limite inferiore, disabilita lo stato logico di abilitazione e riallinea le variabili interne al comando effettivamente applicato agli ESC. In questo modo il comportamento di arresto rimane uniforme indipendentemente dalla causa che lo ha generato.

Il firmware prevede diversi meccanismi di arresto. Il comando `MOTOR_OFF` esegue uno spegnimento controllato, mentre il comando `KILL` rappresenta un arresto di emergenza e imposta una condizione *latched* che impedisce la riattivazione dei motori senza una nuova abilitazione esplicita. A questi comandi software si aggiunge il pulsante fisico della scheda, che consente di interrompere la prova anche indipendentemente dal canale MATLAB-UART.

Le principali cause di arresto o anomalia vengono codificate nella variabile `stop_reason`, trasmessa successivamente in telemetria. Questo permette di distinguere, durante l'analisi dei dati, una prova terminata volontariamente da una prova interrotta per errore UART, overflow del buffer, timeout dei comandi o intervento di emergenza.

Nel complesso, la *safe startup* e le logiche di arresto completano il protocollo di comando e la telemetria, garantendo che il firmware possa riportare il DPDF in una condizione sicura sia all'avvio sia durante l'esecuzione dei test sperimentali.

4 Telemetria

Il presente capitolo descrive l'infrastruttura di telemetria sviluppata per il *Double Propeller Ducted-Fan (DPDF)*: il canale di comunicazione, il protocollo di scambio dati con la *Ground Station*, le logiche di acquisizione lato MATLAB e il profilo dei segnali di eccitazione impiegati durante le prove sperimentali. La descrizione completa il quadro del firmware introdotto nel Capitolo 3 e prepara il terreno per la fase di identificazione del modello matematico, oggetto del Capitolo 5.

4.1 Ruolo della telemetria nel progetto

Nel contesto dello sviluppo di un velivolo intrinsecamente instabile come il *DPDF*, la telemetria non costituisce un semplice strumento di monitoraggio passivo, ma rappresenta uno degli elementi centrali dell'intera pipeline di sviluppo. La necessità di un flusso di dati bidirezionale e ad alta frequenza nasce da tre obiettivi distinti, tutti perseguiti in parallelo durante le campagne sperimentali in laboratorio:

1. **Identificazione del modello matematico (anello aperto)**: il calcolo delle funzioni di trasferimento tramite *System Identification* richiede di registrare con precisione il legame temporale tra gli ingressi forzati (ad esempio le sequenze pseudo-casuali multi-livello imposte ai *flap*) e le risposte cinematiche del drone (angoli di *roll*, *pitch* e *yaw*). Tale procedura, descritta nel Capitolo 5, ha senso solo se i campioni di ingresso e uscita sono allineati nel tempo entro pochi millisecondi.
2. **Validazione dei controllori (anello chiuso)**: una volta sintetizzati i regolatori PID tramite *pidtune*, il loro comportamento sul prototipo fisico deve essere verificato osservando tempi di assestamento, sovraelongazioni e fenomeni di saturazione dei *flap* a $\pm 30^\circ$. Senza una telemetria affidabile, qualsiasi affermazione sulla qualità dei guadagni resterebbe puramente qualitativa.
3. **Sicurezza in laboratorio**: il prototipo, con una massa assemblata di $m \approx 2.2$ [kg] e due eliche *APC 9×4.5* che ruotano a regimi elevati, presenta rischi meccanici non trascurabili. Il fatto di operarlo all'interno di una gabbia vincolata da quattro funi non elimina questi rischi, ma li circoscrive. La telemetria, in questo contesto, fornisce all'operatore una via di terminazione d'emergenza software (*Kill Switch*) che integra le protezioni hardware degli ESC.

Questi tre obiettivi hanno guidato l'intera architettura del canale di comunicazione e dello *script* di acquisizione lato MATLAB, nei termini descritti nelle sezioni successive.

4.2 Architettura del canale di comunicazione

Il flusso telematico si appoggia su un'architettura di tipo *master-slave* cooperativa tra l'unità di calcolo a bordo del drone e la *Ground Station* di laboratorio.

- **Segmento di bordo:** la scheda *STM32 NUCLEO-H745ZI-Q* interroga il sensore IMU *BNO-055* per campionare l'orientamento in centesimi di grado, riceve dal sensore ToF *VL53L1X* la misura di quota in millimetri, e impacchetta queste informazioni assieme allo stato dei motori e dei servomotori in una stringa CSV. La logica di confezionamento dei pacchetti è descritta nel dettaglio nel Capitolo 3.
- **Canale di comunicazione:** la trasmissione avviene su periferica *USART3*, esposta all'esterno attraverso il *Virtual COM Port (VCP)* dell'*ST-LINK/V3E* integrato sulla scheda. Il *baudrate* è stato impostato a **230 400 [bps]**, valore scelto come compromesso tra occupazione del canale e margine operativo. Con un *payload* medio di circa 60 caratteri per pacchetto (15 campi separati da virgole) trasmesso a 50 [Hz], il *bitrate* richiesto in regime UART 8N1 è di circa 30 000 [bps]: il *baudrate* adottato lascia dunque un'occupazione effettiva del canale attorno al 13%, sufficiente ad assorbire i picchi dovuti ai messaggi di stato (*ACK_ON*, *ACK_OFF*, *ACK_KILL*) e a garantire un margine contro il *jitter* del sistema operativo *host*.
- **Segmento di terra (*Ground Station*):** un *personal computer* esegue uno *script* MATLAB dedicato (*AcquiringData.m*), che opera a una frequenza nominale di 50 [Hz], ossia con un passo di campionamento $\Delta t = 20$ [ms]. Lo *script* svolge un duplice compito: in trasmissione genera e invia i comandi di eccitazione sintetizzati dalla funzione *generateSignal*; in ricezione drena continuamente il *buffer* seriale, filtra i pacchetti validi e li scrive su un file di testo strutturato in formato *.csv*.

La scelta di mantenere il segmento di terra su MATLAB, anziché spostare l'intera logica di controllo a bordo, è coerente con l'obiettivo del progetto. Il drone è in questa fase una *piattaforma di acquisizione*, e MATLAB rappresenta l'ambiente naturale in cui i dati saranno successivamente analizzati nel Capitolo 5.

4.3 Parametrizzazione dello script di acquisizione

Per garantire la manutenibilità del software di terra e la massima sicurezza operativa durante i test di laboratorio, lo *script* di telemetria adotta una filosofia di sviluppo strettamente parametrizzata. Tutte le costanti fisiche, geometriche e di saturazione dei motori e dei servomotori sono isolate nel preambolo del codice. Questo approccio previene l'utilizzo di *magic numbers* all'interno dei *loop* operativi, riduce la probabilità di errori dovuti a comandi fuori scala e consente una rapida riconfigurazione del sistema in caso di modifiche ai limiti meccanici del prototipo o alla rigidità del vincolo elastico delle funi.

```

1 %% PARAMETRI PROGRAMMA
2 HOVER          = 1050;
3 MOTOR_MIN     = 700;
4 MOTOR_MAX     = 1500;
5 STEP_SMALL    = 50;
6 STEP_MEDIUM   = 100;
7 SIN_AMP       = 100;
8 CENTER_SERVO  = 1125;
9 DT_TARGET     = 0.02;      % 50 Hz
10 duration     = 30;
```

Listing 4.1: Parametri globali dello script di acquisizione

Il significato dei principali parametri è il seguente.

- `MOTOR_MIN = 700` e `MOTOR_MAX = 1500` definiscono i limiti inferiore e superiore dei segnali di comando in unità CCR (*Capture-Compare Register*) destinati agli ESC dei motori *Turnigy SK3*. Costituiscono la saturazione dell’attuatore principale: senza questi limiti, un errore di calcolo lato MATLAB potrebbe inviare un valore troppo basso (con conseguente spegnimento dei motori) oppure troppo alto (con potenziale danneggiamento degli ESC o uscita dalla zona di linearità della curva di spinta).
- `HOVER = 1050` rappresenta il valore CCR corrispondente alla spinta nominale di equilibrio statico nella configurazione vincolata. Tutti i segnali di eccitazione che agiscono sui motori sono espressi come variazioni rispetto a questo valore.
- `STEP_SMALL = 50`, `STEP_MEDIUM = 100` e `SIN_AMP = 100` sono le ampiezze nominali dei segnali di eccitazione (gradini e sinusoidi) generati dalla funzione `generateSignal`. Questi parametri determinano il rapporto segnale-rumore dell’identificazione: ampiezze troppo piccole non producono uno spostamento angolare distinguibile dal rumore del BNO-055, mentre ampiezze eccessive potrebbero forzare le funi di vincolo oltre il loro regime elastico o portare i servomotori in saturazione meccanica.
- `CENTER_SERVO = 1125` è il valore CCR corrispondente alla posizione neutra dei servomotori *Hitec HS-82MG*, ovvero all’inclinazione nulla dei *flap*.
- `DT_TARGET = 0.02` sancisce il passo di campionamento temporale dell’intero sistema. Coordinare la telemetria alla stessa frequenza a cui opera il ramo IMU/servo del firmware (50 [Hz], come descritto nella Sezione ?? del Capitolo 3) è una scelta metodologica essenziale per evitare effetti di *aliasing* e sfasamenti nei modelli identificati.

4.4 Protocollo di scambio dati

La comunicazione tra la *Ground Station* e il firmware avviene attraverso lo scambio di stringhe formattate testualmente su canale seriale. La scelta di un protocollo testuale CSV (*Comma-Separated Values*), in contrapposizione a un protocollo binario puro, è stata adottata per due ragioni: la facilità di *debug human-readable* durante le prime fasi di integrazione, e l’immediata compatibilità con le funzioni di importazione di MATLAB (`readtable`, `readmatrix`).

4.4.1 Flusso di *uplink*: comandi

Il computer di terra invia al drone i *setpoint* di controllo tramite una stringa strutturata secondo la sintassi:

```
CMD,<top_pwm>,<bottom_pwm>,<roll_pwm>,<pitch_pwm>\n
```

I quattro campi numerici rappresentano, nell'ordine, il PWM del motore superiore, il PWM del motore inferiore, il PWM del servomotore di *roll* e quello del servomotore di *pitch*, tutti espressi in unità CCR. La stringa viene inviata rigorosamente alla frequenza di 50 [Hz] e contiene gli ingressi di eccitazione sintetizzati dalla funzione `generateSignal` (descritta nella Sezione 4.7). L'insieme dei comandi di stato (`MOTOR_ON`, `MOTOR_OFF`, `KILL`), distinti dai comandi periodici di attuazione, è già stato descritto nel Capitolo 3.

4.4.2 Flusso di *downlink*: telemetria

A ogni iterazione, il firmware risponde inviando una stringa CSV contenente **15 campi separati da 14 virgole**, che rappresentano lo stato completo del velivolo:

```
time_ms, roll_cdeg, pitch_cdeg, yaw_cdeg, alt_mm, top_pwm, bottom_pwm, roll_pwm,
pitch_pwm, motors_enabled, uart_rx_errors, uart_rx_overflows, uart_bad_lines,
telemetry_overrun, stop_reason\n
```

I primi nove campi descrivono lo stato cinematico e di attuazione del velivolo, mentre gli ultimi sei campi sono di natura diagnostica:

- `time_ms` — riferimento temporale in millisecondi dall'avvio del firmware, utilizzato lato MATLAB per costruire il vettore tempo dell'identificazione;
- `roll_cdeg`, `pitch_cdeg`, `yaw_cdeg` — angoli di Eulero espressi in **centesimi di grado**. Questa conversione da *float* a *int* evita il costo computazionale e i ritardi di formattazione delle virgole mobili all'interno dello *stack stdio* del microcontrollore, e mantiene una precisione angolare di 0.01°;
- `alt_mm` — quota stimata dal VL53L1X dopo la compensazione del *tilt*, in millimetri;
- `top_pwm`, `bottom_pwm`, `roll_pwm`, `pitch_pwm` — valori CCR effettivamente scritti negli attuatori dopo la saturazione hardware. Vengono trasmessi (e non semplicemente ripetuti rispetto al comando inviato) per consentire al PC di osservare gli effetti delle saturazioni;
- `motors_enabled` — *flag* binaria che indica se il firmware è nello stato *armato* o *disarmato*;
- `uart_rx_errors`, `uart_rx_overflows`, `uart_bad_lines` — contatori cumulativi degli errori UART in ricezione lato firmware. Permettono di monitorare in tempo reale la qualità del collegamento;
- `telemetry_overrun` — *flag* che indica se il firmware ha rilevato un ritardo nella propria stessa trasmissione, fenomeno che si verificherebbe se il *buffer* di uscita non riuscisse a smaltire i pacchetti alla frequenza imposta;
- `stop_reason` — codice numerico che identifica il motivo dell'ultimo arresto controllato, utile per analizzare *a posteriori* l'esito delle prove.

4.5 Watchdog firmware e keep-alive lato MATLAB

Operare un velivolo instabile tramite computer richiede un meccanismo che prevenga il *flyaway* o l'impatto distruttivo in caso di *crash* del PC o di disconnessione del cavo USB. Per questo motivo il firmware è equipaggiato con un *watchdog* di comunicazione, descritto nel Capitolo 3: se la scheda non riceve un nuovo comando **CMD** entro **200 [ms]**, forza autonomamente i motori al PWM minimo e torna allo stato *disarmato*. La scelta di 200 [ms] come finestra di *timeout* costituisce un compromesso tra reattività in caso di guasto della linea di comunicazione (un valore troppo grande lascerebbe il drone in funzione per un tempo eccessivo dopo la perdita del collegamento) e robustezza rispetto al *jitter* accumulato lato MATLAB (un valore troppo piccolo provocherebbe *trigger* spuri durante normali rallentamenti del sistema operativo).

Esiste tuttavia una fase critica in cui questo meccanismo, pensato per la sicurezza, rischia di intervenire in modo non desiderato: la *start sequence*. Durante l'attesa di un *acknowledgement* (in particolare l'**ACK_ON** dopo l'invio di **MOTOR_ON**), MATLAB non sta ancora inviando comandi **CMD**, e il *watchdog* potrebbe scattare prima che l'operatore abbia il controllo effettivo del sistema.

Per risolvere questa situazione, lo *script* MATLAB implementa un meccanismo di *keep-alive*: durante l'attesa degli **ACK** critici, vengono inviati periodicamente comandi neutri (motori al minimo, servomotori centrati) che mantengono attivo il *watchdog* senza alterare lo stato fisico del drone. L'implementazione è inserita nella funzione `sendCommandWaitAck`:

```
1 % Keep-alive durante l'attesa ACK.
2 % Utile soprattutto per ACK_ON, perché dopo MOTOR_ON
3 % il watchdog firmware richiede CMD entro 200 ms.
4 if strlength(keepAliveCmd) > 0 && toc(tKeepAlive) >= keepAlivePeriod
5     writeline(serialObj, keepAliveCmd);
6     tKeepAlive = tic;
7 end
8
```

Listing 4.2: Meccanismo keep-alive durante l'attesa degli **ACK** critici

Il *keep-alive* viene inviato ogni 40 [ms], periodo significativamente più breve della finestra di 200 [ms] del *watchdog*, ma sufficientemente rilassato da non saturare la seriale durante le attese. La stringa inviata è la stessa che il sistema utilizzerà come comando iniziale di prova (motori al PWM minimo, servomotori centrati), garantendo continuità tra la fase di *startup* e l'inizio effettivo dell'eccitazione.

4.6 Loop di acquisizione MATLAB

Il ciclo `while toc(t0) < duration` costituisce il motore dell'acquisizione: alterna l'invio dei comandi di eccitazione e la lettura della telemetria entrante per l'intera durata della prova. La sua corretta progettazione richiede attenzione a quattro aspetti: la temporizzazione anti-burst, il parsing robusto dei dati ricevuti, la gestione asincrona della terminazione d'emergenza e la chiusura controllata della porta seriale.

4.6.1 Scheduling anti-burst

Un problema comune nei sistemi operativi non *real-time* (come *Windows* o *macOS*) è il *jitter*: il sistema può sospendere temporaneamente l'esecuzione di MATLAB per dare priorità ad altri processi. Se MATLAB accumula ritardo, l'istinto del software sarebbe quello di recuperare inviando in rapida successione tutti i comandi arretrati (effetto *burst*). Questo intaserebbe il *buffer* della scheda e invaliderebbe la corrispondenza temporale tra ingressi e uscite, compromettendo l'identificazione. Per evitare questo comportamento, lo *script* adotta uno scheduling di riallineamento:

```
1 % Scheduling anti-burst: se MATLAB resta indietro, non recupera
2 % mandando tanti CMD consecutivi.
3 nextCmdTime = nextCmdTime + DT_TARGET;
4
5 if (t - nextCmdTime) > DT_TARGET
6     nextCmdTime = t + DT_TARGET;
7 end
8
```

Listing 4.3: Scheduling anti-burst

Se il tempo corrente t supera di oltre un periodo `nextCmdTime`, segno di un blocco temporaneo del sistema operativo, lo *script* non cerca di compensare il ritardo accumulando comandi arretrati. Riallinea invece la variabile `nextCmdTime` sul tempo corrente, abbandonando deliberatamente la cadenza nominale precedente in favore di un nuovo riferimento locale. La conseguenza pratica è la perdita di qualche campione in corrispondenza del *glitch* e una piccola discontinuità di fase nei comandi successivi, ma la cadenza media di 50 [Hz] e la coerenza temporale del resto della prova vengono preservate, condizione necessaria per la successiva identificazione.

4.6.2 Parsing CSV e validazione delle righe

La seriale non trasmette esclusivamente i pacchetti di telemetria: veicola anche messaggi di stato, risposte ACK e occasionali messaggi di *debug*. Lo *script* deve dunque classificare ogni riga ricevuta prima di salvarla. La scrematura avviene all'interno della funzione `readSerialAvailable`.

Per distinguere una riga di telemetria valida da un messaggio di testo generico, si utilizza un criterio basato sulla struttura geometrica della stringa: il conteggio dei separatori. Una telemetria CSV ben formata contiene esattamente 15 colonne, ossia **14 virgole**; qualsiasi linea con un conteggio diverso è considerata corrotta o non pertinente.

```
1 % Telemetria CSV valida: 15 colonne = 14 virgole.
2 if count(line, ",") == 14
3     try
4         fprintf(fid, "%s\n", line);
5         stats.telemetryRows = stats.telemetryRows + 1;
6     catch
7         stats.badLines = stats.badLines + 1;
8     end
9     continue;
10 end
11
```

Listing 4.4: Validazione delle righe di telemetria tramite conteggio dei separatori

Questo controllo è un filtro di primo livello, di natura puramente strutturale: garantisce che la riga abbia il numero corretto di campi, ma non verifica che ciascun campo contenga un valore numerico nel range atteso. Una corruzione che lasciasse intatto il numero di virgole ma alterasse il contenuto di un singolo campo (ad esempio un valore di angolo di Eulero fuori dall'intervallo $[-180, +180]^\circ$) non verrebbe intercettata in questa fase. Un controllo più robusto richiederebbe l'inserimento di un *checksum* (ad esempio CRC-8) come ultimo campo del pacchetto, soluzione comune nei protocolli telemetrici embedded ma non implementata in questa iterazione del progetto per non appesantire eccessivamente il *payload* a livello applicativo. Nella pratica, il controllo geometrico si è dimostrato sufficiente per le campagne sperimentali svolte: se un disturbo elettromagnetico sui cavi del laboratorio corrompe un pacchetto UART troncandolo a metà, il conteggio delle virgole fallisce, la riga viene classificata come `badLines` e il file `.csv` resta libero da valori anomali (NaN) che farebbero fallire le successive procedure di identificazione in `tfest`. Eventuali corruzioni più sottili emergerebbero comunque in fase di analisi attraverso il controllo dei limiti fisici dei segnali.

4.6.3 Terminazione asincrona di emergenza

Data la massa del prototipo (≈ 2.2 [kg]), l'energia cinetica delle eliche e il vincolo elastico delle funi, esiste la possibilità che oscillazioni divergenti si sviluppino più velocemente di quanto un operatore possa reagire premendo un pulsante tradizionale. Per questo motivo la telemetria include una gestione prioritaria degli arresti di emergenza, distinta a due livelli:

1. **Arresto controllato (*Safe Stop*)**: innescato al naturale scadere della durata della prova (ad esempio 30 [s]). Lo *script* invia il comando `MOTOR_OFF`; il firmware porta gradualmente i motori al PWM minimo e i servomotori in posizione neutra, garantendo un'uscita controllata dallo stato *armato*.
2. **Terminazione d'emergenza (*Kill*)**: innescata manualmente dall'operatore tramite la pressione della barra spaziatrice o dell'apposito pulsante sulla GUI di MATLAB.

La pressione del tasto non viene valutata all'interno del ciclo principale, ma è gestita tramite il meccanismo dei *callback* di MATLAB, che esegue la funzione associata in modo asincrono rispetto al *loop* a 50 [Hz]:

```

1 function keyKill(~, event)
2     global STOP_FLAG STOP_REASON serialObj_global ...
3         EMERGENCY_KILL_SENT STOP_IS_KILL
4     if strcmp(event.Key, 'space')
5         STOP_FLAG = true;
6         STOP_IS_KILL = true;
7         if ~EMERGENCY_KILL_SENT
8             EMERGENCY_KILL_SENT = true;
9             try
10                 writeline(serialObj_global, "KILL");
11             catch
12             end
13         end
14     end
15 end
16

```

Listing 4.5: Callback asincrono per la terminazione d'emergenza

Nell'istante esatto in cui viene premuta la barra spaziatrice, il *callback* scavalca il ciclo principale, invia immediatamente la stringa KILL sulla seriale e alza le *flag* globali **STOP_FLAG** e **STOP_IS_KILL**. Il firmware, ricevendo questa parola chiave, bypassa qualsiasi logica di volo e forza istantaneamente i motori al PWM minimo, attivando il *latch* di sicurezza descritto nel Capitolo 3. Il tempo che intercorre tra la pressione del tasto e l'arresto effettivo dei motori è dell'ordine di qualche millisecondo, dominato dal tempo di trasmissione seriale della stringa.



Figura 4.1: *Interfaccia grafica del Kill Switch*

4.6.4 Chiusura controllata con onCleanup

Ogni prova si conclude con l'esecuzione dell'oggetto `cleanupObj`, costruito all'inizio dello *script* mediante la primitiva `onCleanup` di MATLAB. Anche nel caso in cui lo *script* terminasse a causa di un'eccezione non gestita, questo meccanismo garantisce l'esecuzione di una funzione di chiusura che invia almeno una sequenza di comandi al minimo e un **MOTOR_OFF**, chiude correttamente il *file handle* del CSV e rilascia la porta seriale.

Alla fine di ogni prova, MATLAB stampa a video un *report* riassuntivo basato sulla *struct stats*, contenente il numero di righe di telemetria salvate, gli ACK ricevuti, gli **ACK_ERR** e le righe scartate. Per una prova di 30 [s] a 50 [Hz] ci si aspetta il salvataggio di circa $30 \cdot 50 = 1500$ righe; uno scostamento significativo da questo valore, o un numero non trascurabile di **badLines**, costituisce un indicatore immediato di anomalie nel canale — tipicamente un *baudrate* insufficiente o una schermatura inadeguata rispetto ai disturbi elettromagnetici dei motori.

4.7 Profili di eccitazione

Per ottenere un modello matematico accurato di un sistema fisico come il *DPDF*, il sistema deve essere opportunamente eccitato. Un drone vincolato presenta dinamiche complesse: inerzie distribuite, attriti sui perni dei *flap*, rigidità delle funi, accoppiamenti aerodinamici tra gli assi e non linearità varie. Non esiste un singolo segnale di test in grado di catturare tutte queste sfumature contemporaneamente.

Per questo motivo lo *script* di acquisizione è stato progettato con un'architettura *multi-modale* che permette all'operatore di selezionare, all'avvio della prova tramite l'interfaccia `listdlg`, fra **23 tipologie** di sollecitazione in anello aperto. Le 23 modalità sono organizzate in cinque famiglie:

- **prove 1.x** — gradini e sinusoidi sui motori, per stimolare la dinamica di spinta verticale;
- **prove 2.x** — eccitazione differenziale dei motori, per stimolare la dinamica di *yaw*;
- **prove 3.x** — gradini, sinusoidi e *chirp* sul *flap* di *roll*;
- **prove 4.x** — gradini, sinusoidi e *chirp* sul *flap* di *pitch*;
- **prove 5.x** — eccitazioni combinate e sequenze pseudo-casuali multilivello su uno o entrambi gli assi.

La selezione della prova imposta lo stato della funzione `generateSignal`, che a ogni iterazione del *loop* principale produce la quadrupla (`top`, `bottom`, `roll`, `pitch`) da inviare al firmware. Nelle sottosezioni seguenti vengono descritte le caratteristiche principali di ciascuna famiglia di segnali.

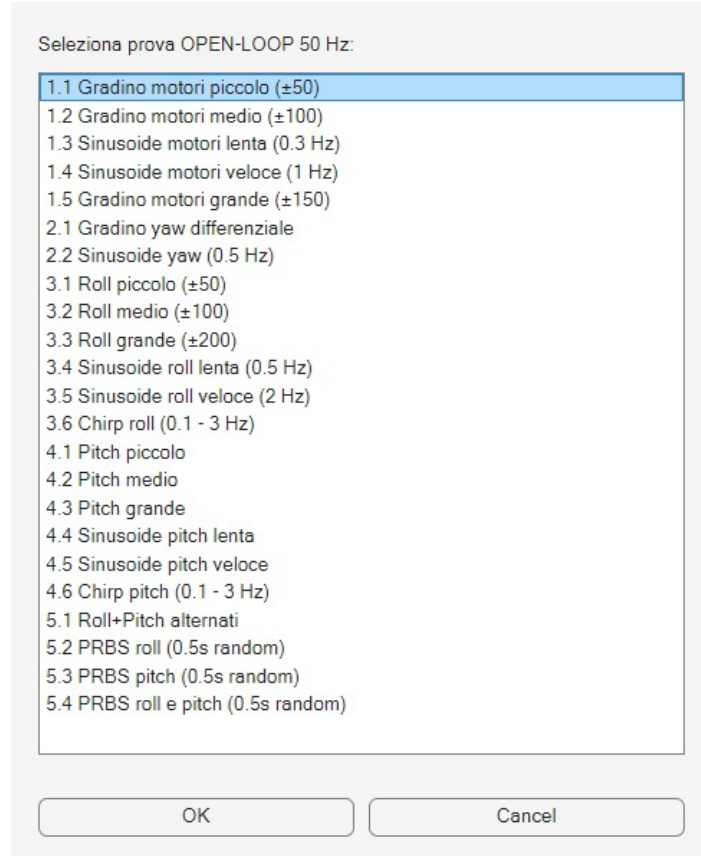


Figura 4.2: Menù di selezione del profilo di eccitazione

4.7.1 Segnali a gradino

Le prove a gradino (ad esempio 1.1, 3.1, 4.1) consistono nell'applicare una variazione istantanea e costante al segnale di comando per un intervallo di tempo prestabilito (tipicamente 5 [s]), prima di riportarlo al valore neutro. Questi test sono utili per una prima ispezione qualitativa della dinamica: permettono di stimare il guadagno statico del sistema misurando il rapporto tra l'ampiezza dell'ingresso e l'angolo di regime raggiunto, ed evidenziano parametri quali il ritardo puro, il tempo di salita e l'eventuale presenza di sovraelongazioni dovute all'elasticità delle funi.

La disponibilità di gradini di ampiezza diversa (*Piccolo, Medio, Grande*) è motivata dalla natura non lineare del sistema. Un gradino piccolo potrebbe non vincere l'attrito statico dei perni dei *flap*, restituendo una risposta dominata dal rumore del BNO-055; un gradino grande potrebbe invece mandare in saturazione i servomotori o caricare in modo asimmetrico le funi. Le diverse ampiezze permettono di individuare la regione di linearità ottimale per la successiva identificazione.

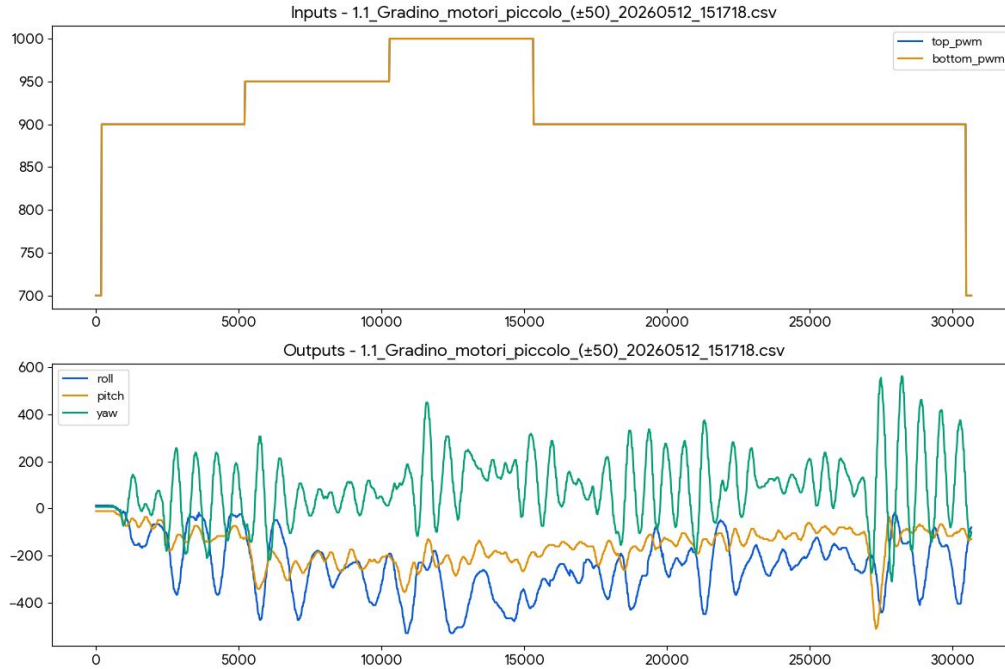


Figura 4.3: Esempio di test a gradino in anello aperto (prova 1.1, gradino motori piccolo ± 50): comandi PWM ai motori e ai servo, e risposta angolare del drone

4.7.2 Segnali armonici e chirp

Le prove armoniche (ad esempio 3.4, 3.5, 4.4, 4.5) comandano i *flap* affinché oscillino con una frequenza specifica (tipicamente 0.5 [Hz] o 2.0 [Hz]). Le prove *chirp* (3.6, 4.6) impongono invece una frequenza spazzolata linearmente tra 0.1 e 3 [Hz] nell'arco della durata della prova.

Questi segnali permettono di tracciare empiricamente il diagramma di Bode del sistema: all'aumentare della frequenza di oscillazione del comando, l'inerzia del velivolo ne attenua progressivamente l'ampiezza di risposta e ne introduce uno sfasamento. Il *chirp* è partico-

larmente prezioso perché, in un'unica prova, copre un'intera decade di frequenze e consente di individuare eventuali frequenze di risonanza del telaio in PLA o del sistema elastico delle funi, informazione utile in fase successiva qualora si rendesse necessario inserire filtri *notch* o passa-basso nei rami di retroazione.

4.7.3 Sequenze pseudo-casuali multilivello

Le prove contrassegnate dai codici 5.2, 5.3 e 5.4 implementano sequenze pseudo-casuali multilivello, riconducibili alla famiglia delle *Amplitude-modulated Pseudo-Random Binary Sequences* (APRBS). A differenza della PRBS classica — generata da un *Linear Feedback Shift Register* e limitata a due livelli di ampiezza — l'APRBS impiegata in questo lavoro adotta cinque livelli discreti di ampiezza, mantenendo invariata la natura pseudo-casuale dei salti. Per coerenza con la nomenclatura adottata nello *script* di acquisizione e nei nomi dei *dataset* prodotti, nel seguito si continuerà a indicare queste sequenze con la sigla *PRBS*, fermo restando che la definizione tecnicamente più corretta è quella di sequenza multilivello.

In queste modalità il comando inviato ai *flap* non segue una legge matematica continua, ma salta in modo istantaneo tra livelli discreti definiti a partire dalle costanti `STEP_SMALL` e `STEP_MEDIUM`, a intervalli temporali regolari (ogni 0.5 [s]).

Le sequenze pseudo-casuali rappresentano uno standard nell'identificazione *black-box* dei sistemi dinamici. Da un punto di vista spettrale, un salto pseudo-casuale tra livelli discreti approssima il comportamento di un rumore bianco entro la banda di interesse: il segnale contiene energia su un'ampia gamma di frequenze, permettendo di stimolare contemporaneamente tutti i modi della dinamica e di alimentare in modo efficiente gli algoritmi di identificazione parametrica come *tfest* (descritto nel Capitolo 5). Va tuttavia osservato che la scelta di cinque livelli (incluso lo zero) anziché due livelli puri — come prescriverebbe la PRBS classica — comporta una lieve riduzione dell'energia spettrale totale a parità di ampiezza massima, in quanto durante i blocchi in cui il livello selezionato è zero il sistema non viene effettivamente eccitato. Questa scelta è stata ritenuta accettabile in quanto rende il segnale meccanicamente meno aggressivo sui servomotori e sulle funi, prolungandone la vita utile durante le ripetute campagne di acquisizione.

Le prove 5.2 (PRBS *roll*) e 5.3 (PRBS *pitch*) hanno svolto il ruolo principale nella stima delle funzioni di trasferimento $G_{\text{roll}}(z)$ e $G_{\text{pitch}}(z)$ presentate nel prossimo capitolo. La scelta è motivata da due proprietà del segnale.

- **Copertura spettrale ampia:** poiché la sequenza di livelli e la direzione dei salti variano in modo stocastico, pur rimanendo ancorate al passo di 0.5 [s], il segnale presenta energia significativa sia alle basse frequenze (quando il livello rimane costante per più blocchi consecutivi) sia alle alte frequenze (quando il livello cambia ad ogni blocco).
- **Stimolazione di transitori e regime:** il sistema viene costretto a eseguire continui transitori intervallati da brevi finestre di assestamento, permettendo all'algoritmo di identificazione di catturare sia gli effetti inerziali sia il comportamento di regime nelle vicinanze dell'equilibrio.

L'implementazione della modalità 5.2 (PRBS *roll*) è riportata di seguito:

```
1 case 21
2 % 5.2 PRBS roll ogni 0.5 s
```

```

3  block = floor(time / 0.5);
4  rng(block);
5  levels = [-MEDIUM, -SMALL, 0, SMALL, MEDIUM];
6  idx = randi(numel(levels));
7  roll = CENTER + levels(idx);
8

```

Listing 4.6: Generazione della sequenza pseudo-casuale multilivello sul flap di roll (case 21)

Il funzionamento si articola in tre passi.

1. L'istruzione `block = floor(time / 0.5)` divide la durata della prova in finestre temporali discrete di mezzo secondo, e associa a ciascuna finestra un indice intero.
2. L'istruzione `rng(block)` reimposta il *seed* del generatore casuale di MATLAB all'inizio di ogni blocco. Questo dettaglio è cruciale: garantisce che, anche ripetendo lo stesso test in giorni diversi, la sequenza di salti casuali estratta sia esattamente la stessa. La ripetibilità della sollecitazione è una condizione necessaria per confrontare i dati di telemetria reali con le uscite del simulatore a parità di ingresso.
3. Infine, la funzione `randi` seleziona in modo equiprobabile uno dei cinque livelli discreti $\{-100, -50, 0, +50, +100\}$ in unità CCR, che viene sommato al valore neutro `CENTER.SERVO` per ottenere il comando finale al servomotore di *roll*.

La prova 5.3 (PRBS *pitch*) ha struttura identica, applicata al servomotore di *pitch*. La prova 5.4 combina le due sequenze sui due assi simultaneamente, con *seed* indipendenti per garantire la decorrelazione fra i due ingressi.

4.7.4 Eccitazioni multi-asse

Le modalità finali dello *script* (prova 5.1 *roll+pitch* alternati, e prova 5.4 PRBS combinata) stimolano contemporaneamente o in modo alternato entrambi gli assi geometrici del velivolo. Questi test sono particolarmente utili per evidenziare gli effetti di accoppiamento dinamico (*cross-coupling*) tra *roll* e *pitch*, dovuti alla geometria interna del condotto, all'effetto giroscopico delle eliche e all'eventuale deviazione asimmetrica del flusso d'aria. Mentre le prove monoasse permettono di identificare modelli SISO per ciascun ramo, le prove multi-asse forniscono il dato necessario per costruire eventuali modelli MIMO più completi nelle estensioni future del progetto.

4.8 Validazione del link telematico

Al termine di ogni prova, lo *script* produce un *report* riassuntivo che costituisce la metrica con cui validare la qualità della singola acquisizione:

```

1  fprintf("\n===== REPORT FINALE =====\n");
2  fprintf("File CSV: %s\n", filename);
3  fprintf("Motivo stop: %s\n", STOP_REASON);
4  fprintf("Righe telemetria salvate: %d\n", stats.telemetryRows);
5  fprintf("ACK_ERR ricevuti: %d\n", stats.ackErr);
6  fprintf("Righe corrette/vuote ignorate: %d\n", stats.badLines);
7  fprintf("===== \n");

```

Listing 4.7: Esempio di report di fine prova

I tre indicatori principali del *report* sono:

- **telemetryRows** — numero totale di righe CSV salvate sul file. In una prova di 30 [s] a 50 [Hz] il valore atteso è prossimo a 1500;
- **badLines** — numero di righe ricevute ma scartate dal parsing per non aver superato il controllo delle 14 virgole. Un valore non nullo segnala disturbi sul canale o un *baudrate* prossimo al limite di saturazione;
- **ackErr** — numero di risposte **ACK_ERR** ricevute dal firmware, indicative di comandi malformati lato MATLAB.

Nelle campagne sperimentali svolte, i *dataset* effettivamente utilizzati per la successiva fase di identificazione sono stati prevalentemente quelli generati dalle prove PRBS sui due assi di assetto. In particolare, il *dataset 5.3_PRBS_pitch_*.csv* contiene oltre 1500 campioni utili con motori accesi, mentre il *dataset 5.2_PRBS_roll_*.csv* contiene circa 840 campioni utili — in entrambi i casi sufficienti, per durata e ricchezza spettrale, ad alimentare in modo efficace gli algoritmi di **tfest** descritti nel Capitolo 5.

Con la produzione di questi file *.csv* il sistema è pronto per la fase successiva, in cui i dati registrati vengono elaborati in MATLAB per l'identificazione delle funzioni di trasferimento del velivolo e per la sintesi dei regolatori PID.

5 Identificazione del modello

Il presente capitolo descrive la fase di costruzione del modello matematico del *Double Propeller Ducted-Fan (DPDF)* e la successiva sintesi dei regolatori PID. La procedura adottata è di tipo *data-driven*: il modello dinamico del velivolo non viene derivato analiticamente dalle equazioni della meccanica, bensì *stimato* a partire dai *dataset* di telemetria acquisiti nel Capitolo 4. I modelli identificati vengono poi impiegati per la sintesi automatica dei controllori tramite `pidtune`, con verifica della risposta in anello chiuso in simulazione prima della validazione sul prototipo fisico.

5.1 Approccio data-driven

La modellistica analitica classica, basata sulle equazioni cardinali della dinamica di Newton-Eulero, descrive accuratamente il comportamento cinematico di un corpo rigido nello spazio. Nel caso di un velivolo con architettura *DPDF*, tuttavia, tale approccio risulta parziale. Le complesse interazioni fluidodinamiche che si generano all'interno del condotto — lo strato limite lungo le pareti interne, l'effetto di scia indotto dalle due eliche controrotanti, le mutue interferenze aerodinamiche tra i flussi e i *flap* di controllo — introducono non-linearità e incertezze parametriche difficilmente quantificabili *a priori*.

Per superare tali limitazioni si è scelto un approccio orientato ai dati, implementando una procedura di identificazione del sistema a **scatola nera** (*Black-Box Identification*). Questa metodologia permette di ricavare un modello matematico lineare e tempo-invariante (LTI) nell'intorno del punto di lavoro statico, partendo esclusivamente dall'analisi delle risposte temporali del prototipo stimolato in anello aperto.

Vale la pena anticipare che il modello LTI così ottenuto è valido localmente, in un intorno limitato della condizione di equilibrio. Le dinamiche globali del velivolo — inclusi gli accoppiamenti tra gli assi, gli effetti delle saturazioni meccaniche e l'interazione con il vincolo elastico delle funi — saranno invece esplorate tramite il simulatore fisico tridimensionale descritto nel Capitolo 6, che adotta un modello a parametri concentrati di tipo *grey-box* e complementa le funzioni di trasferimento qui identificate.

5.2 Pre-elaborazione dei dati

I dati acquisiti dal firmware durante le prove vengono salvati nei file `.csv` descritti nel Capitolo 4. In questi file i comandi impartiti ai servomotori sono espressi in unità CCR (*Capture-Compare Register*), mentre le letture di assetto dell'IMU sono registrate in centesimi di grado.

Prima di procedere alla stima delle funzioni di trasferimento è necessario un processo di conversione e condizionamento dei segnali. L'obiettivo è duplice: convertire le variabili in unità ingegneristiche coerenti (gradi fisici sia per l'ingresso che per l'uscita) e traslare l'origine dei segnali, in modo che rappresentino le pure variazioni dinamiche Δ rispetto alla condizione di equilibrio statico.

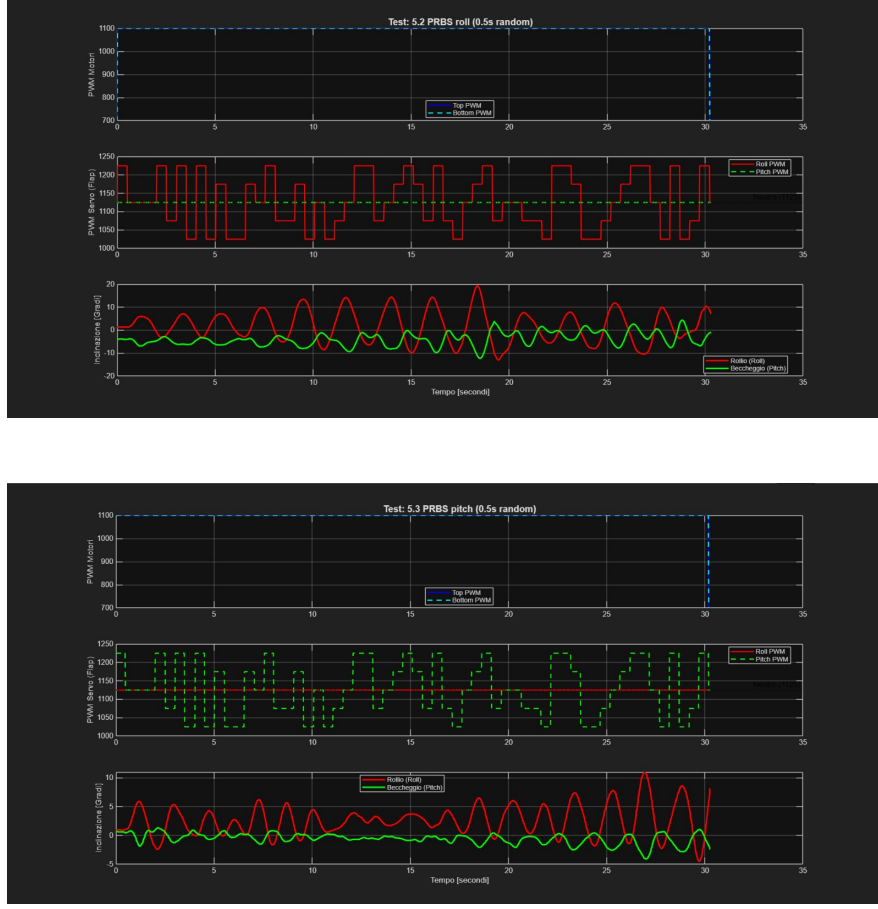


Figura 5.1: *Dataset sperimentale ad anello aperto ottenuto tramite stimolazione PRBS. In alto: comandi PWM ai motori; al centro: comandi PWM ai flap; in basso: risposta angolare del velivolo misurata dall'IMU. A sopra l'asse di roll, a sotto l'asse di pitch*

La centratura del segnale d'ingresso rispetto alla posizione neutra del *flap* ($CCR_{\text{neutro}} = 1125$) è governata dalla relazione:

$$\Delta u(t) = \frac{CCR_{\text{flap}}(t) - CCR_{\text{neutro}}}{K_{\text{CCR}}} \quad (5.1)$$

dove $K_{\text{CCR}} = 10.0 \text{ [CCR/deg]}$ rappresenta la costante di conversione statica del servomotore *Hitec HS-82MG* adottato.

Analogamente, la misura dell'angolo di assetto $\theta(t)$ (*roll* o *pitch*) viene convertita in gradi e depurata dal valore medio di *offset* $\bar{\theta}$ registrato durante la fase di equilibrio:

$$\Delta y(t) = \frac{\theta_{\text{cdeg}}(t)}{100} - \bar{\theta} \quad (5.2)$$

L'implementazione di questa fase di pre-elaborazione è riportata nel codice MATLAB seguente:

```

1 CCR_PER_DEGREE = 10.0;    % Costante di conversione servo
2 CENTER_SERVO   = 1125;    % Zero meccanico flap
3
4 % Condizionamento dell'ingresso (comando flap in gradi)
5 u_raw          = double(T_asse.roll_pwm);    % o pitch_pwm secondo l'asse
6 u_centered     = (u_raw - CENTER_SERVO) / CCR_PER_DEGREE;
7
8 % Condizionamento dell'uscita (angolo IMU in gradi)
9 y_raw          = double(T_asse.roll_cdeg) / 100.0;
10 y_mean         = mean(y_raw);
11 y_centered     = y_raw - y_mean;
12
13 % Creazione dell'oggetto iddata per il System Identification Toolbox
14 Ts             = 0.02;    % Periodo di campionamento (50 Hz)
15 data_idnt      = iddata(y_centered, u_centered, Ts);
16

```

Listing 5.1: Pre-elaborazione dei segnali di ingresso e uscita

Il periodo di campionamento $T_s = 0.02$ [s] coincide con la frequenza di trasmissione della telemetria (50 [Hz]) ed è coerente con la frequenza alla quale il PID, una volta validato, opererà nel firmware. Questa scelta evita disallineamenti tra il modello identificato e l'ambiente in cui i regolatori saranno effettivamente eseguiti.

5.3 Selezione dell'architettura del modello

Per identificare la dinamica del sistema SISO (*Single-Input Single-Output*) che lega lo spostamento angolare dei *flap* alla rotazione del velivolo attorno ai rispettivi assi principali, si è utilizzato il *System Identification Toolbox* di MATLAB, in particolare la funzione `tfest`. La stima è stata condotta nel dominio del tempo continuo, configurando l'algoritmo iterativo di ottimizzazione su una struttura a tempo continuo che viene poi convertita a tempo discreto per il confronto con i dati.

Per evitare fenomeni di sovra-parametrizzazione del rumore di misura ad alta frequenza (causato principalmente dalle vibrazioni dei motori *brushless*), la complessità strutturale della funzione di trasferimento è stata limitata sulla base di considerazioni fisiche. La dinamica complessiva del drone può essere approssimata, in prima istanza, a un sistema del secondo ordine — caratterizzato da un comportamento inerziale e da uno smorzamento aerodinamico — a cui si aggiungono il ritardo di attuazione dei servomotori e l'effetto di un eventuale zero al numeratore.

La struttura generale del modello LTI a tempo continuo che si cerca di identificare è:

$$G(s) = \frac{\Delta Y(s)}{\Delta U(s)} = \frac{b_m s^m + \dots + b_1 s + b_0}{s^n + a_{n-1} s^{n-1} + \dots + a_1 s + a_0} \quad (5.3)$$

Sono state confrontate tre strutture candidate per ciascun asse: **2 poli e 1 zero**, **2 poli e 0 zeri**, **3 poli e 1 zero**. L'analisi iterativa condotta sui *dataset* PRBS ha evidenziato due architetture ottimali per i rispettivi assi:

- **Asse di *roll***: modello a 2 poli e 1 zero ($n = 2$, $m = 1$), che cattura adeguatamente la dinamica inerziale del velivolo nella sezione trasversale.

- **Asse di *pitch*:** modello a 3 poli e 1 zero ($n = 3, m = 1$). Il polo aggiuntivo rispetto all'asse di *roll* è imputabile alla maggiore complessità del flusso aerodinamico in direzione longitudinale, dovuta alla configurazione asimmetrica dei due rotori in cascata.

Le rispettive funzioni di trasferimento hanno dunque forma:

$$G_{\text{roll}}(s) = \frac{b_1 s + b_0}{s^2 + a_1 s + a_0}, \quad G_{\text{pitch}}(s) = \frac{b_1 s + b_0}{s^3 + a_2 s^2 + a_1 s + a_0} \quad (5.4)$$

I valori numerici dei coefficienti a_i e b_j ottenuti dall'identificazione su *dataset* PRBS sono riportati nella Tabella 5.1.

Coefficiente	$G_{\text{roll}}(s)$ (2p, 1z)	$G_{\text{pitch}}(s)$ (3p, 1z)
b_1	(-1.832)	(-2.033)
b_0	(-9.160)	(-8.130)
a_2	(n/a)	(12.00)
a_1	(20.00)	(70.00)
a_0	(100.00)	(100.00)

Tabella 5.1: Coefficienti numerici delle funzioni di trasferimento identificate.

5.4 Stima del modello e validazione

La stima dei coefficienti è stata effettuata tramite la funzione `tfest` del *System Identification Toolbox*, configurata per imporre la stabilità del modello stimato (opzione `EnforceStability`). Lo *script* di identificazione `Identify_and_tune.m` confronta automaticamente le tre strutture candidate e seleziona quella con il *fit* più elevato:

```

1 opt_tf = tfestOptions('EnforceStability', true, 'Display', 'off');
2
3 % Modello A: 2 poli, 1 zero
4 sysA = tfest(data_ident, 2, 1, opt_tf);
5 fitA = goodnessOfFit(sim(sysA, data_ident).y, y_centered, 'NRMSE') * 100;
6
7 % Modello B: 2 poli, 0 zeri
8 sysB = tfest(data_ident, 2, 0, opt_tf);
9 fitB = goodnessOfFit(sim(sysB, data_ident).y, y_centered, 'NRMSE') * 100;
10
11 % Modello C: 3 poli, 1 zero
12 sysC = tfest(data_ident, 3, 1, opt_tf);
13 fitC = goodnessOfFit(sim(sysC, data_ident).y, y_centered, 'NRMSE') * 100;
14
15 [best_fit, best_idx] = max([fitA, fitB, fitC]);
16
```

Listing 5.2: Confronto automatico delle tre strutture candidate (estratto da `Identify_and_tune.m`)

La bontà del modello identificato è valutata mediante l'indice di *fit* percentuale basato sulla NRMSE (*Normalized Root Mean Square Error*):

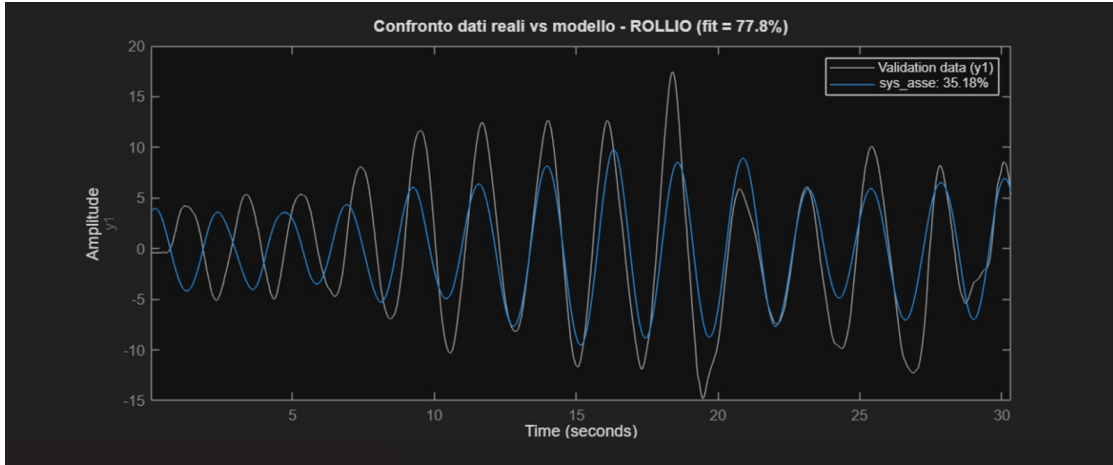
$$\text{Fit} = 100 \cdot \left(1 - \frac{\sqrt{\sum_{k=1}^N |y(k) - \hat{y}(k)|^2}}{\sqrt{\sum_{k=1}^N |y(k) - \bar{y}|^2}} \right) \quad [\%] \quad (5.5)$$

dove $y(k)$ è la misura sperimentale dell'IMU, $\hat{y}(k)$ è la risposta temporale simulata dal modello identificato e $\bar{y}$ è il valore medio del segnale reale. Un valore del 100% indica una corrispondenza perfetta; un valore prossimo a 0% indica che il modello non spiega la varianza del dato meglio di una semplice media costante.

I valori di *fit* ottenuti sui *dataset* PRBS sono:

- Asse di *roll*: Fit = 77.85%
- Asse di *pitch*: Fit = 77.67%

Si tratta di valori soddisfacenti per un'identificazione black-box su un sistema dotato di non linearità importanti e testato in condizioni vincolate. La quota residua di varianza non spiegata dal modello è imputabile a effetti che un modello lineare SISO non può catturare integralmente: in particolare l'attrito non lineare sui perni dei flap, l'azione non perfettamente simmetrica delle funi di vincolo e i piccoli accoppiamenti tra gli assi. I modelli identificati devono quindi essere interpretati come approssimazioni locali della dinamica dominante del DPDF nella configurazione sperimentale adottata, sufficienti per la sintesi preliminare dei regolatori PIDF e per la successiva verifica in simulazione.



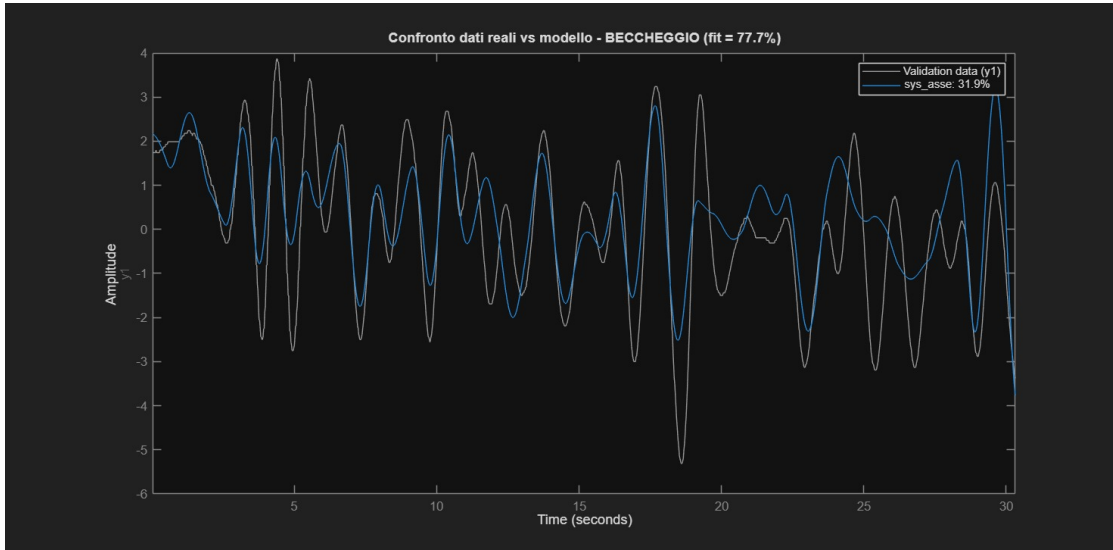


Figura 5.2: *Analisi di cross-validazione nel dominio del tempo per i due assi. Confronto tra risposta angolare reale del prototipo (linea nera) e uscita stimata dal modello LTI (linea colorata). Sopra l'asse di roll (Fit 77.85%), sotto l'asse di pitch (Fit 77.67%)*

5.4.1 Analisi del guadagno statico

Un'analisi che permette di validare fisicamente il modello identificato riguarda l'estrazione del guadagno statico del sistema, calcolato applicando il teorema del valore finale per $s \rightarrow 0$:

$$\mu = \lim_{s \rightarrow 0} G(s) = \frac{b_0}{a_0} \quad (5.6)$$

I test condotti hanno restituito, per entrambi gli assi, un guadagno statico di **segno negativo** ($\mu < 0$). Questo risultato fornisce una conferma fisica del modello stimato: un incremento del PWM del servomotore — corrispondente a un'inclinazione positiva del *flap* secondo la convenzione del firmware — genera una coppia aerodinamica che ruota il drone nel verso opposto rispetto al sistema di riferimento destrorso solidale all'IMU. Tale inversione di segno, conseguenza della geometria meccanica del prototipo, vincolerà la successiva sintesi del controllore e la sua integrazione nel *mixer* di controllo del firmware.

5.5 Il regolatore PID: principi di funzionamento

Prima di descrivere la procedura di sintesi automatica, è opportuno richiamare il principio di funzionamento del regolatore impiegato. Il *DPDF* è un sistema intrinsecamente instabile attorno alla condizione di assetto orizzontale: in assenza di azione correttiva, il momento ribaltante indotto dall'eccentricità del baricentro (Sezione 7.1.3) fa divergere gli angoli di *roll* e *pitch*. Il compito del controllore è chiudere un anello di retroazione che, confrontando istante per istante l'assetto misurato dall'IMU con il riferimento desiderato, generi un comando ai *flap* tale da riportare il velivolo verso l'equilibrio.

Si definisce a tal fine il segnale di **errore** come differenza tra l'angolo di riferimento θ_{rif} e l'angolo misurato $\theta(t)$:

$$e(t) = \theta_{\text{rif}} - \theta(t) \quad (5.7)$$

Nelle prove di stabilizzazione il riferimento è nullo ($\theta_{\text{rif}} = 0^\circ$), poiché l'obiettivo è mantenere il drone in assetto orizzontale; l'errore coincide dunque, a meno del segno, con l'inclinazione istantanea del velivolo. A partire da questo errore, il regolatore PID elabora il comando di controllo combinando tre azioni distinte, ciascuna con un significato fisico preciso.

5.5.1 Le tre azioni di controllo

Nella sua forma ideale a tempo continuo, l'uscita di un regolatore PID è la somma di un termine proporzionale, uno integrale e uno derivativo dell'errore:

$$u(t) = \underbrace{K_p e(t)}_{\text{proporzionale}} + \underbrace{K_i \int_0^t e(\tau) d\tau}_{\text{integrale}} + \underbrace{K_d \frac{de(t)}{dt}}_{\text{derivativo}} \quad (5.8)$$

I tre guadagni K_p , K_i , K_d pesano il contributo di ciascuna azione e costituiscono i parametri da determinare in fase di taratura.

Azione proporzionale. Il termine $K_p e(t)$ produce una correzione istantanea proporzionale all'entità dell'errore: quanto più il drone è inclinato, tanto più ampia è la deflessione richiesta ai *flap*. È l'azione di richiamo principale, ma da sola non è sufficiente. In presenza di un disturbo costante — come il momento ribaltante gravitazionale del baricentro eccentrico — l'azione proporzionale lascia infatti un **errore residuo a regime** (*steady-state error*): per generare un momento correttivo permanente che bilanci il disturbo costante, è necessario un errore non nullo che alimenti il termine proporzionale.

Azione integrale. Il termine $K_i \int e d\tau$ accumula nel tempo l'errore e ne integra la storia passata. La sua funzione è proprio quella di **annullare l'errore a regime**: finché permane un errore, anche piccolo, l'integrale continua a crescere aumentando la correzione, fino a quando il momento correttivo eguaglia esattamente il disturbo costante e l'errore si azzerà. Nel caso del *DPDF* questa azione è essenziale per compensare lo sbilanciamento statico del baricentro. Il prezzo da pagare è il rischio di *windup*: se l'attuatore satura (i *flap* raggiungono il limite di $\pm 30^\circ$) e l'errore persiste, l'integrale può accumulare valori molto elevati, generando sovralongazioni e ritardi di recupero. Per questo motivo l'azione integrale è dotata di un meccanismo **anti-windup** a saturazione, descritto nel seguito.

Azione derivativa. Il termine $K_d \dot{e}(t)$ reagisce alla *velocità* di variazione dell'errore, anticipando la dinamica del sistema. Esso introduce uno **smorzamento** che frena le oscillazioni e riduce la sovralongazione, opponendosi alle variazioni rapide di assetto. La derivazione pura è però estremamente sensibile al rumore di misura: la derivata del rumore del BNO-055 produrrebbe comandi ad alta frequenza e ampiezza ai servomotori, con conseguente *chattering*. Per questa ragione, nell'implementazione effettiva (Sezione 5.6) l'azione derivativa è asservita a un filtro passa-basso del primo ordine, dando luogo alla forma PIDF di equazione (5.13).

5.5.2 Implementazione discreta in forma posizionale

Il regolatore non opera nel continuo ma viene eseguito nel firmware come algoritmo a tempo discreto, all'interno dell'*interrupt* deterministico del *Timer 7* alla frequenza di 50 [Hz] ($T_s = 0.02$ [s]). L'implementazione adottata è in **forma posizionale** (*positional form*): a ogni passo di campionamento k l'azione di controllo viene *ricalcolata per intero* a partire dall'errore corrente, dall'accumulatore integrale e dalla derivata filtrata. Questa scelta si distingue dalla forma *incrementale* (*velocity form*), nella quale si calcola invece l'incremento $\Delta u[k]$ rispetto al comando precedente; la forma posizionale è stata preferita per la maggiore trasparenza nella gestione esplicita della saturazione dell'attuatore e del termine anti-windup.

Le tre azioni sono discretizzate come segue. Il termine proporzionale è immediato:

$$P[k] = K_p e[k] \quad (5.9)$$

L'azione integrale è realizzata mediante un accumulatore aggiornato a ogni passo e limitato (*clamping*) entro l'escursione fisica del flap, così da impedire il *windup*:

$$I[k] = \text{sat}\left(I[k-1] + K_i T_s e[k]\right), \quad |I[k]| \leq 30^\circ \quad (5.10)$$

L'azione derivativa filtrata è discretizzata con la trasformazione di Eulero all'indietro introdotta in equazione (5.15), da cui discende il coefficiente $\alpha = T_s/(T_s + T_f)$:

$$D[k] = (1 - \alpha) D[k-1] + \alpha \frac{K_d}{T_s} (e[k] - e[k-1]) \quad (5.11)$$

Il comando complessivo applicato al servomotore è infine la somma delle tre azioni, a sua volta saturata al limite meccanico dei *flap*:

$$u[k] = \text{sat}\left(P[k] + I[k] + D[k]\right), \quad |u[k]| \leq 30^\circ \quad (5.12)$$

Il comando $u[k]$, espresso in gradi di deflessione del *flap*, viene poi convertito nel corrispondente valore CCR tramite la costante di conversione $K_{\text{CCR}} = 10.0$ [CCR/deg] (Sezione 5.2) prima di essere inviato al timer di generazione del segnale PWM. Poiché i guadagni sono stati identificati lavorando in unità fisiche di gradi (Sezione 5.3), i valori K_p , K_i , K_d sintetizzati in MATLAB sono direttamente trasferibili nei campi del file `config.c` senza ulteriori riscalature.

5.6 Sintesi del controllore PIDF

Identificato il modello matematico $G(s)$ del processo, si è proceduto alla sintesi dell'algoritmo di controllo ad anello chiuso. L'architettura scelta è un controllore PID in forma parallela con l'azione derivativa asservita a un filtro passa-basso del primo ordine, configurazione comunemente indicata come **PIDF**.

5.6.1 Architettura del controllore

La presenza del filtro derivativo, caratterizzato dalla costante di tempo T_f , è indispensabile per limitare l'amplificazione del rumore di misura alle alte frequenze: senza filtraggio,

la derivata pura del rumore del BNO-055 produrrebbe comandi ad alta ampiezza e alta frequenza ai servomotori, con conseguente fenomeno di *chattering* (vibrazioni rapide e meccanicamente dannose). La funzione di trasferimento del controllore nel dominio di Laplace ha forma:

$$C(s) = K_p + \frac{K_i}{s} + \frac{K_d s}{T_f s + 1} \quad (5.13)$$

5.6.2 Sintesi tramite pidtune

Il calcolo dei guadagni ottimali è stato effettuato tramite l'algoritmo di ottimizzazione `pidtune` di MATLAB. Per mitigare i rischi di danneggiamento meccanico durante le prime prove sul prototipo, i vincoli di progetto sono stati impostati in modo conservativo:

1. Il parametro `DesignFocus` è stato configurato su `'reference-tracking'` per privilegiare la dolcezza di inseguimento del riferimento ed eliminare sovraelongazioni pronunciate; tale impostazione produce guadagni più contenuti rispetto all'alternativa `'disturbance-rejection'`, più aggressiva ma anche più rischiosa per un primo collaudo.
2. L'azione di controllo $u_{\text{ctrl}}(t)$ è stata costantemente monitorata in simulazione per garantire il rispetto del vincolo di saturazione fisica dei *flap*, fissato a $\pm 30^\circ$:

$$\max |u_{\text{ctrl}}(t)| \leq 30^\circ \quad (5.14)$$

```

1 pid_opts      = pidtuneOptions('DesignFocus', 'reference-tracking');
2 [C_pid, info] = pidtune(sys_asse, 'PIDF', pid_opts);
3
4 % Verifica in anello chiuso e azione di controllo
5 sys_cl = feedback(C_pid * sys_asse, 1); % risposta a gradino
6 sys_ctrl = feedback(C_pid, sys_asse);   % sforzo di controllo
7
```

Listing 5.3: Sintesi del controllore PIDF con pidtune

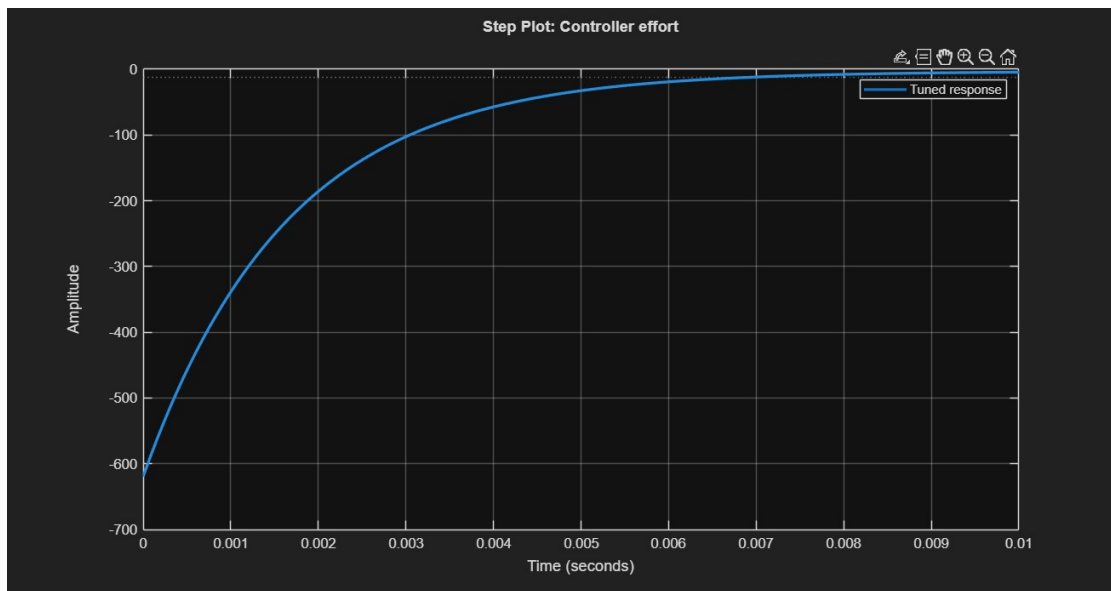
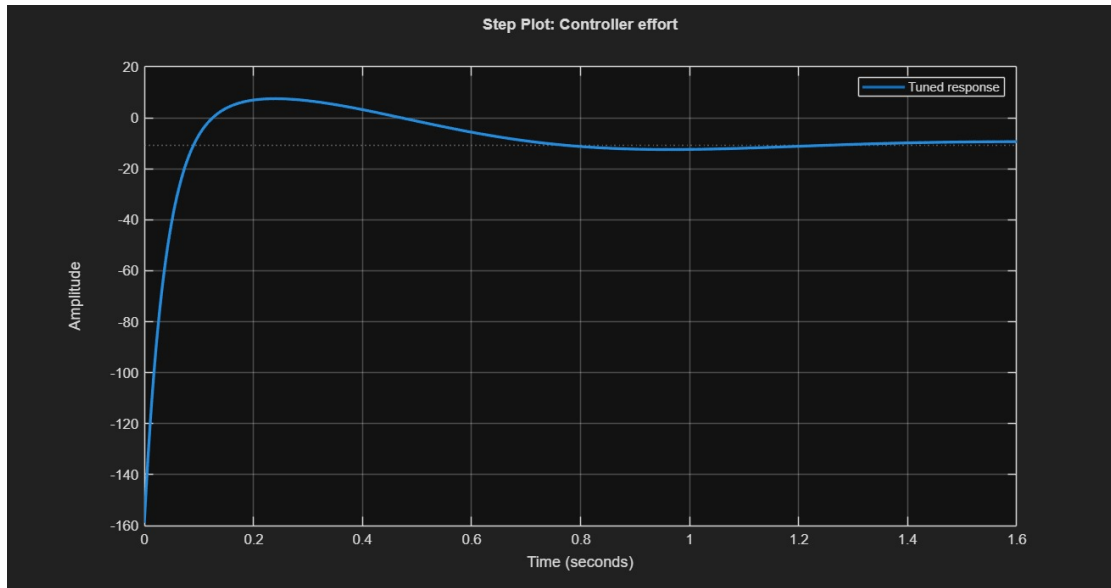


Figura 5.3: *Analisi della risposta al gradino ad anello chiuso (sopra) e del corrispondente sforzo di controllo dei servomotori (sotto). Il picco iniziale dell'azione di controllo si attesta entro il vincolo fisico di $\pm 30^\circ$*

Una nota implementativa importante riguarda la **convenzione di segno** adottata. Il modello identificato presenta guadagno statico negativo ($\mu < 0$, come discusso nella Sezione 5.4). Tuttavia, nel codice sorgente del firmware i guadagni K_p , K_i , K_d vengono inseriti come valori **positivi**: l'inversione di segno necessaria per produrre un momento correttivo opposto al disturbo viene gestita a valle, all'interno della logica di *mixer* che combina l'uscita del PID con il comando applicato al servomotore. Questa scelta separa la legge di controllo

matematica dalla *convenzione di segno* dell'attuatore meccanico, semplificando la lettura del codice firmware.

5.6.3 Discretizzazione per il firmware

Il firmware esegue il *loop* di controllo all'interno di un *interrupt* deterministico generato dal *Timer 7* alla frequenza di 50 [Hz], corrispondente a un periodo di campionamento $T_s = 0.02$ [s]. Per mappare la costante di tempo continua del filtro T_f nel coefficiente adimensionale discreto α utilizzato nell'equazione alle differenze del firmware, è stata applicata la trasformazione di Eulero all'indietro (*Backward Euler approximation*):

$$s \approx \frac{1 - z^{-1}}{T_s} \implies \alpha = \frac{T_s}{T_s + T_f} \quad (5.15)$$

Il coefficiente α è compreso nell'intervallo $[0, 1]$ e determina il peso del filtraggio: un valore prossimo a 1 indica un filtraggio leggero (adatto a segnali poco rumorosi), mentre un valore basso indica un'azione di smorzamento più incisiva. Per il caso in esame i due assi richiedono filtri differenti, come risulterà dalla Tabella 5.2 riportata nella sezione successiva.

5.7 Guadagni finali e parametri PID

I parametri finali, validati in simulazione e pronti per la compilazione nel file `config.c` del firmware, sono riassunti nella Tabella 5.2.

Parametro	Asse di Roll	Asse di Pitch
Modello di partenza	2 poli, 1 zero	3 poli, 1 zero
Guadagno proporzionale K_p	16.49	10.38
Guadagno integrale K_i	43.16	11.45
Guadagno derivativo K_d	1.24	2.32
Coefficiente filtro discreto α	0.27	0.94
Saturazione del canale	$\pm 30^\circ$	$\pm 30^\circ$

Tabella 5.2: Guadagni PIDF identificati per i due assi di controllo

I valori riportati nella Tabella 5.2 si interpretano nel modo seguente. I guadagni proporzionali e integrali dell'asse di *roll* sono significativamente più elevati di quelli dell'asse di *pitch*, conseguenza diretta dei rispettivi modelli identificati: l'asse di *roll* presenta un guadagno statico più piccolo in modulo, dunque richiede un'azione di controllo proporzionalmente più *forte* per riportare il velivolo nella condizione di riferimento. Per quanto riguarda i coefficienti del filtro derivativo, il valore $\alpha = 0.27$ per il *roll* indica un filtraggio aggressivo, necessario per attenuare il rumore di assetto presente su quest'asse, mentre $\alpha = 0.94$ per il *pitch* indica un filtraggio appena percettibile, segno di un canale di misura più pulito.

5.8 Considerazioni sull'approccio black-box

I modelli identificati nel presente capitolo forniscono una descrizione lineare e tempo-invariante della risposta angolare del velivolo, valida *localmente* attorno alla condizione di equilibrio. Questa rappresentazione è perfettamente adeguata allo scopo per cui è stata costruita — la sintesi automatica dei regolatori PID tramite `pdtune` — ed è coerente con l'ipotesi di funzionamento del PID stesso, che opera sulla linearizzazione del sistema attorno al riferimento.

Esistono però aspetti della dinamica del *DPDF* che, per loro natura, non possono essere catturati da un modello LTI SISO. Tra questi: l'accoppiamento tra gli assi di *roll* e *pitch*, particolarmente significativo a grandi inclinazioni; gli effetti delle saturazioni meccaniche dei *flap* e degli ESC; l'interazione non lineare tra il velivolo e le quattro funi di vincolo che lo sostengono nella gabbia di sicurezza; e gli effetti geometrici di grande angolo, in cui le proiezioni della spinta sulle componenti orizzontali e verticali cessano di poter essere approssimate dai loro sviluppi al primo ordine.

Per questa ragione, il modello *black-box* qui presentato è stato affiancato da un **simulatore fisico tridimensionale** a parametri concentrati, descritto nel Capitolo 6, che integra le equazioni di Newton-Eulero a 6 gradi di libertà del velivolo e include esplicitamente la geometria, l'accoppiamento tra assi, le saturazioni e il modello elastico delle funi. I due approcci sono complementari e operano su due piani distinti:

- l'**identificazione *black-box*** (Capitolo 5) produce un modello LTI dedicato alla *sintesi del controllore* con strumenti consolidati come `pdtune`, e si concentra sul singolo grado di libertà alla volta;
- la **simulazione fisica *grey-box*** (Capitolo 6) fornisce un ambiente di *verifica* in cui i regolatori sintetizzati possono essere testati in condizioni più realistiche, includendo gli effetti non catturati dal modello LTI.

I due strumenti si chiudono dunque su un percorso unitario: i parametri PID derivati dall'identificazione vengono inseriti nel simulatore per verifica visiva del comportamento complessivo, e solo dopo questa doppia validazione vengono trasferiti al firmware per il collaudo sul prototipo fisico.

Si segnala infine, per completezza, una piccola discrepanza temporale tra i due ambienti: l'identificazione lavora con periodo di campionamento $T_s = 0.02$ [s] coerente con la frequenza del firmware, mentre il simulatore del Capitolo 6 adotta un passo di visualizzazione $\Delta t = 0.025$ [s] suddiviso in più *substep* di integrazione fisica. La scelta è motivata da ragioni di compromesso tra fluidità della visualizzazione 3D e fedeltà numerica dell'integratore, e non altera la validità dei regolatori sintetizzati, che il simulatore continua a eseguire alla loro frequenza nominale.

6 Simulazione 3D

La validazione sperimentale di un algoritmo di controllo su un prototipo fisico di velivolo comporta intrinsecamente rischi di instabilità e di danneggiamento dell'hardware, specialmente nelle prime fasi di collaudo. Per mitigare tali rischi e per disporre di un ambiente in cui osservare la dinamica complessiva del sistema in condizioni controllate, si è sviluppato un **simulatore tridimensionale** in ambiente MATLAB (`Sim.m`), concepito come un *gemello digitale* del banco di prova di laboratorio.

A differenza dei modelli LTI identificati nel Capitolo 5 — che descrivono la dinamica locale di un singolo asse di assetto attorno al punto di equilibrio — il simulatore adotta un approccio a parametri concentrati di tipo *grey-box*: integra direttamente le equazioni di Newton-Eulero a 6 gradi di libertà del corpo rigido, include esplicitamente la geometria del prototipo e il modello visco-elastico delle quattro funi di vincolo, e incorpora un modello di spinta calibrato sui dati di telemetria reali. L'obiettivo non è dunque sostituire l'identificazione *black-box* (che resta lo strumento naturale per la sintesi automatica dei regolatori tramite `pidtune`), ma fornire un ambiente di verifica che includa fenomeni — accoppiamenti tra assi, momenti ribaltanti, vincolo elastico, saturazioni — non catturabili da un modello SISO lineare.

6.1 Obiettivi e architettura del simulatore

Il simulatore è stato sviluppato con quattro obiettivi principali:

1. **Verifica dei regolatori PIDF** sintetizzati nel Capitolo 5, in un ambiente che include anche le dinamiche non lineari trascurate dal modello LTI;
2. **Comprensione qualitativa** del comportamento del sistema nella configurazione vincolata adottata in laboratorio;
3. **Calibrazione iterativa** dei parametri fisici incerti (rigidezza delle funi, smorzamento aerodinamico, autorità dei *flap*) tramite confronto qualitativo con la telemetria sperimentale;
4. **De-rating** dei guadagni per il primo collaudo sul prototipo fisico, in modo da disporre di valori conservativi sicuramente stabili.

6.1.1 Architettura a doppio loop temporale

Il simulatore implementa l'integrazione numerica delle equazioni del moto mediante un'**architettura a doppio ciclo temporale**. Il ciclo esterno, gestito da un `timer` MATLAB, opera alla frequenza macroscopica di visualizzazione e aggiornamento dello stato del controllore, con periodo $\Delta t = 0.025$ [s]. La fisica dei corpi rigidi e delle forze di vincolo viene invece risolta a una frequenza sub-campionata superiore, secondo il metodo dei *sub-stepping* con fattore $N_{\text{sub}} = 8$, corrispondente a un passo elementare di integrazione $\Delta t_s \approx 0.0031$ [s]. Questa

scelta è necessaria per garantire la stabilità numerica dell'integratore esplicito di Eulero impiegato, in particolare in presenza dei coefficienti di rigidità elevati che caratterizzano il modello delle funi.

Come anticipato nel Capitolo 5, il passo del simulatore (0.025 [s]) differisce leggermente dal periodo di campionamento del firmware ($T_s = 0.02$ [s]) per ragioni di compromesso tra fluidità della visualizzazione 3D e fedeltà numerica dell'integrazione: i regolatori vengono comunque eseguiti dal simulatore mantenendo la loro forma discreta, con effetti del tutto trascurabili sulla validità della verifica.

6.1.2 Interfaccia interattiva

Lo *script* `Sim.m` include una doppia interfaccia grafica. La finestra principale presenta la scena 3D del drone all'interno della gabbia, con visualizzazione in tempo reale di assetto, traiettoria, tensione delle funi e quota; tre grafici laterali mostrano l'evoluzione temporale di quota, angoli di *roll/pitch* e angolo di *yaw*. Una seconda finestra dedicata espone gli *slider* di controllo: due per il CCR dei motori superiore e inferiore (range [750, 1500]), sei per i guadagni dei PID di *roll* e *pitch* (K_p , K_i , K_d per asse), oltre ai pulsanti di START, STOP e RESET. Questa configurazione permette di esplorare interattivamente lo spazio dei parametri, osservando immediatamente l'effetto di ogni modifica sul comportamento del sistema — modalità di lavoro che si è dimostrata particolarmente efficace durante la fase di calibrazione dei parametri fisici.

6.2 Parametrizzazione fisica e geometrica

Il modello integra i parametri geometrici e inerziali del prototipo, derivati dal modello CAD del *DPDF* e verificati sperimentalmente:

- **Massa totale del velivolo:** $M = 2.2$ [kg];
- **Matrice di inerzia (diagonale):** $I_{xx} = 0.0062$ [kg·m²], $I_{yy} = 0.0062$ [kg·m²], $I_{zz} = 0.0100$ [kg·m²];
- **Eccentricità statica del centro di massa** rispetto all'asse geometrico, causata dalle tolleranze di montaggio: $d_{cg,x} = 0.0028$ [m], $d_{cg,y} = 0.0038$ [m];
- **Geometria del cilindro:** altezza $H = 220$ [mm], raggio esterno $R_{ext} = 117.5$ [mm];
- **Geometria della gabbia di vincolo:** cubo di lato $3 \cdot H = 660$ [mm] con quattro funi ai vertici superiori, ancorate al bordo inferiore del drone in corrispondenza degli angoli a 45°, 135°, 225°, 315°.

Tutti questi parametri sono raccolti nella *struct* di configurazione `C`, costruita una sola volta all'avvio del simulatore e successivamente passata per riferimento alle funzioni di integrazione fisica. La medesima *struct* contiene anche le costanti elettriche del sistema propulsivo (tensione batteria, coefficiente di spinta dell'elica, costante motore K_V), utilizzate dal modello di spinta descritto nella Sezione 6.3.2.

6.3 Modello dinamico del corpo rigido

La rotazione del velivolo attorno agli assi di rollio (x) e beccheggio (y) è governata dalle equazioni della dinamica rotazionale in terna solidale, accoppiate alle non-linearità dovute all'eccentricità del baricentro e allo smorzamento aerodinamico del condotto:

$$I_{xx} \cdot \dot{p} = M_{fr} + M_{gr} + M_{x,rope} - C_{d,ang} \cdot p \cdot I_{xx} \quad (6.1)$$

$$I_{yy} \cdot \dot{q} = M_{fp} + M_{gp} + M_{y,rope} - C_{d,ang} \cdot q \cdot I_{yy} \quad (6.2)$$

dove p e q rappresentano le velocità angolari attorno agli assi x e y , $C_{d,ang} = 0.40$ è il coefficiente di smorzamento angolare equivalente, calibrato come descritto nella Sezione 6.5. I termini $M_{x,rope}$ e $M_{y,rope}$ sono i momenti generati dalle quattro funi di vincolo (descritti nella Sezione 6.4), M_{fr} e M_{fp} sono i momenti aerodinamici dei *flap* (Sezione 6.3.2), mentre M_{gr} e M_{gp} modellano il momento ribaltante dovuto alla forza di gravità agente sul baricentro eccentrico.

6.3.1 Momento ribaltante per eccentricità del baricentro

Il momento ribaltante gravitazionale è uno degli elementi che, in assenza di una corretta azione di controllo, porterebbe il velivolo a inclinarsi spontaneamente nella direzione del baricentro eccentrico. Esso è modellato come:

$$M_{gr} = K_g \cdot d_{cg,x} \cdot \cos(\theta) \cdot F_t, \quad M_{gp} = K_g \cdot d_{cg,y} \cdot \cos(\phi) \cdot F_t \quad (6.3)$$

dove θ è l'angolo di *pitch*, ϕ è l'angolo di *roll*, F_t è la spinta totale dei due motori e K_g è un coefficiente di amplificazione geometrica. L'origine della struttura della (6.3) è il classico bilancio dei momenti rispetto al centro geometrico del velivolo, in cui la forza peso applicata nel baricentro reale (offset di d_{cg} dal centro) genera un momento dipendente dall'angolo di inclinazione. Il fattore K_g è stato calibrato empiricamente sulla base del comportamento osservato in simulazione e delle telemetrie reali, per riprodurre l'entità del ribaltamento osservato nei test in cui i *flap* erano mantenuti in posizione neutra.

6.3.2 Momenti aerodinamici dei flap

Il controllo d'assetto effettivo viene esercitato dai momenti aerodinamici puri M_{fr} e M_{fp} , generati dalla deviazione del flusso d'aria operata dall'inclinazione fisica dei *flap*. Indicando con f_r e f_p gli angoli di inclinazione dei *flap* (in gradi, prodotti dal regolatore PID e saturati a $\pm 30^\circ$), i momenti generati sono modellati come:

$$M_{fr} = K_{aer} \cdot f_r \cdot F_t, \quad M_{fp} = K_{aer} \cdot f_p \cdot F_t \quad (6.4)$$

dove $K_{aer} = 0.0025$ [m/deg] è il **braccio aerodinamico equivalente** dei *flap*. Questo coefficiente non deriva da un calcolo analitico CFD, ma è stato **calibrato empiricamente** attraverso il confronto fra le risposte angolari registrate dalla telemetria e le risposte prodotte dal simulatore a parità di ingresso. La forma proporzionale a F_t riflette il fatto che la forza aerodinamica risultante sul *flap* è proporzionale alla pressione dinamica del flusso che lo investe, a sua volta proporzionale alla spinta dei motori.

6.3.3 Modello di spinta calibrato sui dati reali

La spinta verticale totale $F_t = T_u + T_b$ è generata dalla combinazione dei contributi del motore superiore T_u e di quello inferiore T_b , calcolati in funzione del valore CCR di comando, della tensione di batteria $V_{\text{batt}} = 14.8$ [V] e del coefficiente di spinta dell'elica $C_t \approx 0.10$. Il modello adottato segue la formulazione classica della teoria del disco attuatore, ma con un **profilo di throttle** calibrato sui dati di telemetria reali del prototipo:

$$\eta(CCR) = \begin{cases} 0 & \text{se } CCR \leq 1000 \\ 0.08 \cdot \left(\frac{CCR-1000}{150}\right)^2 & \text{se } 1000 < CCR \leq 1150 \\ 0.08 + 0.37 \cdot \min\left(1, \frac{CCR-1150}{350}\right) & \text{se } CCR > 1150 \end{cases} \quad (6.5)$$

Da questo profilo, la spinta del singolo propulsore segue la legge:

$$T = C_t \cdot \rho \cdot D^4 \cdot \left(\frac{\eta \cdot V_{\text{batt}} \cdot K_V}{60}\right)^2 \quad (6.6)$$

dove $D = 0.2286$ [m] è il diametro dell'elica e $K_V = 1400$ [RPM/V] è la costante del motore. La densità dell'aria che investe il motore inferiore è ridotta di un fattore $F_{\text{scia}} = 0.7$ rispetto a quella standard, per modellare l'effetto della scia di rallentamento prodotta dal motore superiore.

Vale la pena evidenziare un aspetto strutturale della calibrazione adottata: con $\eta_{\text{max}} = 0.45$ a $CCR = 1500$, la spinta totale massima del sistema risulta pari a circa $13 \div 14$ [N], significativamente inferiore alla forza peso del velivolo ($F_p = M \cdot g \approx 21.6$ [N]). Il simulatore riproduce quindi fedelmente il deficit di spinta che caratterizza il prototipo fisico: nessuna configurazione dei guadagni o degli ingressi è in grado, nel simulatore così come nella realtà, di produrre il decollo libero del drone. Questa scelta progettuale, voluta e calibrata sui dati reali, conferisce al simulatore la sua principale caratteristica di onestà fisica: non è un simulatore di volo, ma un simulatore del *comportamento dinamico del prototipo nella sua configurazione vincolata*.

6.4 Modello visco-elastico delle funi

Un elemento cruciale per la fedeltà del simulatore è la modellazione del vincolo cinematico a cui il drone è sottoposto durante i test fisici: la sospensione mediante quattro funi all'interno della gabbia di sicurezza. Tali funi, collegate ai vertici superiori della gabbia e ancorate al bordo inferiore del cilindro del drone, sono state modellate secondo una formulazione **visco-elastica monodimensionale** (modello di *Kelvin-Voigt*).

La forza vettoriale $\mathbf{F}_{\text{fune},i}$ esercitata dalla singola fune interviene esclusivamente quando la corda si trova in stato di tensione, ovvero quando la sua lunghezza istantanea $L_i(t)$ supera la lunghezza a riposo $L_{\text{rest},i}$. Detta $\mathbf{u}_i$ la direzione orientata della fune dall'ancoraggio sul drone verso il rispettivo vertice della gabbia, l'espressione della forza è:

$$\mathbf{F}_{\text{fune},i} = \begin{cases} (K_{\text{rope}} \cdot \Delta L_i + D_{\text{rope}} \cdot v_{\text{assiale},i}) \mathbf{u}_i & \text{se } \Delta L_i > 0 \\ \mathbf{0} & \text{se } \Delta L_i \leq 0 \end{cases} \quad (6.7)$$

dove $\Delta L_i = L_i(t) - L_{\text{rest},i}$ rappresenta lo stiramento lineare e $v_{\text{assiale},i} = \mathbf{v}_{\text{punto},i} \cdot (-\mathbf{u}_i)$ è la velocità relativa lungo l'asse della fune nel punto di ancoraggio sul drone, calcolata tenendo

conto sia della velocità lineare del centro di massa sia della velocità angolare del corpo. I coefficienti di rigidezza elastica K_{rope} e di smorzamento viscoso D_{rope} sono stati calibrati come descritto nella Sezione 6.5.

L'implementazione algoritmica, deputata anche al calcolo dei momenti indotti dalle funi sul baricentro del drone tramite prodotto vettoriale, è riportata nello snippet seguente:

```

1 function [Fx, Fy, Fz, Mx, My] = rope_forces(S, C)
2     Fx = 0; Fy = 0; Fz = 0; Mx = 0; My = 0;
3     R    = Rzyx(S.yaw, S.pitch, S.roll);
4     pos   = [S.x; S.y; S.z];
5     vel   = [S.vx; S.vy; S.vz];
6     omega = [S.p_ang; S.q_ang; S.r_ang];
7
8     for ci = 1:4
9         a      = deg2rad(C.ROPE_ANG(ci));
10        p_body  = [C.R_EXT*cos(a); C.R_EXT*sin(a); -C.H_CYL/2];
11        Rp      = R * p_body;
12        p_world = Rp + pos;
13        anchor  = C.CAGE_ANCHORS(ci,:)' ;
14        d_vec   = anchor - p_world;
15        L       = norm(d_vec);
16        if L < 1e-6, continue; end
17        u       = d_vec / L;
18        stretch = L - C.L_REST(ci);
19        if stretch > 0
20            v_pt   = vel + cross(omega, Rp);
21            v_axial = dot(v_pt, -u);
22            F_t     = C.K_ROPE * stretch + C.D_ROPE * (-v_axial);
23            F_t     = min(max(F_t, 0), C.F_ROPE_MAX);
24            F_vec   = F_t * u;
25            Fx = Fx + F_vec(1);
26            Fy = Fy + F_vec(2);
27            Fz = Fz + F_vec(3);
28            M     = cross(Rp, F_vec);
29            Mx = Mx + M(1);
30            My = My + M(2);
31        end
32    end
33 end
34

```

Listing 6.1: Calcolo delle forze e dei momenti generati dalle quattro funi di vincolo

Per ciascuna delle quattro funi, la routine calcola il vettore che congiunge il punto di attacco solidale al corpo con il rispettivo vertice della gabbia (espresso in coordinate *world*), determina la lunghezza istantanea e, se questa supera la lunghezza a riposo, calcola la forza visco-elastica risultante. La forza viene sommata al vettore complessivo delle forze esterne, mentre il prodotto vettoriale $\mathbf{r} \times \mathbf{F}$ contribuisce ai momenti M_x e M_y delle equazioni (6.1) e (6.2). Il limite $F_{\text{ROPE_MAX}}$ rappresenta una saturazione di sicurezza che corrisponde, fisicamente, al carico oltre il quale la fune entrerebbe in regime non lineare o cederebbe.

6.5 Calibrazione empirica dei parametri

Diversi parametri del modello non possono essere derivati analiticamente con sufficiente accuratezza dalle sole specifiche dei componenti, e sono stati pertanto **calibrati empiricamente** attraverso un processo iterativo di confronto tra le risposte del simulatore e i *dataset* di telemetria raccolti nel Capitolo 4. I principali parametri così calibrati sono i seguenti.

- **Coefficiente aerodinamico dei flap** $K_{\text{aer}} = 0.0025$ [m/deg]: calibrato osservando l'ampiezza della risposta angolare a un dato comando di flap nelle prove a gradino. Un valore troppo basso produrrebbe un drone insensibile ai comandi di assetto; un valore troppo alto produrrebbe risposte irrealisticamente vivaci.
- **Coefficiente di amplificazione del momento gravitazionale** K_g : calibrato confrontando il ribaltamento osservato nelle prove a motori accesi e *flap* centrati con quello generato dal simulatore. Tiene conto del fatto che il braccio meccanico effettivo del momento ribaltante è significativamente maggiore della pura eccentricità d_{cg} del baricentro, a causa della geometria distribuita del telaio.
- **Smorzamento angolare aerodinamico** $C_{d,\text{ang}} = 1.50$: calibrato osservando la frequenza di oscillazione naturale del drone nelle prove di rilascio e regolato per riprodurre il rapido smorzamento osservato sperimentalmente.
- **Rigidezza e smorzamento delle funi** $K_{\text{rope}}, D_{\text{rope}}$: calibrati osservando l'entità degli spostamenti residui e l'assenza di oscillazioni divergenti nei test con drone vincolato. La scelta di un coefficiente di rigidezza elevato (K_{rope} dell'ordine di 10^5 [N/m]) impone l'utilizzo del *sub-stepping* descritto nella Sezione 6.1: la frequenza naturale del sistema vincolato cresce con $\sqrt{K_{\text{rope}}/M}$, e per garantire la stabilità dell'integratore di Eulero il passo Δt_s deve essere significativamente più piccolo del periodo associato.

Questo approccio — calibrazione iterativa guidata dalla telemetria piuttosto che da formule analitiche *a priori* — rappresenta una scelta metodologica coerente con l'intero progetto: il drone reale costituisce il riferimento, e il simulatore viene aggiustato per riprodurlo nel modo più fedele possibile entro la struttura del modello adottato.

6.6 Risultati della simulazione

Il comportamento del sistema ad anello chiuso è stato osservato conducendo sessioni di simulazione continuative della durata di 60 [s], a partire dalla configurazione di equilibrio statico del drone appeso alle funi, con i motori al *throttle* di equilibrio ($CCR = 1050$) e i regolatori PID attivi sui due assi di assetto. I risultati di una sessione rappresentativa sono riportati in Figura 6.1.

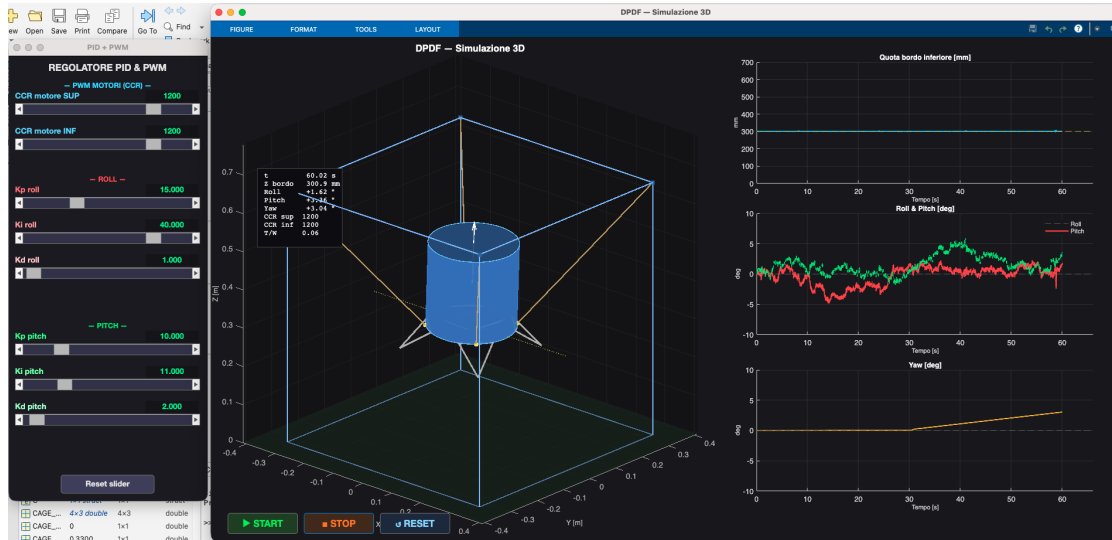


Figura 6.1: Risposta temporale del DPDF nel simulatore (sessione di 60 [s], PID attivo) con parametri PID in configurazione di sicurezza. A $t = 30$ [s] viene applicato un gradino sul CCR da 1050 a 1200. In alto: quota del bordo inferiore del cilindro, che rimane stabile a circa 300 mm indipendentemente dalla variazione di throttle, a causa del vincolo delle funi. Al centro: angoli di roll (linea rossa) e pitch (linea tratteggiata verde), che mostrano un incremento delle oscillazioni in seguito al gradino, evidenziando come l'aumento di spinta amplifichi il momento dovuto all'offset del centro di massa. In basso: angolo di yaw.

I risultati osservati possono essere riassunti nei tre punti seguenti.

- **Il drone permane nella configurazione vincolata**, sostenuto dalle quattro funi che restano in tensione per l'intera durata della prova. La quota del bordo inferiore si stabilizza intorno a 300 [mm] dal pavimento della gabbia, valore corrispondente alla configurazione di equilibrio meccanico in cui la componente verticale della tensione delle funi compensa la differenza tra peso del velivolo e spinta dei motori. Coerentemente con quanto anticipato nella Sezione 6.3.2, la spinta complessiva è insufficiente per produrre il decollo libero.
- **L'azione dei regolatori PID converge correttamente.** Gli angoli di *roll* e *pitch*, dopo un breve transitorio iniziale dovuto al momento ribaltante generato dall'eccentricità del baricentro, vengono riportati e mantenuti in un intorno dello zero per l'intera durata della prova. Le oscillazioni residue sono di ampiezza contenuta (dell'ordine di pochi gradi picco-picco) e non mostrano alcuna tendenza alla divergenza né segni di *chattering* sugli attuatori.
- **L'yaw resta sostanzialmente fermo**, coerentemente con il fatto che il simulatore non include una sollecitazione differenziale tra i due motori. Eventuali piccoli scostamenti sono dovuti alla componente non perfettamente compensata dei momenti torcenti delle due eliche, che il modello include in forma semplificata.

Il simulatore fornisce dunque due indicazioni complementari, entrambe di valore. Da un lato **conferma la validità della legge di controllo PID** sintetizzata nel Capitolo 5,

mostrando che i regolatori sono in grado di stabilizzare l'assetto del velivolo in presenza dei principali disturbi non lineari (eccentricità del baricentro, accoppiamento con il vincolo elastico). Dall'altro **conferma il deficit di spinta** che impedisce il volo libero: nessuna combinazione dei guadagni testati produce il decollo, e questo non è un limite dei regolatori ma una caratteristica intrinseca della catena propulsiva nella configurazione attuale. Le cause di tale deficit e le possibili strade per superarlo sono analizzate in dettaglio nel Capitolo 7.

6.7 De-rating dei guadagni per il transfer al prototipo fisico

I guadagni sintetizzati da **pidtune** sul modello LTI identificato (Tabella 5.2) costituiscono il punto di partenza teorico ottimale. Tuttavia, nel passaggio dal modello al sistema fisico intervengono fattori non modellati che possono favorire l'insorgenza di oscillazioni ad alta frequenza (*chattering*) sugli attuatori: i giochi meccanici dei servomotori *Hitec HS-82MG*, l'attrito statico dei perni dei *flap*, le piccole non-linearità della catena ESC-motore, e il rumore residuo dell'IMU.

Per il primo collaudo sul prototipo fisico si è scelto di adottare una configurazione di sicurezza, ottenuta arrotondando in modo conservativo i guadagni del modello LTI rispetto ai valori sintetizzati nel Capitolo 5. I valori adottati, distinti per i due assi di assetto, sono riportati in Tabella 6.1.

Parametro	Asse di Roll	Asse di Pitch
$K_{p,\text{safe}}$	15.0	10.0
$K_{i,\text{safe}}$	40.0	11.0
$K_{d,\text{safe}}$	1.0	2.0

Tabella 6.1: Guadagni PIDF adottati per il primo collaudo sul prototipo fisico, distinti per i due assi di controllo

L'asimmetria tra i due assi — K_p più elevato e K_i significativamente più aggressivo sul *roll* rispetto al *pitch* — riflette quella già emersa nella sintesi automatica di **pidtune** (Tabella 5.2), ed è imputabile alla differente struttura dei modelli identificati sui due assi: un secondo ordine con uno zero per il *roll* e un terzo ordine con uno zero per il *pitch*, con guadagni statici di modulo differente. L'azione integrale più forte sul *roll* è dunque necessaria per compensare un guadagno statico più piccolo, mentre il valore più elevato di K_d sul *pitch* contribuisce a smorzare la dinamica leggermente più oscillante predetta dal modello del terzo ordine.

Questi valori sono stati prima validati nel simulatore (producendo la risposta mostrata in Figura 6.1) e successivamente impiegati come punto di partenza nel firmware. La pratica dell'arrotondamento conservativo è una scelta metodologica coerente con la prudenza richiesta nella validazione di sistemi non lineari: i guadagni più aggressivi suggeriti da **pidtune** possono essere reintrodotti incrementalmente in fase di taratura fine sul prototipo, una volta acquisita confidenza con il comportamento reale del sistema.

6.8 Limiti del simulatore e relazione con i capitoli successivi

Il simulatore tridimensionale qui descritto è uno strumento di verifica utile e fedele entro le ipotesi di costruzione, ma non è privo di limiti. È opportuno enumerarli esplicitamente, sia per circoscrivere la portata dei risultati ottenuti sia per indicare possibili direzioni di sviluppo.

- **Modello aerodinamico semplificato:** l'azione dei *flap* è descritta da una singola costante K_{aer} , che racchiude in un parametro empirico l'intero comportamento della superficie aerodinamica. Effetti come la dipendenza dall'angolo di incidenza, lo stallo dinamico e l'interazione con la scia delle eliche non sono modellati esplicitamente.
- **Modello di spinta calibrato a posteriori:** il profilo di *throttle* (6.5) riproduce il comportamento osservato della catena propulsiva nella configurazione attuale, ma non costituisce un modello propulsivo predittivo. Una modifica significativa del sistema (per esempio sostituzione di motori o eliche) richiederebbe una nuova calibrazione sperimentale.
- **Geometria del vincolo idealizzata:** le funi sono modellate come elementi monodimensionali visco-elastici puri, senza considerare effetti come l'attrito sull'ancoraggio, la massa propria della corda o eventuali oscillazioni trasversali.
- **Integratore esplicito:** l'utilizzo di un'integrazione di Eulero esplicita, anche se mitigata dal *sub-stepping*, è meno robusta di metodi impliciti più sofisticati nei confronti dei sistemi rigidi (*stiff*). Un'integrazione di tipo Runge-Kutta del quarto ordine o uno schema implicito potrebbero ulteriormente migliorare la fedeltà numerica, in particolare in presenza di alte rigidità delle funi.

Pur con questi limiti, il simulatore ha svolto efficacemente il ruolo di banco di prova *a basso rischio* per i regolatori sintetizzati nel Capitolo 5, e ha contribuito significativamente alla comprensione qualitativa della dinamica del prototipo. La sua principale conferma, ribadita anche dai test reali, è che il **deficit di spinta dell'attuale catena propulsiva preclude il volo libero del velivolo**, indipendentemente dalla qualità del controllo. L'analisi dettagliata delle cause di questa limitazione e le proposte per superarla sono l'oggetto del prossimo capitolo.

7 Conclusioni e Sviluppi Futuri

Il lavoro descritto in questo documento ha condotto allo sviluppo di un'infrastruttura completa per la caratterizzazione e il controllo del prototipo *Double Propeller Ducted-Fan (DPDF)*: l'intera struttura meccanica è stata riprogettata e ristampata in 3D, è stata implementata un'architettura firmware *event-driven* sulla scheda STM32 NUCLEO-H745ZI-Q, è stato realizzato un canale di telemetria bidirezionale a 230 400 [bps] in grado di campionare lo stato cinematico del velivolo a 50 [Hz], sono state identificate le funzioni di trasferimento dei singoli assi di assetto tramite **tfest** ed è stato costruito un simulatore fisico tridimensionale *grey-box* in MATLAB per la verifica dei regolatori in ambiente controllato.

Tutti questi elementi, presi singolarmente e nel loro insieme, costituiscono un risultato tangibile del progetto. Resta tuttavia un dato di fatto che il prototipo, nella configurazione hardware attuale, **non è stato in grado di sostenere il volo libero**: tutte le prove sperimentali sono state condotte con il drone vincolato all'interno della gabbia di sicurezza, e nei tentativi di valutare la spinta in configurazione svincolata il velivolo si è limitato a sollevarsi parzialmente per poi tornare in contatto con il banco di prova. Il presente capitolo è dunque dedicato a un'analisi critica delle cause di questo limite e a una proposta di linee guida per il superamento nelle iterazioni future del progetto. L'obiettivo è duplice: rendere conto in modo trasparente del comportamento osservato, ed estrarre dalle limitazioni incontrate indicazioni concrete per chi si occuperà del seguito del progetto.

7.1 Analisi critica delle cause di mancato volo libero

Le cause del mancato volo libero sono state isolate attraverso il confronto incrociato di tre fonti di evidenza: l'analisi analitica delle caratteristiche fisiche della catena propulsiva, l'osservazione del comportamento del simulatore (Capitolo 6) e i test sul prototipo fisico. Le cinque criticità identificate, presentate di seguito in ordine di importanza decrescente rispetto al volo libero, contribuiscono in maniera congiunta a precludere il sollevamento autonomo del velivolo.

7.1.1 Verifica analitica del deficit di spinta

La prima e più rilevante delle limitazioni del prototipo riguarda il **sistema propulsivo**. In questa sezione si conduce una verifica quantitativa volta a stabilire se la combinazione propulsiva adottata — motore *Turnigy SK3-3536 1400 KV* ed elica *APC 9×4.5* — sia in grado di generare la spinta necessaria al sollevamento del velivolo. L'analisi confronta la spinta massima erogabile dai due motori con la forza peso del sistema, prima attraverso un limite teorico basato sulla potenza disponibile e successivamente attraverso una stima fondata su dati di spinta misurati sperimentalmente per la medesima famiglia di eliche.

Condizione di hovering e forza peso

La condizione necessaria affinché il velivolo possa sollevarsi da terra e mantenere una quota costante impone che la spinta totale generata dai due motori superi la forza peso del sistema:

$$T_{\text{tot}} > F_p = m \cdot g \quad (7.1)$$

Considerando la massa complessiva del velivolo assemblato $m = 2.2$ [kg] e l'accelerazione di gravità $g = 9.81$ [m/s²], la forza peso da vincere risulta:

$$F_p = 2.2 \cdot 9.81 = 21.58 \text{ [N]} \quad (7.2)$$

Tale valore costituisce la soglia minima di spinta che il sistema propulsivo deve essere in grado di erogare per garantire il volo libero.

Limite teorico imposto dalla potenza disponibile

Un primo limite superiore alla spinta ottenibile può essere ricavato dalla teoria del disco attuatore, indipendentemente dal regime di rotazione effettivamente raggiunto dal motore. Per un disco attuatore di area A che elabora una potenza P , la spinta massima teorica in condizioni statiche vale:

$$T_{\text{id}} = (2 \rho A P^2)^{1/3} \quad (7.3)$$

dove $\rho = 1.225$ [kg/m³] è la densità dell'aria e $A = \pi(D/2)^2$ è l'area spazzata dall'elica. Con un diametro $D = 0.2286$ [m] (elica APC 9×4.5) si ottiene:

$$A = \pi \cdot (0.1143)^2 = 0.04105 \text{ [m}^2\text{]} \quad (7.4)$$

Assumendo la potenza massima di targa del motore $P = 590$ [W], la spinta ideale del singolo motore risulta:

$$T_{\text{id}} = (2 \cdot 1.225 \cdot 0.04105 \cdot 590^2)^{1/3} \approx 32.7 \text{ [N]} \quad (7.5)$$

Questo valore rappresenta un limite puramente ideale, ottenuto ipotizzando un'efficienza propulsiva unitaria. Le eliche reali presentano una *Figure of Merit (FM)* tipicamente compresa tra 0.5 e 0.7; assumendo un valore realistico $FM = 0.6$ e tenendo conto della riduzione di efficienza del motore inferiore dovuta al *downwash* (fattore 0.7), la spinta totale teorica si riduce a:

$$T_{\text{tot}}^{\text{teorico}} = FM \cdot T_{\text{id}} \cdot (1 + 0.7) = 0.6 \cdot 32.7 \cdot 1.7 \approx 33.3 \text{ [N]} \quad (7.6)$$

Tale risultato indica che, qualora i motori operassero effettivamente alla potenza di picco di 590 [W], la spinta disponibile sarebbe teoricamente sufficiente al sollevamento. Come si mostrerà nel paragrafo seguente, tuttavia, tale condizione di funzionamento non viene mai raggiunta nelle reali condizioni operative del sistema.

Limite operativo reale e dati sperimentali dell'elica

La potenza di picco di 590 [W] è erogabile dal motore unicamente al regime di rotazione massimo a vuoto, pari a:

$$\text{RPM}_{\text{vuoto}} = K_V \cdot V = 1400 \cdot 14.8 = 20\,720 \text{ [RPM]} \quad (7.7)$$

Una volta calettata l'elica, però, la coppia aerodinamica resistente opposta da quest'ultima riduce drasticamente il regime di rotazione effettivamente raggiungibile. Poiché la potenza assorbita da un'elica scala con il cubo della velocità di rotazione ($P \propto n^3$), il motore si stabilizza, in condizioni statiche, a un regime sensibilmente inferiore a quello a vuoto.

I dati di letteratura specialistica, tra cui quelli raccolti dallo *UIUC Propeller Database* [7] per eliche APC della stessa famiglia (9×4.7 , geometricamente affine alla 9×4.5 impiegata sul prototipo), indicano un regime statico tipico per motori di questa classe nell'intorno di 9 000 [RPM], corrispondente a:

$$n = \frac{9\,000}{60} = 150 \text{ [giri/s]} \quad (7.8)$$

Il medesimo database fornisce inoltre un coefficiente di spinta statico misurato $C_T \approx 0.105$ (definito secondo la convenzione $C_T = T/(\rho n^2 D^4)$). Questo valore è notevolmente inferiore a quello impiegato nei modelli puramente ideali ($C_T \approx 0.2$) ed è decisamente più rappresentativo del comportamento reale dell'elica in condizioni statiche.

La spinta del singolo motore superiore (operante in aria indisturbata) risulta quindi:

$$T_{\text{up}} = C_T \cdot \rho \cdot n^2 \cdot D^4 = 0.105 \cdot 1.225 \cdot 150^2 \cdot 0.2286^4 \approx 7.9 \text{ [N]} \quad (7.9)$$

Applicando il fattore di efficienza dovuto al *downwash* sul motore inferiore (fattore di scia pari a 0.7), la spinta totale erogata dal sistema vale:

$$T_{\text{tot}}^{\text{reale}} = T_{\text{up}} \cdot (1 + 0.7) = 7.9 \cdot 1.7 \approx 13.4 \text{ [N]} \quad (7.10)$$

Confronto e conclusione

Confrontando la spinta totale realisticamente disponibile della relazione (7.10) con la forza peso (7.2) si ottiene:

$$T_{\text{tot}}^{\text{reale}} \approx 13.4 \text{ [N]} < F_p = 21.58 \text{ [N]} \quad (7.11)$$

La spinta disponibile risulta dunque inferiore alla forza peso di circa il 38%, da cui si conclude analiticamente che **il velivolo, nella configurazione propulsiva adottata, non è in grado di sollevarsi dal suolo.**

È opportuno sottolineare che la stima (7.10) costituisce uno scenario ottimistico. Il deficit di spinta reale è verosimilmente superiore, poiché non sono stati considerati gli effetti di ostruzione aerodinamica e l'attrito della struttura del condotto, le perdite meccaniche complessive e le inefficienze degli ESC. La coerenza con le osservazioni sperimentali e con la calibrazione del simulatore (Capitolo 6, in cui la spinta massima del modello è stata fissata a circa $13 \div 14$ [N] proprio sulla base delle misurazioni reali) conferma la correttezza dell'analisi.

La causa fisica del fenomeno risiede nel netto **sottodimensionamento dell'accoppiamento motore-elica**: la combinazione di un motore ad elevato K_V (1400 [RPM/V]), ottimizzato per alti regimi di rotazione, con un'elica di piccolo diametro (9 pollici) privilegia la velocità di rotazione a discapito della spinta statica erogata. Per ottenere una spinta adeguata al sollevamento di una massa di 2.2 [kg] sarebbe necessario adottare eliche di diametro maggiore (nell'intorno di $12 \div 15$ pollici) abbinate a motori a K_V inferiore e coppia più elevata, oppure incrementare il numero di unità propulsive.

7.1.2 Distribuzione delle masse ed elevata inerzia perimetrale

La seconda criticità, indipendente dalla prima ma con essa concorrente, riguarda la distribuzione geometrica delle masse all'interno del prototipo. La scelta progettuale di collocare l'intera infrastruttura hardware — comprendente la scheda di controllo *NUCLEO*, i due convertitori di tensione *DFRobot*, la *Power Distribution Board*, i due ESC *Turnigy Plush 40A* e, soprattutto, il doppio pacco batteria *Tattu LiPo 4S* — lungo la superficie cilindrica esterna del condotto introduce un effetto collaterale meccanico significativo.

Dal punto di vista della fisica dei corpi rigidi, distribuire componenti ad alta densità di massa sul perimetro esterno allontana la materia dall'asse di rotazione geometrico, incrementando i momenti d'inerzia principali del velivolo (I_{xx} e I_{yy}). Un valore elevato di inerzia richiede coppie di controllo più elevate per imprimere le accelerazioni angolari necessarie a stabilizzare un sistema intrinsecamente instabile come il *ducted-fan*. Questo effetto, da solo, non sarebbe sufficiente a precludere il volo libero in presenza di una catena propulsiva adeguata, ma in combinazione con le altre criticità contribuisce a rendere il problema di controllo molto più impegnativo del necessario.

7.1.3 Eccentricità del baricentro e momento ribaltante gravitazionale

I test di identificazione hanno evidenziato la presenza di accoppiamenti dinamici (*cross-coupling*) e sbilanciamenti statici costanti, riconducibili a un disallineamento del centro di gravità rispetto all'asse di spinta teorico pari a $d_{cg,x} = 2.8$ [mm] e $d_{cg,y} = 3.8$ [mm]. Sebbene tali valori possano apparire trascurabili in scala assoluta, in un velivolo intubato a singolo condotto essi generano un **momento ribaltante destabilizzante** non appena i motori principali erogano la spinta verticale F_t :

$$M_{\text{gravity}} = F_t \cdot d_{cg} \quad (7.12)$$

Poiché il baricentro si trova geometricamente al di sopra del punto di applicazione delle forze aerodinamiche correttive dei *flap*, il sistema si comporta come un pendolo inverso accelerato dalla gravità, la cui tendenza al ribaltamento cresce linearmente con la spinta richiesta. In altre parole, paradossalmente, maggiore è la potenza fornita ai motori, maggiore è anche il momento destabilizzante che il sistema di controllo deve compensare.

7.1.4 Deficit di autorità di controllo dei flap

Una terza criticità riguarda l'**autorità di controllo** (*control authority*) offerta dai due *flap* direzionali posti alla base del condotto. A differenza dei multirotori convenzionali, che generano momenti di ripristino imponenti variando differenzialmente la velocità dei rotori posti alle estremità di bracci meccanici estesi, il DPDF si affida unicamente alla deviazione del getto d'aria discendente.

Il momento correttivo massimo generabile dai *flap* è strutturalmente limitato dal piccolo braccio aerodinamico equivalente $K_{\text{aer}} = 0.0025$ [m/deg] e dalla superficie ridotta delle alette stesse, secondo la legge:

$$\max|M_{\text{flap}}| = \max|f_{\text{flap}}| \cdot K_{\text{aer}} \cdot F_t \quad (7.13)$$

Sotto l'effetto della saturazione fisica imposta a $\pm 30^\circ$ per preservare l'integrità dei servomotori *Hitec HS-82MG*, l'azione correttiva massima erogabile dai *flap* risulta dello stesso ordine di

grandezza — e in alcune condizioni inferiore — del momento destabilizzante indotto dal disallineamento del baricentro. Il sistema di attuazione si trova quindi a operare costantemente in prossimità della saturazione geometrica, con margine di controllo ridotto per fronteggiare disturbi aerodinamici aggiuntivi.

7.1.5 L'effetto mascheramento del vincolo

Un'osservazione metodologica importante riguarda il **motivo per cui il drone è apparso stabile** sia nel simulatore sia nei test in laboratorio, in contrasto con la previsione di mancato volo libero. All'interno della gabbia di sicurezza, le quattro funi disposte ai vertici superiori agiscono come elementi visco-elastici stabilizzanti, secondo il modello di Kelvin-Voigt descritto nel Capitolo 6.

Quando il drone tende a inclinarsi per via del baricentro eccentrico o di un disturbo aerodinamico, le funi del lato opposto si caricano in tensione, esercitando una forza di richiamo:

$$F_{\text{fune}} = K_{\text{rope}} \cdot \Delta L + D_{\text{rope}} \cdot v_{\text{assiale}} \quad (7.14)$$

Questa forza, applicata in un punto distante dal centro di massa del drone, genera un **momento stabilizzante esogeno** che si somma all'azione correttiva dei *flap*. Il vincolo meccanico esterno, pensato come misura di sicurezza, ha così *mascherato* il problema di stabilità: nei test vincolati la combinazione di PID + funi è sufficiente a mantenere il drone in posizione, mentre nel volo libero verrebbe a mancare proprio l'energia di ripristino delle funi — la più potente delle due componenti — lasciando il sistema in balia di coppie aerodinamiche non sufficientemente compensate.

Questa osservazione ha un'importanza non solo descrittiva ma anche *metodologica*, per gli sviluppi futuri: il banco di prova vincolato è uno strumento utile e necessario per la fase di taratura iniziale e per le prove di sicurezza, ma non costituisce di per sé una prova della stabilità del velivolo in volo libero. Quest'ultima richiede inevitabilmente un test in configurazione svincolata, a fronte del quale però è imprescindibile aver prima risolto le criticità strutturali identificate nelle sezioni precedenti.

7.2 Sviluppi futuri e linee guida per la riprogettazione

L'analisi delle cause condotta nella Sezione 7.1 suggerisce con chiarezza le direzioni di intervento per le iterazioni future del progetto. Le seguenti linee guida sono presentate in ordine di priorità decrescente, coerentemente con l'impatto atteso sulla capacità di volo libero del velivolo.

7.2.1 Riprogettazione del sistema propulsivo

La priorità assoluta è il superamento del deficit di spinta documentato nella Sezione 7.1.1. La via maestra è la **sostituzione della combinazione motore-elica** con una catena propulsiva dimensionata correttamente per la massa del velivolo:

- **Adozione di eliche di diametro maggiore**, nell'intorno dei $12 \div 15$ pollici. Il coefficiente di spinta statico aumenta con la quarta potenza del diametro (a parità di altre

condizioni), e già un passaggio a un'elica da 12 pollici raddoppierebbe approssimativamente la spinta erogabile a parità di regime di rotazione.

- **Accoppiamento con motori a K_V ridotto** (tipicamente nell'intervallo $700 \div 900$ [RPM/V]), caratterizzati da coppia più elevata e regimi di rotazione contenuti, più adatti al pilotaggio di eliche di grande diametro.
- **In alternativa, aumento del numero di unità propulsive**, ad esempio con una configurazione a quattro motori coassiali (due coppie controrotanti) anziché due. Questa soluzione moltiplicherebbe la spinta totale a parità di tipologia di motore-elica, ma introdurrebbe complessità ulteriore nel *mixer* di controllo.

Indipendentemente dalla scelta specifica, il dimensionamento futuro dovrà essere preceduto da una verifica analitica del tipo riportato nella Sezione 7.1.1, in modo da non incorrere nuovamente in deficit di spinta non riconosciuti in fase di progettazione.

7.2.2 Centatura e accostamento delle masse

Risolto il problema di spinta, la seconda priorità è la riduzione dell'inerzia perimetrale e l'azzeramento dell'eccentricità del baricentro. La direzione di intervento è la **riprogettazione del layout interno** del drone:

- Eliminazione di ogni componente dal perimetro esterno del cilindro;
- Alloggiamento della scheda di controllo, degli ESC e della batteria in un *mozzo centrale* coassiale all'asse di spinta, protetto dal flusso d'aria da un involucro aerodinamico;
- Bilanciamento sperimentale del prototipo finale tramite verifica diretta dei momenti residui, prima delle prove di volo.

Questa modifica ridurrebbe significativamente I_{xx} e I_{yy} (con conseguente aumento della reattività ai comandi di assetto) e abbatterebbe il momento ribaltante gravitazionale che è oggi il principale disturbo statico osservato.

7.2.3 Aumento dell'autorità di controllo

A complemento delle prime due linee guida, si rende necessario incrementare l'autorità correttiva del sistema di controllo. Due strade sono percorribili:

- **Aumento dell'area efficace dei *flap*** o sostituzione con superfici aerodinamiche più estese, eventualmente affiancate a un secondo stadio di *flap* ruotati di 90° per ridurre l'accoppiamento tra i due assi.
- **Abbandono dell'attuazione a *flap* fissi** in favore di un sistema a **vettorizzazione della spinta** (*thrust vectoring*): l'intero blocco motore inferiore viene inclinato dinamicamente tramite un anello cardanico (*gimbal ring layout*). Questa soluzione, già adottata su numerosi velivoli a propulsione intubata, fornisce un'autorità di controllo significativamente superiore a quella delle deflessioni di flusso.

7.2.4 Riduzione del peso strutturale

Infine, la riduzione del peso complessivo del velivolo agisce favorevolmente su tutte le criticità sopra menzionate: riduce la spinta richiesta per il sollevamento, riduce le inerzie, e amplifica i margini operativi del sistema di controllo. Direzioni di intervento:

- Sostituzione del PLA delle stampe 3D con strutture alveolari in fibra di carbonio o filamenti compositi alleggeriti (PLA-CF, PA-CF) per i componenti strutturali principali;
- Verifica della reale necessità del doppio pacco batteria e valutazione di un'unica batteria di pari capacità ma minor peso complessivo (la duplicazione introduce ridondanza ma anche zavorra);
- Razionalizzazione del cablaggio e dell'elettronica accessoria.

Un obiettivo realistico, raggiungibile con questi accorgimenti, è una massa complessiva attorno a $1.5 \div 1.7$ [kg], che ridurrebbe la forza peso da circa 21.6 [N] a circa $14.7 \div 16.7$ [N] e — in combinazione con la riprogettazione propulsiva descritta nella Sezione 7.2.1 — porterebbe il sistema in una regione operativa nettamente più favorevole al volo libero.

7.3 Considerazioni conclusive

Il presente lavoro non ha condotto al volo libero del prototipo, ma ha prodotto un risultato in larga parte indipendente da questo obiettivo: una **piattaforma metodologica completa** per la caratterizzazione, l'identificazione e la simulazione del *DPDF*, fondata su strumenti riutilizzabili e direttamente trasferibili alle future iterazioni del progetto. In particolare, l'infrastruttura di telemetria a 50 [Hz], lo *script* di acquisizione automatizzato, lo *script* di identificazione `Identify_and_tune.m`, il simulatore tridimensionale `Sim.m` e la procedura di sintesi automatica dei regolatori PIDF costituiscono un patrimonio software direttamente impiegabile sulla prossima generazione del prototipo, una volta risolte le criticità hardware identificate.

A questo si aggiunge un risultato di natura analitica non meno significativo: la verifica quantitativa, esposta nella Sezione 7.1.1, secondo cui la configurazione propulsiva attuale è strutturalmente insufficiente per il sollevamento del velivolo. Questa conclusione, raggiunta attraverso un confronto rigoroso tra la spinta teorica e quella reale misurata su eliche della stessa famiglia, fornisce un'indicazione tecnica precisa per la riprogettazione futura, sottraendo all'incertezza il dimensionamento della catena propulsiva.

Riteniamo che la combinazione di questi due esiti — una pipeline di sviluppo metodologicamente solida e un'analisi che spiega in modo trasparente i limiti incontrati — rappresenti un contributo utile per chi proseguirà il lavoro sul *Double Propeller Ducted-Fan*, e costituisca il punto di partenza naturale per un prototipo di nuova generazione che, recependo le indicazioni qui formulate, possa finalmente trasferire al volo libero il sistema di controllo qui sviluppato.

Bibliografia

- [1] STMicroelectronics, *STM32H7 Nucleo-144 boards (MB1363) – User Manual*, 2025. UM2408.
- [2] DFRobot, *Power Module (SKU: DFR0205) – Datasheet*, 2017.
- [3] Turnigy, *Turnigy Manual for Brushless Motor Speed Controller*, n.d.
- [4] Hitec Multiplex Japan, Inc., *HS-82MG Spec Sheet*, 2012.
- [5] Bosch Sensortec, *BNO055 Data Sheet: Intelligent 9-axis Absolute Orientation Sensor*, 2021.
- [6] STMicroelectronics, *VL53L1X Time-of-Flight Ranging Sensor – Datasheet*, 2018.
- [7] J. B. Brandt and M. S. Selig, “UIUC Propeller Data Site.” Online, 2014. Department of Aerospace Engineering.